

Introduction

This Reference manual targets application developers. It provides complete information on how to use the memory and the peripherals of the STM32F411xC/E microcontroller.

STM32F411xC/E is part of the family of microcontrollers with different memory sizes, packages and peripherals.

For ordering information, mechanical and electrical device characteristics refer to the datasheets.

For information on the Arm[®] Cortex[®]-M4 with FPU core, refer to the *Cortex[®]-M4 with FPU Technical Reference Manual*.

Related documents

Available from STMicroelectronics web site (<http://www.st.com>):

- STM32F411xC/E datasheet

For information on the Arm[®]-M4 core with FPU, refer to the *STM32F3 Series, STM32F4 Series, STM32L4 Series and STM32L4+ Series Cortex[®]-M4 programming manual (PM0214)*.

Contents

1	Documentation conventions	34
1.1	List of abbreviations for registers	34
1.2	Glossary	35
1.3	Peripheral availability	35
2	Memory and bus architecture	36
2.1	System architecture	36
2.1.1	I-bus	37
2.1.2	D-bus	37
2.1.3	S-bus	37
2.1.4	DMA memory bus	37
2.1.5	DMA peripheral bus	37
2.1.6	BusMatrix	37
2.1.7	AHB/APB bridges (APB)	37
2.2	Memory organization	38
2.3	Memory map	38
2.3.1	Embedded SRAM	40
2.3.2	Flash memory overview	40
2.3.3	Bit banding	40
2.4	Boot configuration	41
3	Embedded Flash memory interface	43
3.1	Introduction	43
3.2	Main features	43
3.3	Embedded Flash memory in STM32F411xC/E	44
3.4	Read interface	45
3.4.1	Relation between CPU clock frequency and Flash memory read time	45
3.4.2	Adaptive real-time memory accelerator (ART Accelerator™)	46
3.5	Erase and program operations	48
3.5.1	Unlocking the Flash control register	48
3.5.2	Program/erase parallelism	49
3.5.3	Erase	49
3.5.4	Programming	50

3.5.5	Interrupts	51
3.6	Option bytes	51
3.6.1	Description of user option bytes	51
3.6.2	Programming user option bytes	53
3.6.3	Read protection (RDP)	53
3.6.4	Write protections	55
3.6.5	Proprietary code readout protection (PCROP)	56
3.7	One-time programmable bytes	58
3.8	Flash interface registers	59
3.8.1	Flash access control register (FLASH_ACR)	59
3.8.2	Flash key register (FLASH_KEYR)	60
3.8.3	Flash option key register (FLASH_OPTKEYR)	60
3.8.4	Flash status register (FLASH_SR)	61
3.8.5	Flash control register (FLASH_CR)	62
3.8.6	Flash option control register (FLASH_OPTCR)	63
3.8.7	Flash interface register map	66
4	CRC calculation unit	67
4.1	CRC introduction	67
4.2	CRC main features	67
4.3	CRC functional description	67
4.4	CRC registers	68
4.4.1	Data register (CRC_DR)	68
4.4.2	Independent data register (CRC_IDR)	68
4.4.3	Control register (CRC_CR)	69
4.4.4	CRC register map	69
5	Power controller (PWR)	70
5.1	Power supplies	70
5.1.1	Independent A/D converter supply and reference voltage	71
5.1.2	Battery backup domain	71
5.1.3	Voltage regulator	72
5.2	Power supply supervisor	73
5.2.1	Power-on reset (POR)/power-down reset (PDR)	73
5.2.2	Brownout reset (BOR)	74
5.2.3	Programmable voltage detector (PVD)	75

5.3	Low-power modes	75
5.3.1	Slowing down system clocks	77
5.3.2	Peripheral clock gating	77
5.3.3	Sleep mode	78
5.3.4	Stop mode	79
5.3.5	Standby mode	81
5.3.6	Programming the RTC alternate functions to wake up the device from the Stop and Standby modes	83
5.4	Power control registers	86
5.4.1	PWR power control register (PWR_CR)	86
5.4.2	PWR power control/status register (PWR_CSR)	88
5.5	PWR register map	89
6	Reset and clock control (RCC) for STM32F411xC/E	90
6.1	Reset	90
6.1.1	System reset	90
6.1.2	Power reset	91
6.1.3	Backup domain reset	92
6.2	Clocks	92
6.2.1	HSE clock	95
6.2.2	HSI clock	96
6.2.3	PLL configuration	96
6.2.4	LSE clock	97
6.2.5	LSI clock	97
6.2.6	System clock (SYSCLK) selection	97
6.2.7	Clock security system (CSS)	98
6.2.8	RTC/AWU clock	98
6.2.9	Watchdog clock	99
6.2.10	Clock-out capability	99
6.2.11	Internal/external clock measurement using TIM5/TIM11	99
6.3	RCC registers	102
6.3.1	RCC clock control register (RCC_CR)	102
6.3.2	RCC PLL configuration register (RCC_PLLCFGR)	104
6.3.3	RCC clock configuration register (RCC_CFGR)	106
6.3.4	RCC clock interrupt register (RCC_CIR)	109
6.3.5	RCC AHB1 peripheral reset register (RCC_AHB1RSTR)	111
6.3.6	RCC AHB2 peripheral reset register (RCC_AHB2RSTR)	113

6.3.7	RCC APB1 peripheral reset register for (RCC_APB1RSTR)	113
6.3.8	RCC APB2 peripheral reset register (RCC_APB2RSTR)	115
6.3.9	RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)	117
6.3.10	RCC AHB2 peripheral clock enable register (RCC_AHB2ENR)	118
6.3.11	RCC APB1 peripheral clock enable register (RCC_APB1ENR)	118
6.3.12	RCC APB2 peripheral clock enable register (RCC_APB2ENR)	121
6.3.13	RCC AHB1 peripheral clock enable in low power mode register (RCC_AHB1LPENR)	123
6.3.14	RCC AHB2 peripheral clock enable in low power mode register (RCC_AHB2LPENR)	124
6.3.15	RCC APB1 peripheral clock enable in low power mode register (RCC_APB1LPENR)	125
6.3.16	RCC APB2 peripheral clock enabled in low power mode register (RCC_APB2LPENR)	127
6.3.17	RCC Backup domain control register (RCC_BDCR)	129
6.3.18	RCC clock control & status register (RCC_CSR)	130
6.3.19	RCC spread spectrum clock generation register (RCC_SSCGR)	132
6.3.20	RCC PLLI2S configuration register (RCC_PLLI2SCFGR)	133
6.3.21	RCC Dedicated Clocks Configuration Register (RCC_DCKCFGR)	135
6.3.22	RCC register map	136
7	System configuration controller (SYSCFG)	139
7.1	I/O compensation cell	139
7.2	SYSCFG registers	139
7.2.1	SYSCFG memory remap register (SYSCFG_MEMRMP)	139
7.2.2	SYSCFG peripheral mode configuration register (SYSCFG_PMC)	140
7.2.3	SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)	141
7.2.4	SYSCFG external interrupt configuration register 2 (SYSCFG_EXTICR2)	141
7.2.5	SYSCFG external interrupt configuration register 3 (SYSCFG_EXTICR3)	142
7.2.6	SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)	142
7.2.7	Compensation cell control register (SYSCFG_CMPCR)	143
7.2.8	SYSCFG register map	144
8	General-purpose I/Os (GPIO)	145
8.1	GPIO introduction	145

8.2	GPIO main features	145
8.3	GPIO functional description	145
8.3.1	General-purpose I/O (GPIO)	147
8.3.2	I/O pin multiplexer and mapping	148
8.3.3	I/O port control registers	151
8.3.4	I/O port data registers	151
8.3.5	I/O data bitwise handling	151
8.3.6	GPIO locking mechanism	151
8.3.7	I/O alternate function input/output	152
8.3.8	External interrupt/wakeup lines	152
8.3.9	Input configuration	152
8.3.10	Output configuration	153
8.3.11	Alternate function configuration	154
8.3.12	Analog configuration	155
8.3.13	Using the OSC32_IN/OSC32_OUT pins as GPIO PC14/PC15 port pins	155
8.3.14	Using the OSC_IN/OSC_OUT pins as GPIO PH0/PH1 port pins	155
8.3.15	Selection of RTC functions	156
8.4	GPIO registers	157
8.4.1	GPIO port mode register (GPIOx_MODER) (x = A..E and H)	157
8.4.2	GPIO port output type register (GPIOx_OTYPER) (x = A..E and H)	157
8.4.3	GPIO port output speed register (GPIOx_OSPEEDR) (x = A..E and H)	158
8.4.4	GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A..E and H)	158
8.4.5	GPIO port input data register (GPIOx_IDR) (x = A..E and H)	159
8.4.6	GPIO port output data register (GPIOx_ODR) (x = A..E and H)	159
8.4.7	GPIO port bit set/reset register (GPIOx_BSRR) (x = A..E and H)	160
8.4.8	GPIO port configuration lock register (GPIOx_LCKR) (x = A..E and H)	160
8.4.9	GPIO alternate function low register (GPIOx_AFRL) (x = A..E and H)	161
8.4.10	GPIO alternate function high register (GPIOx_AFRH) (x = A..E and H)	162
8.4.11	GPIO register map	163
9	DMA controller (DMA)	165
9.1	DMA introduction	165

9.2	DMA main features	165
9.3	DMA functional description	167
9.3.1	General description	167
9.3.2	DMA transactions	168
9.3.3	Channel selection	169
9.3.4	Arbiter	170
9.3.5	DMA streams	171
9.3.6	Source, destination and transfer modes	171
9.3.7	Pointer incrementation	174
9.3.8	Circular mode	175
9.3.9	Double buffer mode	175
9.3.10	Programmable data width, packing/unpacking, endianness	176
9.3.11	Single and burst transfers	178
9.3.12	FIFO	179
9.3.13	DMA transfer completion	181
9.3.14	DMA transfer suspension	182
9.3.15	Flow controller	183
9.3.16	Summary of the possible DMA configurations	184
9.3.17	Stream configuration procedure	184
9.3.18	Error management	185
9.4	DMA interrupts	186
9.5	DMA registers	187
9.5.1	DMA low interrupt status register (DMA_LISR)	187
9.5.2	DMA high interrupt status register (DMA_HISR)	188
9.5.3	DMA low interrupt flag clear register (DMA_LIFCR)	189
9.5.4	DMA high interrupt flag clear register (DMA_HIFCR)	189
9.5.5	DMA stream x configuration register (DMA_SxCR) (x = 0..7)	190
9.5.6	DMA stream x number of data register (DMA_SxNDTR) (x = 0..7)	193
9.5.7	DMA stream x peripheral address register (DMA_SxPAR) (x = 0..7)	194
9.5.8	DMA stream x memory 0 address register (DMA_SxM0AR) (x = 0..7)	194
9.5.9	DMA stream x memory 1 address register (DMA_SxM1AR) (x = 0..7)	194
9.5.10	DMA stream x FIFO control register (DMA_SxFCR) (x = 0..7)	195
9.5.11	DMA register map	197
10	Interrupts and events	201
10.1	Nested vectored interrupt controller (NVIC)	201
10.1.1	NVIC features	201

10.1.2	SysTick calibration value register	201
10.1.3	Interrupt and exception vectors	201
10.2	External interrupt/event controller (EXTI)	201
10.2.1	EXTI main features	204
10.2.2	EXTI block diagram	205
10.2.3	Wakeup event management	205
10.2.4	Functional description	205
10.2.5	External interrupt/event line mapping	207
10.3	EXTI registers	208
10.3.1	Interrupt mask register (EXTI_IMR)	208
10.3.2	Event mask register (EXTI_EMR)	208
10.3.3	Rising trigger selection register (EXTI_RTSR)	209
10.3.4	Falling trigger selection register (EXTI_FTSR)	209
10.3.5	Software interrupt event register (EXTI_SWIER)	210
10.3.6	Pending register (EXTI_PR)	210
10.3.7	EXTI register map	211
11	Analog-to-digital converter (ADC)	212
11.1	ADC introduction	212
11.2	ADC main features	212
11.3	ADC functional description	213
11.3.1	ADC on-off control	214
11.3.2	ADC clock	214
11.3.3	Channel selection	214
11.3.4	Single conversion mode	215
11.3.5	Continuous conversion mode	215
11.3.6	Timing diagram	216
11.3.7	Analog watchdog	216
11.3.8	Scan mode	217
11.3.9	Injected channel management	217
11.3.10	Discontinuous mode	218
11.4	Data alignment	220
11.5	Channel-wise programmable sampling time	221
11.6	Conversion on external trigger and trigger polarity	221
11.7	Fast conversion mode	223
11.8	Data management	224

11.8.1	Using the DMA	224
11.8.2	Managing a sequence of conversions without using the DMA	224
11.8.3	Conversions without DMA and without overrun detection	225
11.9	Temperature sensor	225
11.10	Battery charge monitoring	226
11.11	ADC interrupts	227
11.12	ADC registers	228
11.12.1	ADC status register (ADC_SR)	228
11.12.2	ADC control register 1 (ADC_CR1)	229
11.12.3	ADC control register 2 (ADC_CR2)	231
11.12.4	ADC sample time register 1 (ADC_SMPR1)	233
11.12.5	ADC sample time register 2 (ADC_SMPR2)	233
11.12.6	ADC injected channel data offset register x (ADC_JOFRx) (x=1..4)	234
11.12.7	ADC watchdog higher threshold register (ADC_HTR)	234
11.12.8	ADC watchdog lower threshold register (ADC_LTR)	235
11.12.9	ADC regular sequence register 1 (ADC_SQR1)	235
11.12.10	ADC regular sequence register 2 (ADC_SQR2)	236
11.12.11	ADC regular sequence register 3 (ADC_SQR3)	236
11.12.12	ADC injected sequence register (ADC_JSQR)	237
11.12.13	ADC injected data register x (ADC_JDRx) (x= 1..4)	238
11.12.14	ADC regular data register (ADC_DR)	238
11.12.15	ADC common control register (ADC_CCR)	238
11.12.16	ADC register map	239
12	Advanced-control timer (TIM1)	242
12.1	TIM1 introduction	242
12.2	TIM1 main features	243
12.3	TIM1 functional description	245
12.3.1	Time-base unit	245
12.3.2	Counter modes	247
12.3.3	Repetition counter	256
12.3.4	Clock selection	258
12.3.5	Capture/compare channels	261
12.3.6	Input capture mode	264
12.3.7	PWM input mode	265
12.3.8	Forced output mode	265

12.3.9	Output compare mode	266
12.3.10	PWM mode	267
12.3.11	Complementary outputs and dead-time insertion	270
12.3.12	Using the break function	272
12.3.13	Clearing the OCxREF signal on an external event	275
12.3.14	6-step PWM generation	276
12.3.15	One-pulse mode	277
12.3.16	Encoder interface mode	278
12.3.17	Timer input XOR function	281
12.3.18	Interfacing with Hall sensors	281
12.3.19	TIMx and external trigger synchronization	283
12.3.20	Timer synchronization	286
12.3.21	Debug mode	286
12.4	TIM1 registers	287
12.4.1	TIM1 control register 1 (TIMx_CR1)	287
12.4.2	TIM1 control register 2 (TIMx_CR2)	288
12.4.3	TIM1 slave mode control register (TIMx_SMCR)	291
12.4.4	TIM1 DMA/interrupt enable register (TIMx_DIER)	293
12.4.5	TIM1 status register (TIMx_SR)	295
12.4.6	TIM1 event generation register (TIMx_EGR)	296
12.4.7	TIM1 capture/compare mode register 1 (TIMx_CCMR1)	298
12.4.8	TIM1 capture/compare mode register 2 (TIMx_CCMR2)	301
12.4.9	TIM1 capture/compare enable register (TIMx_CCER)	302
12.4.10	TIM1 counter (TIMx_CNT)	306
12.4.11	TIM1 prescaler (TIMx_PSC)	306
12.4.12	TIM1 auto-reload register (TIMx_ARR)	306
12.4.13	TIM1 repetition counter register (TIMx_RCR)	307
12.4.14	TIM1 capture/compare register 1 (TIMx_CCR1)	307
12.4.15	TIM1 capture/compare register 2 (TIMx_CCR2)	308
12.4.16	TIM1 capture/compare register 3 (TIMx_CCR3)	308
12.4.17	TIM1 capture/compare register 4 (TIMx_CCR4)	309
12.4.18	TIM1 break and dead-time register (TIMx_BDTR)	309
12.4.19	TIM1 DMA control register (TIMx_DCR)	311
12.4.20	TIM1 DMA address for full transfer (TIMx_DMAR)	312
12.4.21	TIM1 register map	313
13	General-purpose timers (TIM2 to TIM5)	315

13.1	TIM2 to TIM5 introduction	315
13.2	TIM2 to TIM5 main features	315
13.3	TIM2 to TIM5 functional description	316
13.3.1	Time-base unit	316
13.3.2	Counter modes	318
13.3.3	Clock selection	327
13.3.4	Capture/compare channels	330
13.3.5	Input capture mode	332
13.3.6	PWM input mode	333
13.3.7	Forced output mode	334
13.3.8	Output compare mode	334
13.3.9	PWM mode	335
13.3.10	One-pulse mode	338
13.3.11	Clearing the OCxREF signal on an external event	339
13.3.12	Encoder interface mode	340
13.3.13	Timer input XOR function	343
13.3.14	Timers and external trigger synchronization	343
13.3.15	Timer synchronization	346
13.3.16	Debug mode	351
13.4	TIM2 to TIM5 registers	352
13.4.1	TIMx control register 1 (TIMx_CR1)	352
13.4.2	TIMx control register 2 (TIMx_CR2)	354
13.4.3	TIMx slave mode control register (TIMx_SMCR)	355
13.4.4	TIMx DMA/Interrupt enable register (TIMx_DIER)	357
13.4.5	TIMx status register (TIMx_SR)	358
13.4.6	TIMx event generation register (TIMx_EGR)	360
13.4.7	TIMx capture/compare mode register 1 (TIMx_CCMR1)	361
13.4.8	TIMx capture/compare mode register 2 (TIMx_CCMR2)	364
13.4.9	TIMx capture/compare enable register (TIMx_CCER)	365
13.4.10	TIMx counter (TIMx_CNT)	367
13.4.11	TIMx prescaler (TIMx_PSC)	367
13.4.12	TIMx auto-reload register (TIMx_ARR)	367
13.4.13	TIMx capture/compare register 1 (TIMx_CCR1)	368
13.4.14	TIMx capture/compare register 2 (TIMx_CCR2)	368
13.4.15	TIMx capture/compare register 3 (TIMx_CCR3)	369
13.4.16	TIMx capture/compare register 4 (TIMx_CCR4)	369
13.4.17	TIMx DMA control register (TIMx_DCR)	370

13.4.18	TIMx DMA address for full transfer (TIMx_DMAR)	371
13.4.19	TIM2 option register (TIM2_OR)	372
13.4.20	TIM5 option register (TIM5_OR)	372
13.4.21	TIMx register map	373
14	General-purpose timers (TIM9 to TIM11)	375
14.1	TIM9/10/11 introduction	375
14.2	TIM9/10/11 main features	375
14.2.1	TIM9 main features	375
14.2.2	TIM10/TIM11 main features	376
14.3	TIM9 to TIM11 functional description	378
14.3.1	Time-base unit	378
14.3.2	Counter modes	380
14.3.3	Clock selection	383
14.3.4	Capture/compare channels	385
14.3.5	Input capture mode	386
14.3.6	PWM input mode (only for TIM9)	388
14.3.7	Forced output mode	389
14.3.8	Output compare mode	389
14.3.9	PWM mode	390
14.3.10	One-pulse mode	391
14.3.11	TIM9 external trigger synchronization	393
14.3.12	Timer synchronization (TIM9)	396
14.3.13	Debug mode	396
14.4	TIM9 registers	397
14.4.1	TIM9 control register 1 (TIMx_CR1)	397
14.4.2	TIM9 slave mode control register (TIMx_SMCR)	398
14.4.3	TIM9 Interrupt enable register (TIMx_DIER)	399
14.4.4	TIM9 status register (TIMx_SR)	401
14.4.5	TIM9 event generation register (TIMx_EGR)	402
14.4.6	TIM9 capture/compare mode register 1 (TIMx_CCMR1)	403
14.4.7	TIM9 capture/compare enable register (TIMx_CCER)	406
14.4.8	TIM9 counter (TIMx_CNT)	407
14.4.9	TIM9 prescaler (TIMx_PSC)	407
14.4.10	TIM9 auto-reload register (TIMx_ARR)	407
14.4.11	TIM9 capture/compare register 1 (TIMx_CCR1)	408
14.4.12	TIM9 capture/compare register 2 (TIMx_CCR2)	408

14.4.13	TIM9 register map	409
14.5	TIM10/11 registers	411
14.5.1	TIM10/11 control register 1 (TIMx_CR1)	411
14.5.2	TIM10/11 Interrupt enable register (TIMx_DIER)	412
14.5.3	TIM status register (TIMx_SR)	412
14.5.4	TIM event generation register (TIMx_EGR)	413
14.5.5	TIM10/11 capture/compare mode register 1 (TIMx_CCMR1)	413
14.5.6	TIM10/11 capture/compare enable register (TIMx_CCER)	416
14.5.7	TIM10/11 counter (TIMx_CNT)	417
14.5.8	TIM10/11 prescaler (TIMx_PSC)	417
14.5.9	TIM10/11 auto-reload register (TIMx_ARR)	417
14.5.10	TIM10/11 capture/compare register 1 (TIMx_CCR1)	418
14.5.11	TIM11 option register 1 (TIM11_OR)	418
14.5.12	TIM10/11 register map	419
15	Independent watchdog (IWDG)	421
15.1	IWDG introduction	421
15.2	IWDG main features	421
15.3	IWDG functional description	421
15.3.1	Hardware watchdog	421
15.3.2	Register access protection	421
15.3.3	Debug mode	422
15.4	IWDG registers	423
15.4.1	Key register (IWDG_KR)	423
15.4.2	Prescaler register (IWDG_PR)	423
15.4.3	Reload register (IWDG_RLR)	424
15.4.4	Status register (IWDG_SR)	424
15.4.5	IWDG register map	425
16	Window watchdog (WWDG)	426
16.1	WWDG introduction	426
16.2	WWDG main features	426
16.3	WWDG functional description	426
16.4	How to program the watchdog timeout	428
16.5	Debug mode	429
16.6	WWDG registers	430

16.6.1	Control register (WWDG_CR)	430
16.6.2	Configuration register (WWDG_CFR)	431
16.6.3	Status register (WWDG_SR)	431
16.6.4	WWDG register map	432
17	Real-time clock (RTC)	433
17.1	Introduction	433
17.2	RTC main features	434
17.3	RTC functional description	435
17.3.1	Clock and prescalers	435
17.3.2	Real-time clock and calendar	436
17.3.3	Programmable alarms	436
17.3.4	Periodic auto-wakeup	437
17.3.5	RTC initialization and configuration	438
17.3.6	Reading the calendar	439
17.3.7	Resetting the RTC	440
17.3.8	RTC synchronization	441
17.3.9	RTC reference clock detection	441
17.3.10	RTC coarse digital calibration	442
17.3.11	RTC smooth digital calibration	443
17.3.12	Timestamp function	445
17.3.13	Tamper detection	446
17.3.14	Calibration clock output	447
17.3.15	Alarm output	448
17.4	RTC and low-power modes	448
17.5	RTC interrupts	448
17.6	RTC registers	450
17.6.1	RTC time register (RTC_TR)	450
17.6.2	RTC date register (RTC_DR)	451
17.6.3	RTC control register (RTC_CR)	452
17.6.4	RTC initialization and status register (RTC_ISR)	454
17.6.5	RTC prescaler register (RTC_PRER)	456
17.6.6	RTC wakeup timer register (RTC_WUTR)	457
17.6.7	RTC calibration register (RTC_CALIBR)	458
17.6.8	RTC alarm A register (RTC_ALRMAR)	459
17.6.9	RTC alarm B register (RTC_ALRMBR)	460

17.6.10	RTC write protection register (RTC_WPR)	461
17.6.11	RTC sub second register (RTC_SSR)	461
17.6.12	RTC shift control register (RTC_SHIFTR)	462
17.6.13	RTC time stamp time register (RTC_TSTR)	462
17.6.14	RTC time stamp date register (RTC_TSDR)	463
17.6.15	RTC timestamp sub second register (RTC_TSSSR)	464
17.6.16	RTC calibration register (RTC_CALR)	464
17.6.17	RTC tamper and alternate function configuration register (RTC_TAFCR)	466
17.6.18	RTC alarm A sub second register (RTC_ALRMASR)	467
17.6.19	RTC alarm B sub second register (RTC_ALRMBSSR)	468
17.6.20	RTC backup registers (RTC_BKPxR)	470
17.6.21	RTC register map	470
18	Inter-integrated circuit (I2C) interface	472
18.1	I ² C introduction	472
18.2	I ² C main features	472
18.3	I ² C functional description	473
18.3.1	Mode selection	473
18.3.2	I2C slave mode	475
18.3.3	I2C master mode	478
18.3.4	Error conditions	483
18.3.5	Programmable noise filter	484
18.3.6	SDA/SCL line control	485
18.3.7	SMBus	485
18.3.8	DMA requests	488
18.3.9	Packet error checking	489
18.4	I ² C interrupts	490
18.5	I ² C debug mode	492
18.6	I ² C registers	492
18.6.1	I ² C Control register 1 (I2C_CR1)	492
18.6.2	I ² C Control register 2 (I2C_CR2)	494
18.6.3	I ² C Own address register 1 (I2C_OAR1)	496
18.6.4	I ² C Own address register 2 (I2C_OAR2)	496
18.6.5	I ² C Data register (I2C_DR)	497
18.6.6	I ² C Status register 1 (I2C_SR1)	497
18.6.7	I ² C Status register 2 (I2C_SR2)	500

18.6.8	I ² C Clock control register (I2C_CCR)	502
18.6.9	I ² C TRISE register (I2C_TRISE)	503
18.6.10	I ² C FLTR register (I2C_FLTR)	503
18.6.11	I2C register map	504
19	Universal synchronous asynchronous receiver transmitter (USART)	505
19.1	USART introduction	505
19.2	USART main features	505
19.3	USART functional description	506
19.3.1	USART character description	509
19.3.2	Transmitter	510
19.3.3	Receiver	513
19.3.4	Fractional baud rate generation	518
19.3.5	USART receiver tolerance to clock deviation	528
19.3.6	Multiprocessor communication	529
19.3.7	Parity control	531
19.3.8	LIN (local interconnection network) mode	532
19.3.9	USART synchronous mode	534
19.3.10	Single-wire half-duplex communication	536
19.3.11	Smartcard	537
19.3.12	IrDA SIR ENDEC block	539
19.3.13	Continuous communication using DMA	541
19.3.14	Hardware flow control	543
19.4	USART interrupts	546
19.5	USART mode configuration	547
19.6	USART registers	547
19.6.1	Status register (USART_SR)	547
19.6.2	Data register (USART_DR)	550
19.6.3	Baud rate register (USART_BRR)	550
19.6.4	Control register 1 (USART_CR1)	550
19.6.5	Control register 2 (USART_CR2)	553
19.6.6	Control register 3 (USART_CR3)	554
19.6.7	Guard time and prescaler register (USART_GTPR)	556
19.6.8	USART register map	557
20	Serial peripheral interface (SPI)	558

20.1	SPI introduction	558
20.2	SPI and I ² S main features	559
20.2.1	SPI features	559
20.2.2	I ² S features	560
20.3	SPI functional description	561
20.3.1	General description	561
20.3.2	Configuring the SPI in slave mode	564
20.3.3	Configuring the SPI in master mode	567
20.3.4	Configuring the SPI for half-duplex communication	569
20.3.5	Data transmission and reception procedures	570
20.3.6	CRC calculation	576
20.3.7	Status flags	578
20.3.8	Disabling the SPI	579
20.3.9	SPI communication using DMA (direct memory addressing)	580
20.3.10	Error flags	582
20.3.11	SPI interrupts	583
20.4	I ² S functional description	584
20.4.1	I ² S general description	584
20.4.2	I2S full duplex	585
20.4.3	Supported audio protocols	586
20.4.4	Clock generator	592
20.4.5	I ² S master mode	594
20.4.6	I ² S slave mode	596
20.4.7	Status flags	598
20.4.8	Error flags	599
20.4.9	I ² S interrupts	600
20.4.10	DMA features	600
20.5	SPI and I ² S registers	601
20.5.1	SPI control register 1 (SPI_CR1)(not used in I ² S mode)	601
20.5.2	SPI control register 2 (SPI_CR2)	603
20.5.3	SPI status register (SPI_SR)	604
20.5.4	SPI data register (SPI_DR)	605
20.5.5	SPI CRC polynomial register (SPI_CRCPR)(not used in I ² S mode)	605
20.5.6	SPI RX CRC register (SPI_RXCR)(not used in I ² S mode)	606
20.5.7	SPI TX CRC register (SPI_TXCR)(not used in I ² S mode)	607
20.5.8	SPI_I ² S configuration register (SPI_I2SCFGR)	607

20.5.9	SPI_I ² S prescaler register (SPI_I2SPR)	608
20.5.10	SPI register map	610
21	Secure digital input/output interface (SDIO)	611
21.1	SDIO main features	611
21.2	SDIO bus topology	611
21.3	SDIO functional description	613
21.3.1	SDIO adapter	615
21.3.2	SDIO APB2 interface	625
21.4	Card functional description	626
21.4.1	Card identification mode	626
21.4.2	Card reset	626
21.4.3	Operating voltage range validation	626
21.4.4	Card identification process	627
21.4.5	Block write	628
21.4.6	Block read	629
21.4.7	Stream access, stream write and stream read (MultiMediaCard only)	629
21.4.8	Erase: group erase and sector erase	631
21.4.9	Wide bus selection or deselection	631
21.4.10	Protection management	631
21.4.11	Card status register	634
21.4.12	SD status register	637
21.4.13	SD I/O mode	641
21.4.14	Commands and responses	642
21.5	Response formats	646
21.5.1	R1 (normal response command)	646
21.5.2	R1b	646
21.5.3	R2 (CID, CSD register)	646
21.5.4	R3 (OCR register)	647
21.5.5	R4 (Fast I/O)	647
21.5.6	R4b	648
21.5.7	R5 (interrupt request)	648
21.5.8	R6	649
21.6	SDIO I/O card-specific operations	649
21.6.1	SDIO I/O read wait operation by SDIO_D2 signaling	650
21.6.2	SDIO read wait operation by stopping SDIO_CLK	650

21.6.3	SDIO suspend/resume operation	650
21.6.4	SDIO interrupts	650
21.7	CE-ATA specific operations	651
21.7.1	Command completion signal disable	651
21.7.2	Command completion signal enable	651
21.7.3	CE-ATA interrupt	651
21.7.4	Aborting CMD61	651
21.8	HW flow control	652
21.9	SDIO registers	652
21.9.1	SDIO power control register (SDIO_POWER)	652
21.9.2	SDI clock control register (SDIO_CLKCR)	653
21.9.3	SDIO argument register (SDIO_ARG)	654
21.9.4	SDIO command register (SDIO_CMD)	654
21.9.5	SDIO command response register (SDIO_RESPCMD)	655
21.9.6	SDIO response 1..4 register (SDIO_RESPx)	656
21.9.7	SDIO data timer register (SDIO_DTIMER)	656
21.9.8	SDIO data length register (SDIO_DLEN)	657
21.9.9	SDIO data control register (SDIO_DCTRL)	658
21.9.10	SDIO data counter register (SDIO_DCOUNT)	659
21.9.11	SDIO status register (SDIO_STA)	660
21.9.12	SDIO interrupt clear register (SDIO_ICR)	661
21.9.13	SDIO mask register (SDIO_MASK)	663
21.9.14	SDIO FIFO counter register (SDIO_FIFOCNT)	665
21.9.15	SDIO data FIFO register (SDIO_FIFO)	666
21.9.16	SDIO register map	666
22	USB on-the-go full-speed (OTG_FS)	668
22.1	OTG_FS introduction	668
22.2	OTG_FS main features	669
22.2.1	General features	669
22.2.2	Host-mode features	670
22.2.3	Peripheral-mode features	670
22.3	OTG_FS functional description	671
22.3.1	OTG pins	671
22.3.2	OTG full-speed core	671
22.3.3	Full-speed OTG PHY	672

22.4	OTG dual role device (DRD)	673
22.4.1	ID line detection	673
22.4.2	HNP dual role device	673
22.4.3	SRP dual role device	674
22.5	USB peripheral	674
22.5.1	SRP-capable peripheral	675
22.5.2	Peripheral states	675
22.5.3	Peripheral endpoints	676
22.6	USB host	678
22.6.1	SRP-capable host	679
22.6.2	USB host states	679
22.6.3	Host channels	681
22.6.4	Host scheduler	682
22.7	SOF trigger	683
22.7.1	Host SOFs	683
22.7.2	Peripheral SOFs	684
22.8	OTG low-power modes	684
22.9	Dynamic update of the OTG_FS_HFIR register	685
22.10	USB data FIFOs	686
22.11	Peripheral FIFO architecture	687
22.11.1	Peripheral Rx FIFO	687
22.11.2	Peripheral Tx FIFOs	688
22.12	Host FIFO architecture	688
22.12.1	Host Rx FIFO	688
22.12.2	Host Tx FIFOs	689
22.13	FIFO RAM allocation	689
22.13.1	Device mode	689
22.13.2	Host mode	690
22.14	USB system performance	690
22.15	OTG_FS interrupts	691
22.16	OTG_FS control and status registers	693
22.16.1	CSR memory map	694
22.16.2	OTG_FS global registers	699
22.16.3	Host-mode registers	720
22.16.4	Device-mode registers	730

22.16.5	OTG_FS power and clock gating control register (OTG_FS_PCGCCTL)	753
22.16.6	OTG_FS register map	754
22.17	OTG_FS programming model	763
22.17.1	Core initialization	763
22.17.2	Host initialization	764
22.17.3	Device initialization	764
22.17.4	Host programming model	765
22.17.5	Device programming model	781
22.17.6	Operational model	783
22.17.7	Worst case response time	799
22.17.8	OTG programming model	801
23	Debug support (DBG)	807
23.1	Overview	807
23.2	Reference Arm® documentation	808
23.3	SWJ debug port (serial wire and JTAG)	808
23.3.1	Mechanism to select the JTAG-DP or the SW-DP	809
23.4	Pinout and debug port pins	809
23.4.1	SWJ debug port pins	810
23.4.2	Flexible SWJ-DP pin assignment	810
23.4.3	Internal pull-up and pull-down on JTAG pins	811
23.4.4	Using serial wire and releasing the unused debug pins as GPIOs ...	812
23.5	STM32F411xC/E JTAG TAP connection	812
23.6	ID codes and locking mechanism	814
23.6.1	MCU device ID code	814
23.6.2	Boundary scan TAP	814
23.6.3	Cortex®-M4 with FPU TAP	815
23.6.4	Cortex®-M4 with FPU JEDEC-106 ID code	815
23.7	JTAG debug port	815
23.8	SW debug port	817
23.8.1	SW protocol introduction	817
23.8.2	SW protocol sequence	817
23.8.3	SW-DP state machine (reset, idle states, ID code)	818
23.8.4	DP and AP read/write accesses	818
23.8.5	SW-DP registers	819

23.8.6	SW-AP registers	820
23.9	AHB-AP (AHB access port) - valid for both JTAG-DP and SW-DP	820
23.10	Core debug	821
23.11	Capability of the debugger host to connect under system reset	822
23.12	FPB (Flash patch breakpoint)	822
23.13	DWT (data watchpoint trigger)	823
23.14	ITM (instrumentation trace macrocell)	823
23.14.1	General description	823
23.14.2	Time stamp packets, synchronization and overflow packets	823
23.15	ETM (Embedded trace macrocell)	825
23.15.1	ETM general description	825
23.15.2	ETM signal protocol and packet types	825
23.15.3	Main ETM registers	826
23.15.4	ETM configuration example	826
23.16	MCU debug component (DBGMCU)	826
23.16.1	Debug support for low-power modes	826
23.16.2	Debug support for timers, watchdog and I ² C	827
23.16.3	Debug MCU configuration register	827
23.16.4	Debug MCU APB1 freeze register (DBGMCU_APB1_FZ)	829
23.16.5	Debug MCU APB2 Freeze register (DBGMCU_APB2_FZ)	830
23.17	TPIU (trace port interface unit)	831
23.17.1	Introduction	831
23.17.2	TRACE pin assignment	832
23.17.3	TPUI formatter	833
23.17.4	TPUI frame synchronization packets	834
23.17.5	Transmission of the synchronization frame packet	834
23.17.6	Synchronous mode	834
23.17.7	Asynchronous mode	835
23.17.8	TRACECLKIN connection inside the STM32F411xC/E	835
23.17.9	TPIU registers	835
23.17.10	Example of configuration	836
23.18	DBG register map	837
24	Device electronic signature	838
24.1	Unique device ID register (96 bits)	838

	24.2	Flash size	839
25		Revision history	840

List of tables

Table 1.	STM32F411xC/E register boundary addresses	38
Table 2.	Boot modes	41
Table 3.	Memory mapping vs. Boot mode/physical remap in STM32F411xC/E	42
Table 4.	Flash module organization (STM32F411xC/E)	44
Table 5.	Number of wait states according to CPU clock (HCLK) frequency	45
Table 6.	Program/erase parallelism	49
Table 7.	Flash interrupt request	51
Table 8.	Option byte organization	51
Table 9.	Description of the option bytes	52
Table 10.	Access versus read protection level	55
Table 11.	OTP area organization	58
Table 12.	Flash register map and reset values	66
Table 13.	CRC calculation unit register map and reset values	69
Table 14.	Low-power mode summary	77
Table 15.	Sleep-now entry and exit	78
Table 16.	Sleep-on-exit entry and exit	78
Table 17.	Stop operating modes	79
Table 18.	Stop mode entry and exit	81
Table 19.	Standby mode entry and exit	82
Table 20.	PWR - register map and reset values	89
Table 21.	RCC register map and reset values for STM32F411xC/E	136
Table 22.	SYSCFG register map and reset values	144
Table 23.	Port bit configuration table	146
Table 24.	Flexible SWJ-DP pin assignment	149
Table 25.	RTC additional functions	156
Table 26.	GPIO register map and reset values	163
Table 27.	DMA1 request mapping (STM32F411xC/E)	170
Table 28.	DMA2 request mapping (STM32F411xC/E)	170
Table 29.	Source and destination address	171
Table 30.	Source and destination address registers in Double buffer mode (DBM=1)	176
Table 31.	Packing/unpacking & endian behavior (bit PINC = MINC = 1)	177
Table 32.	Restriction on NDT versus PSIZE and MSIZE	178
Table 33.	FIFO threshold configurations	180
Table 34.	Possible DMA configurations	184
Table 35.	DMA interrupt requests	186
Table 36.	DMA register map and reset values	197
Table 37.	Vector table for STM32F411xC/E	201
Table 38.	External interrupt/event controller register map and reset values	211
Table 39.	ADC pins	214
Table 40.	Analog watchdog channel selection	217
Table 41.	Configuring the trigger polarity	221
Table 42.	External trigger for regular channels	222
Table 43.	External trigger for injected channels	223
Table 44.	ADC interrupts	227
Table 45.	ADC global register map	239
Table 46.	ADC register map and reset values for each ADC	240
Table 47.	ADC register map and reset values (common ADC registers)	241
Table 48.	Counting direction versus encoder signals	279

Table 49.	TIMx Internal trigger connection	293
Table 50.	Output control bits for complementary OCx and OCxN channels with break feature.	305
Table 51.	TIM1 register map and reset values	313
Table 52.	Counting direction versus encoder signals	341
Table 53.	TIMx internal trigger connection	357
Table 54.	Output control bit for standard OCx channels.	366
Table 55.	TIM2 to TIM5 register map and reset values	373
Table 56.	TIMx internal trigger connection	399
Table 57.	Output control bit for standard OCx channels.	407
Table 58.	TIM9 register map and reset values	409
Table 59.	Output control bit for standard OCx channels.	416
Table 60.	TIM10/11 register map and reset values	419
Table 61.	Min/max IWDG timeout period (in ms) at 32 kHz (LSI).	422
Table 62.	IWDG register map and reset values	425
Table 63.	Minimum and maximum timeout values at 30 MHz (f_{PCLK1}).	429
Table 64.	WWDG register map and reset values	432
Table 65.	Effect of low-power modes on RTC	448
Table 66.	Interrupt control bits	449
Table 67.	RTC register map and reset values	470
Table 68.	Maximum DNF[3:0] value to be compliant with Thd:dat(max)	484
Table 69.	SMBus vs. I2C	486
Table 70.	I2C Interrupt requests.	490
Table 71.	I2C register map and reset values	504
Table 72.	Noise detection from sampled data	517
Table 73.	Error calculation for programmed baud rates at $f_{PCLK} = 8$ MHz or $f_{PCLK} = 12$ MHz, oversampling by 16.	520
Table 74.	Error calculation for programmed baud rates at $f_{PCLK} = 8$ MHz or $f_{PCLK} = 12$ MHz, oversampling by 8.	521
Table 75.	Error calculation for programmed baud rates at $f_{PCLK} = 16$ MHz or $f_{PCLK} = 24$ MHz, oversampling by 16.	521
Table 76.	Error calculation for programmed baud rates at $f_{PCLK} = 16$ MHz or $f_{PCLK} = 24$ MHz, oversampling by 8.	522
Table 77.	Error calculation for programmed baud rates at $f_{PCLK} = 8$ MHz or $f_{PCLK} = 16$ MHz, oversampling by 16.	523
Table 78.	Error calculation for programmed baud rates at $f_{PCLK} = 8$ MHz or $f_{PCLK} = 16$ MHz, oversampling by 8.	523
Table 79.	Error calculation for programmed baud rates at $f_{PCLK} = 30$ MHz or $f_{PCLK} = 60$ MHz, oversampling by 16.	524
Table 80.	Error calculation for programmed baud rates at $f_{PCLK} = 30$ MHz or $f_{PCLK} = 60$ MHz, oversampling by 8	525
Table 81.	Error calculation for programmed baud rates at $f_{PCLK} = 42$ MHz or $f_{PCLK} = 84$ Hz, oversampling by 16.	526
Table 82.	Error calculation for programmed baud rates at $f_{PCLK} = 42$ MHz or $f_{PCLK} = 84$ MHz, oversampling by 8.	527
Table 83.	USART receiver's tolerance when DIV fraction is 0	528
Table 84.	USART receiver tolerance when DIV_Fraction is different from 0	529
Table 85.	Frame formats	531
Table 86.	USART interrupt requests.	546
Table 87.	USART mode configuration	547
Table 88.	USART register map and reset values	557
Table 89.	SPI interrupt requests	583

Table 90.	Audio frequency precision (for PLLM VCO = 1 MHz or 2 MHz)	594
Table 91.	I ² S interrupt requests	600
Table 92.	SPI register map and reset values	610
Table 93.	SDIO I/O definitions	614
Table 94.	Command format	619
Table 95.	Short response format	620
Table 96.	Long response format	620
Table 97.	Command path status flags	620
Table 98.	Data token format	623
Table 99.	Transmit FIFO status flags	624
Table 100.	Receive FIFO status flags	625
Table 101.	Card status	635
Table 102.	SD status	638
Table 103.	Speed class code field	639
Table 104.	Performance move field	639
Table 105.	AU_SIZE field	640
Table 106.	Maximum AU size	640
Table 107.	Erase size field	640
Table 108.	Erase timeout field	641
Table 109.	Erase offset field	641
Table 110.	Block-oriented write commands	643
Table 111.	Block-oriented write protection commands	644
Table 112.	Erase commands	644
Table 113.	I/O mode commands	645
Table 114.	Lock card	645
Table 115.	Application-specific commands	645
Table 116.	R1 response	646
Table 117.	R2 response	647
Table 118.	R3 response	647
Table 119.	R4 response	647
Table 120.	R4b response	648
Table 121.	R5 response	648
Table 122.	R6 response	649
Table 123.	Response type and SDIO_RESPx registers	656
Table 124.	SDIO register map	666
Table 125.	OTG_FS input/output pins	671
Table 126.	Compatibility of STM32 low power modes with the OTG	684
Table 127.	Core global control and status registers (CSRs)	694
Table 128.	Host-mode control and status registers (CSRs)	695
Table 129.	Device-mode control and status registers	696
Table 130.	Data FIFO (DFIFO) access register map	697
Table 131.	Power and clock gating control and status registers	698
Table 132.	TRDT values	704
Table 133.	Minimum duration for soft disconnect	732
Table 134.	OTG_FS register map and reset values	754
Table 135.	SWJ debug port pins	810
Table 136.	Flexible SWJ-DP pin assignment	810
Table 137.	JTAG debug port data registers	815
Table 138.	32-bit debug port registers addressed through the shifted value A[3:2]	816
Table 139.	Packet request (8-bits)	817
Table 140.	ACK response (3 bits)	818
Table 141.	DATA transfer (33 bits)	818

Table 142.	SW-DP registers	819
Table 143.	Cortex [®] -M4 with FPU AHB-AP registers	820
Table 144.	Core debug registers	821
Table 145.	Main ITM registers	824
Table 146.	Main ETM registers	826
Table 147.	Asynchronous TRACE pin assignment	832
Table 148.	Synchronous TRACE pin assignment	832
Table 149.	Flexible TRACE pin assignment	833
Table 150.	Important TPIU registers	835
Table 151.	DBG register map and reset values	837
Table 152.	Document revision history	840

List of figures

Figure 1.	System architecture	36
Figure 2.	Flash memory interface connection inside system architecture (STM32F411xC/E)	43
Figure 3.	Sequential 32-bit instruction execution	47
Figure 4.	RDP levels	55
Figure 5.	PCROP levels	57
Figure 6.	CRC calculation unit block diagram	67
Figure 7.	Power supply overview	70
Figure 8.	Power-on reset/power-down reset waveform	73
Figure 9.	BOR thresholds	74
Figure 10.	PVD thresholds	75
Figure 11.	Simplified diagram of the reset circuit	91
Figure 12.	Clock tree	93
Figure 13.	HSE/ LSE clock sources	95
Figure 14.	Frequency measurement with TIM5 in Input capture mode	100
Figure 15.	Frequency measurement with TIM11 in Input capture mode	101
Figure 16.	Basic structure of a five-volt tolerant I/O port bit	146
Figure 17.	Selecting an alternate function on STM32F411xC/E	150
Figure 18.	Input floating/pull up/pull down configurations	153
Figure 19.	Output configuration	154
Figure 20.	Alternate function configuration	154
Figure 21.	High impedance-analog configuration	155
Figure 22.	DMA block diagram	167
Figure 23.	System implementation of the two DMA controllers (STM32F411xC/E)	168
Figure 24.	Channel selection	169
Figure 25.	Peripheral-to-memory mode	172
Figure 26.	Memory-to-peripheral mode	173
Figure 27.	Memory-to-memory mode	174
Figure 28.	FIFO structure	179
Figure 29.	External interrupt/event controller block diagram	205
Figure 30.	External interrupt/event GPIO mapping	207
Figure 31.	Single ADC block diagram	213
Figure 32.	Timing diagram	216
Figure 33.	Analog watchdog's guarded area	216
Figure 34.	Injected conversion latency	218
Figure 35.	Right alignment of 12-bit data	220
Figure 36.	Left alignment of 12-bit data	220
Figure 37.	Left alignment of 6-bit data	220
Figure 38.	Temperature sensor and VREFINT channel block diagram	225
Figure 39.	Advanced-control timer block diagram	244
Figure 40.	Counter timing diagram with prescaler division change from 1 to 2	246
Figure 41.	Counter timing diagram with prescaler division change from 1 to 4	246
Figure 42.	Counter timing diagram, internal clock divided by 1	247
Figure 43.	Counter timing diagram, internal clock divided by 2	248
Figure 44.	Counter timing diagram, internal clock divided by 4	248
Figure 45.	Counter timing diagram, internal clock divided by N	248
Figure 46.	Counter timing diagram, update event when ARPE=0 (TIMx_ARR not preloaded)	249

Figure 47.	Counter timing diagram, update event when ARPE=1 (TIMx_ARR preloaded)	249
Figure 48.	Counter timing diagram, internal clock divided by 1	251
Figure 49.	Counter timing diagram, internal clock divided by 2	251
Figure 50.	Counter timing diagram, internal clock divided by 4	252
Figure 51.	Counter timing diagram, internal clock divided by N	252
Figure 52.	Counter timing diagram, update event when repetition counter is not used	253
Figure 53.	Counter timing diagram, internal clock divided by 1, TIMx_ARR = 0x6	254
Figure 54.	Counter timing diagram, internal clock divided by 2	254
Figure 55.	Counter timing diagram, internal clock divided by 4, TIMx_ARR=0x36	255
Figure 56.	Counter timing diagram, internal clock divided by N	255
Figure 57.	Counter timing diagram, update event with ARPE=1 (counter underflow)	256
Figure 58.	Counter timing diagram, Update event with ARPE=1 (counter overflow)	256
Figure 59.	Update rate examples depending on mode and TIMx_RCR register settings	257
Figure 60.	Control circuit in normal mode, internal clock divided by 1	258
Figure 61.	TI2 external clock connection example	259
Figure 62.	Control circuit in external clock mode 1	260
Figure 63.	External trigger input block	260
Figure 64.	Control circuit in external clock mode 2	261
Figure 65.	Capture/compare channel (example: channel 1 input stage)	262
Figure 66.	Capture/compare channel 1 main circuit	262
Figure 67.	Output stage of capture/compare channel (channel 1 to 3)	263
Figure 68.	Output stage of capture/compare channel (channel 4)	263
Figure 69.	PWM input mode timing	265
Figure 70.	Output compare mode, toggle on OC1	267
Figure 71.	Edge-aligned PWM waveforms (ARR=8)	268
Figure 72.	Center-aligned PWM waveforms (ARR=8)	269
Figure 73.	Complementary output with dead-time insertion	271
Figure 74.	Dead-time waveforms with delay greater than the negative pulse	271
Figure 75.	Dead-time waveforms with delay greater than the positive pulse	271
Figure 76.	Output behavior in response to a break	274
Figure 77.	Clearing TIMx_OCxREF	275
Figure 78.	6-step generation, COM example (OSSR=1)	276
Figure 79.	Example of one pulse mode	277
Figure 80.	Example of counter operation in encoder interface mode	280
Figure 81.	Example of encoder interface mode with TI1FP1 polarity inverted	280
Figure 82.	Example of Hall sensor interface	282
Figure 83.	Control circuit in reset mode	283
Figure 84.	Control circuit in gated mode	284
Figure 85.	Control circuit in trigger mode	285
Figure 86.	Control circuit in external clock mode 2 + trigger mode	286
Figure 87.	General-purpose timer block diagram	316
Figure 88.	Counter timing diagram with prescaler division change from 1 to 2	317
Figure 89.	Counter timing diagram with prescaler division change from 1 to 4	318
Figure 90.	Counter timing diagram, internal clock divided by 1	319
Figure 91.	Counter timing diagram, internal clock divided by 2	319
Figure 92.	Counter timing diagram, internal clock divided by 4	319
Figure 93.	Counter timing diagram, internal clock divided by N	320
Figure 94.	Counter timing diagram, Update event when ARPE=0 (TIMx_ARR not preloaded)	320
Figure 95.	Counter timing diagram, Update event when ARPE=1 (TIMx_ARR preloaded)	321
Figure 96.	Counter timing diagram, internal clock divided by 1	322
Figure 97.	Counter timing diagram, internal clock divided by 2	322
Figure 98.	Counter timing diagram, internal clock divided by 4	322

Figure 99.	Counter timing diagram, internal clock divided by N	323
Figure 100.	Counter timing diagram, Update event	323
Figure 101.	Counter timing diagram, internal clock divided by 1, TIMx_ARR=0x6	324
Figure 102.	Counter timing diagram, internal clock divided by 2	325
Figure 103.	Counter timing diagram, internal clock divided by 4, TIMx_ARR=0x36	325
Figure 104.	Counter timing diagram, internal clock divided by N	325
Figure 105.	Counter timing diagram, Update event with ARPE=1 (counter underflow)	326
Figure 106.	Counter timing diagram, Update event with ARPE=1 (counter overflow)	326
Figure 107.	Control circuit in normal mode, internal clock divided by 1	327
Figure 108.	TI2 external clock connection example	328
Figure 109.	Control circuit in external clock mode 1	329
Figure 110.	External trigger input block	329
Figure 111.	Control circuit in external clock mode 2	330
Figure 112.	Capture/compare channel (example: channel 1 input stage)	330
Figure 113.	Capture/compare channel 1 main circuit	331
Figure 114.	Output stage of capture/compare channel (channel 1)	331
Figure 115.	PWM input mode timing	333
Figure 116.	Output compare mode, toggle on OC1	335
Figure 117.	Edge-aligned PWM waveforms (ARR=8)	336
Figure 118.	Center-aligned PWM waveforms (ARR=8)	337
Figure 119.	Example of one-pulse mode	338
Figure 120.	Clearing TIMx_OCxREF	340
Figure 121.	Example of counter operation in encoder interface mode	342
Figure 122.	Example of encoder interface mode with TI1FP1 polarity inverted	342
Figure 123.	Control circuit in reset mode	343
Figure 124.	Control circuit in gated mode	344
Figure 125.	Control circuit in trigger mode	345
Figure 126.	Control circuit in external clock mode 2 + trigger mode	346
Figure 127.	Master/Slave timer example	346
Figure 128.	Gating timer 2 with OC1REF of timer 1	347
Figure 129.	Gating timer 2 with Enable of timer 1	348
Figure 130.	Triggering timer 2 with update of timer 1	349
Figure 131.	Triggering timer 2 with Enable of timer 1	350
Figure 132.	Triggering timer 1 and 2 with timer 1 TI1 input	351
Figure 133.	General-purpose timer block diagram (TIM9)	376
Figure 134.	General-purpose timer block diagram (TIM10/11)	377
Figure 135.	Counter timing diagram with prescaler division change from 1 to 2	379
Figure 136.	Counter timing diagram with prescaler division change from 1 to 4	379
Figure 137.	Counter timing diagram, internal clock divided by 1	380
Figure 138.	Counter timing diagram, internal clock divided by 2	381
Figure 139.	Counter timing diagram, internal clock divided by 4	381
Figure 140.	Counter timing diagram, internal clock divided by N	381
Figure 141.	Counter timing diagram, update event when ARPE=0 (TIMx_ARR not preloaded)	382
Figure 142.	Counter timing diagram, update event when ARPE=1 (TIMx_ARR preloaded)	382
Figure 143.	Control circuit in normal mode, internal clock divided by 1	383
Figure 144.	TI2 external clock connection example	384
Figure 145.	Control circuit in external clock mode 1	384
Figure 146.	Capture/compare channel (example: channel 1 input stage)	385
Figure 147.	Capture/compare channel 1 main circuit	386
Figure 148.	Output stage of capture/compare channel (channel 1)	386
Figure 149.	PWM input mode timing	388
Figure 150.	Output compare mode, toggle on OC1	390

Figure 151. Edge-aligned PWM waveforms (ARR=8)	391
Figure 152. Example of one pulse mode.	392
Figure 153. Control circuit in reset mode	394
Figure 154. Control circuit in gated mode	395
Figure 155. Control circuit in trigger mode	395
Figure 156. Independent watchdog block diagram	422
Figure 157. Watchdog block diagram	427
Figure 158. Window watchdog timing diagram	428
Figure 159. RTC block diagram	435
Figure 160. I2C bus protocol	474
Figure 161. I2C block diagram	475
Figure 162. Transfer sequence diagram for slave transmitter	477
Figure 163. Transfer sequence diagram for slave receiver	478
Figure 164. Transfer sequence diagram for master transmitter	481
Figure 165. Transfer sequence diagram for master receiver	482
Figure 166. I2C interrupt mapping diagram	491
Figure 167. USART block diagram	508
Figure 168. Word length programming	509
Figure 169. Configurable stop bits	511
Figure 170. TC/TXE behavior when transmitting	512
Figure 171. Start bit detection when oversampling by 16 or 8	513
Figure 172. Data sampling when oversampling by 16	516
Figure 173. Data sampling when oversampling by 8	517
Figure 174. Mute mode using Idle line detection	530
Figure 175. Mute mode using address mark detection	530
Figure 176. Break detection in LIN mode (11-bit break length - LBDL bit is set)	533
Figure 177. Break detection in LIN mode vs. Framing error detection	534
Figure 178. USART example of synchronous transmission	535
Figure 179. USART data clock timing diagram (M=0)	535
Figure 180. USART data clock timing diagram (M=1)	536
Figure 181. RX data setup/hold time	536
Figure 182. ISO 7816-3 asynchronous protocol	537
Figure 183. Parity error detection using the 1.5 stop bits	538
Figure 184. IrDA SIR ENDEC- block diagram	540
Figure 185. IrDA data modulation (3/16) -Normal mode	540
Figure 186. Transmission using DMA	542
Figure 187. Reception using DMA	543
Figure 188. Hardware flow control between 2 USARTs	543
Figure 189. RTS flow control	544
Figure 190. CTS flow control	545
Figure 191. USART interrupt mapping diagram	546
Figure 192. SPI block diagram	561
Figure 193. Single master/ single slave application	562
Figure 194. Data clock timing diagram	564
Figure 195. TI mode - Slave mode, single transfer	566
Figure 196. TI mode - Slave mode, continuous transfer	567
Figure 197. TI mode - master mode, single transfer	568
Figure 198. TI mode - master mode, continuous transfer	569
Figure 199. TXE/RXNE/BSY behavior in Master / full-duplex mode (BIDIMODE=0 and RXONLY=0) in case of continuous transfers	572
Figure 200. TXE/RXNE/BSY behavior in Slave / full-duplex mode (BIDIMODE=0, RXONLY=0) in case of continuous transfers	573

Figure 201. TXE/BSY behavior in Master transmit-only mode (BIDIMODE=0 and RXONLY=0) in case of continuous transfers	574
Figure 202. TXE/BSY in Slave transmit-only mode (BIDIMODE=0 and RXONLY=0) in case of continuous transfers	574
Figure 203. RXNE behavior in receive-only mode (BIDIRMODE=0 and RXONLY=1) in case of continuous transfers	575
Figure 204. TXE/BSY behavior when transmitting (BIDIRMODE=0 and RXONLY=0) in case of discontinuous transfers	576
Figure 205. Transmission using DMA	581
Figure 206. Reception using DMA	581
Figure 207. TI mode frame format error detection	583
Figure 208. I ² S block diagram	584
Figure 209. I ² S full duplex block diagram	585
Figure 210. I ² S Philips protocol waveforms (16/32-bit full accuracy, CPOL = 0)	587
Figure 211. I ² S Philips standard waveforms (24-bit frame with CPOL = 0)	587
Figure 212. Transmitting 0x8EAA33	587
Figure 213. Receiving 0x8EAA33	588
Figure 214. I ² S Philips standard (16-bit extended to 32-bit packet frame with CPOL = 0)	588
Figure 215. Example	588
Figure 216. MSB justified 16-bit or 32-bit full-accuracy length with CPOL = 0	589
Figure 217. MSB justified 24-bit frame length with CPOL = 0	589
Figure 218. MSB justified 16-bit extended to 32-bit packet frame with CPOL = 0	589
Figure 219. LSB justified 16-bit or 32-bit full-accuracy with CPOL = 0	590
Figure 220. LSB justified 24-bit frame length with CPOL = 0	590
Figure 221. Operations required to transmit 0x3478AE	590
Figure 222. Operations required to receive 0x3478AE	591
Figure 223. LSB justified 16-bit extended to 32-bit packet frame with CPOL = 0	591
Figure 224. Example of LSB justified 16-bit extended to 32-bit packet frame	591
Figure 225. PCM standard waveforms (16-bit)	592
Figure 226. PCM standard waveforms (16-bit extended to 32-bit packet frame)	592
Figure 227. Audio sampling frequency definition	593
Figure 228. I ² S clock generator architecture	593
Figure 229. SDIO “no response” and “no data” operations	612
Figure 230. SDIO (multiple) block read operation	612
Figure 231. SDIO (multiple) block write operation	612
Figure 232. SDIO sequential read operation	613
Figure 233. SDIO sequential write operation	613
Figure 234. SDIO block diagram	613
Figure 235. SDIO adapter	615
Figure 236. Control unit	616
Figure 237. SDIO adapter command path	617
Figure 238. Command path state machine (CPSM)	618
Figure 239. SDIO command transfer	619
Figure 240. Data path	621
Figure 241. Data path state machine (DPSM)	622
Figure 242. OTG full-speed block diagram	671
Figure 243. OTG A-B device connection	673
Figure 244. USB peripheral-only connection	675
Figure 245. USB host-only connection	679
Figure 246. SOF connectivity	683
Figure 247. Updating OTG_FS_HFIR dynamically	686
Figure 248. Device-mode FIFO address mapping and AHB FIFO access mapping	687

Figure 249. Host-mode FIFO address mapping and AHB FIFO access mapping	688
Figure 250. Interrupt hierarchy	692
Figure 251. CSR memory map	694
Figure 252. Transmit FIFO write task	766
Figure 253. Receive FIFO read task	767
Figure 254. Normal bulk/control OUT/SETUP and bulk/control IN transactions	768
Figure 255. Bulk/control IN transactions	771
Figure 256. Normal interrupt OUT/IN transactions	773
Figure 257. Normal isochronous OUT/IN transactions	778
Figure 258. Receive FIFO packet read	784
Figure 259. Processing a SETUP packet	786
Figure 260. Bulk OUT transaction	792
Figure 261. TRDT max timing case	800
Figure 262. A-device SRP	801
Figure 263. B-device SRP	802
Figure 264. A-device HNP	803
Figure 265. B-device HNP	805
Figure 266. Block diagram of STM32 MCU and Cortex [®] -M4 with FPU-level debug support	807
Figure 267. SWJ debug port	809
Figure 268. JTAG TAP connections	813
Figure 269. TPIU block diagram	831

1 Documentation conventions

The STM32F411xx devices have an Arm^{®(a)} Cortex[®]-M4 with FPU.



1.1 List of abbreviations for registers

The following abbreviations are used in register descriptions:

read/write (rw)	Software can read and write to these bits.
read-only (r)	Software can only read these bits.
write-only (w)	Software can only write to this bit. Reading the bit returns the reset value.
read/clear (rc_w1)	Software can read as well as clear this bit by writing 1. Writing '0' has no effect on the bit value.
read/clear (rc_w0)	Software can read as well as clear this bit by writing 0. Writing '1' has no effect on the bit value.
read/clear by read (rc_r)	Software can read this bit. Reading this bit automatically clears it to '0'. Writing '0' has no effect on the bit value.
read/set (rs)	Software can read as well as set this bit. Writing '0' has no effect on the bit value.
read-only write trigger (rt_w)	Software can read this bit. Writing '0' or '1' triggers an event but has no effect on the bit value.
toggle (t)	Software can only toggle this bit by writing '1'. Writing '0' has no effect.
Reserved (Res.)	Reserved bit, must be kept at reset value.

a. Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.

1.2 Glossary

This section gives a brief definition of acronyms and abbreviations used in this document:

- The CPU core integrates two debug ports:
 - JTAG debug port (JTAG-DP) provides a 5-pin standard interface based on the Joint Test Action Group (JTAG) protocol.
 - SWD debug port (SWD-DP) provides a 2-pin (clock and data) interface based on the Serial Wire Debug (SWD) protocol.

For both the JTAG and SWD protocols, please refer to the Cortex[®]-M4 with FPU Technical Reference Manual
- Word: data/instruction of 32-bit length.
- Half word: data/instruction of 16-bit length.
- Byte: data of 8-bit length.
- Double word: data of 64-bit length.
- IAP (in-application programming): IAP is the ability to reprogram the Flash memory of a microcontroller while the user program is running.
- ICP (in-circuit programming): ICP is the ability to program the Flash memory of a microcontroller using the JTAG protocol, the SWD protocol or the bootloader while the device is mounted on the user application board.
- I-Code: this bus connects the Instruction bus of the CPU core to the Flash instruction interface. Prefetch is performed on this bus.
- D-Code: this bus connects the D-Code bus (literal load and debug access) of the CPU to the Flash data interface.
- Option bytes: product configuration bits stored in the Flash memory.
- OBL: option byte loader.
- AHB: advanced high-performance bus.
- CPU: refers to the Cortex[®]-M4 with FPU core.

1.3 Peripheral availability

For information on the availability and the number of instances of each peripheral, refer to the STM32F411xC/E datasheet.

2 Memory and bus architecture

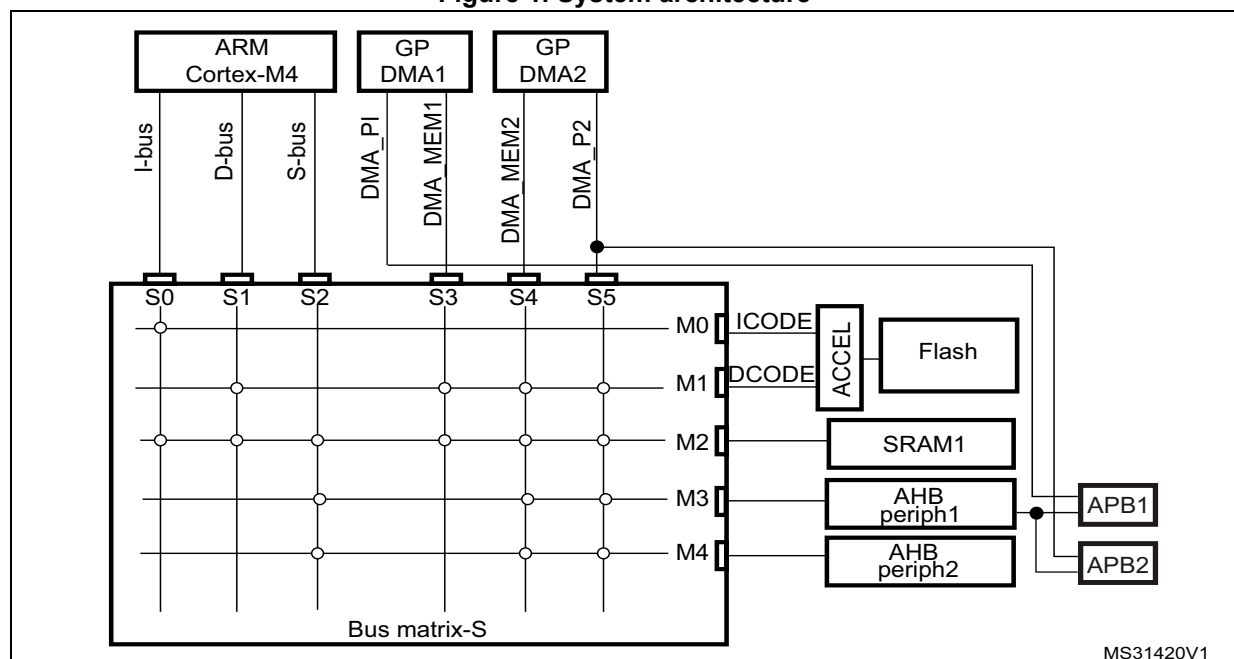
2.1 System architecture

In STM32F411xC/E, the main system consists of 32-bit multilayer AHB bus matrix that interconnects:

- Six masters:
 - Cortex[®]-M4 with FPU core I-bus, D-bus and S-bus
 - DMA1 memory bus
 - DMA2 memory bus
 - DMA2 peripheral bus
- Five slaves:
 - Internal Flash memory ICode bus
 - Internal Flash memory DCode bus
 - Main internal SRAM
 - AHB1 peripherals including AHB to APB bridges and APB peripherals
 - AHB2 peripherals

The bus matrix provides access from a master to a slave, enabling concurrent access and efficient operation even when several high-speed peripherals work simultaneously. This architecture is shown in [Figure 1](#).

Figure 1. System architecture



1. STM32F411xC/E: 256 KBytes / 512KBytes Flash with 128 KBytes SRAM

2.1.1 I-bus

This bus connects the Instruction bus of the Cortex[®]-M4 with FPU core to the BusMatrix. This bus is used by the core to fetch instructions. The target of this bus is a memory containing code (internal Flash memory/SRAM).

2.1.2 D-bus

This bus connects the databus of the Cortex[®]-M4 with FPU to the BusMatrix. This bus is used by the core for literal load and debug access. The target of this bus is a memory containing code or data (internal Flash memory/SRAM).

2.1.3 S-bus

This bus connects the system bus of the Cortex[®]-M4 with FPU core to a BusMatrix. This bus is used to access data located in a peripheral or in SRAM. Instructions may also be fetched on this bus (less efficient than ICode). The targets of this bus are the internal SRAM1, the AHB1 peripherals including the APB peripherals and the AHB2 peripherals.

2.1.4 DMA memory bus

This bus connects the DMA memory bus master interface to the BusMatrix. It is used by the DMA to perform transfer to/from memories. The targets of this bus are data memories: internal Flash memory, internal SRAM and additionally for S4 the AHB1/AHB2 peripherals including the APB peripherals.

2.1.5 DMA peripheral bus

This bus connects the DMA peripheral master bus interface to the BusMatrix. This bus is used by the DMA to access AHB peripherals or to perform memory-to-memory transfers. The targets of this bus are the AHB and APB peripherals plus data memories: Flash memory and internal SRAM.

2.1.6 BusMatrix

The BusMatrix manages the access arbitration between masters. The arbitration uses a round-robin algorithm.

2.1.7 AHB/APB bridges (APB)

The two AHB/APB bridges, APB1 and APB2, provide full synchronous connections between the AHB and the two APB buses, allowing flexible selection of the peripheral frequency.

Refer to the device datasheets for more details on APB1 and APB2 maximum frequencies, and to [Table 1](#) for the address mapping of AHB and APB peripherals.

After each device reset, all peripheral clocks are disabled (except for the SRAM and Flash memory interface). Before using a peripheral you have to enable its clock in the RCC_AHBxENR or RCC_APBxENR register.

Note: When a 16- or an 8-bit access is performed on an APB register, the access is transformed into a 32-bit access: the bridge duplicates the 16- or 8-bit data to feed the 32-bit vector.

2.2 Memory organization

Program memory, data memory, registers and I/O ports are organized within the same linear 4 Gbyte address space.

The bytes are coded in memory in little endian format. The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte, the word's most significant.

For the detailed mapping of peripheral registers, please refer to the related chapters.

The addressable memory space is divided into 8 main blocks, each of 512 MB.

All the memory areas that are not allocated to on-chip memories and peripherals are considered "Reserved"). Refer to the memory map figure in the product datasheet.

2.3 Memory map

See the datasheet corresponding to your device for a comprehensive diagram of the memory map. [Table 1](#) gives the boundary addresses of the peripherals available in STM32F411xC/E device.

Table 1. STM32F411xC/E register boundary addresses

Boundary address	Peripheral	Bus	Register map
0x5000 0000 - 0x5003 FFFF	USB OTG FS	AHB2	Section 22.16.6: OTG_FS register map on page 754
0x4002 6400 - 0x4002 67FF	DMA2	AHB1	Section 9.5.11: DMA register map on page 197
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 3C00 - 0x4002 3FFF	Flash interface register		Section 3.8: Flash interface registers on page 59
0x4002 3800 - 0x4002 3BFF	RCC		Section 6.3.22: RCC register map on page 136
0x4002 3000 - 0x4002 33FF	CRC		Section 4.4.4: CRC register map on page 69
0x4002 1C00 - 0x4002 1FFF	GPIOH		Section 8.4.11: GPIO register map on page 163
0x4002 1000 - 0x4002 13FF	GPIOE		
0x4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		

Table 1. STM32F411xC/E register boundary addresses (continued)

Boundary address	Peripheral	Bus	Register map
0x4001 5000 - 0x4001 53FFF	SPI5/I2S5	APB2	Section 20.5.10: SPI register map on page 610
0x4001 4800 - 0x4001 4BFF	TIM11		Section 14.5.12: TIM10/11 register map on page 419
0x4001 4400 - 0x4001 47FF	TIM10		
0x4001 4000 - 0x4001 43FF	TIM9		Section 14.4.13: TIM9 register map on page 409
0x4001 3C00 - 0x4001 3FFF	EXTI		Section 10.3.7: EXTI register map on page 211
0x4001 3800 - 0x4001 3BFF	SYSCFG		Section 7.2.8: SYSCFG register map
0x4001 3400 - 0x4001 37FF	SPI4/I2S4		Section 20.5.10: SPI register map on page 610
0x4001 3000 - 0x4001 33FF	SPI1/I2S1		
0x4001 2C00 - 0x4001 2FFF	SDIO		Section 21.9.16: SDIO register map on page 666
0x4001 2000 - 0x4001 23FF	ADC1		Section 11.12.16: ADC register map on page 239
0x4001 1400 - 0x4001 17FF	USART6		Section 19.6.8: USART register map on page 557
0x4001 1000 - 0x4001 13FF	USART1		
0x4001 0000 - 0x4001 03FF	TIM1		Section 12.4.21: TIM1 register map on page 313
0x4000 7000 - 0x4000 73FF	PWR	APB1	Section 5.5: PWR register map on page 89
0x4000 5C00 - 0x4000 5FFF	I2C3		
0x4000 5800 - 0x4000 5BFF	I2C2		Section 18.6.11: I2C register map on page 504
0x4000 5400 - 0x4000 57FF	I2C1		
0x4000 4400 - 0x4000 47FF	USART2		Section 19.6.8: USART register map on page 557
0x4000 4000 - 0x4000 43FF	I2S3ext		
0x4000 3C00 - 0x4000 3FFF	SPI3 / I2S3		Section 20.5.10: SPI register map on page 610
0x4000 3800 - 0x4000 3BFF	SPI2 / I2S2		
0x4000 3400 - 0x4000 37FF	I2S2ext		
0x4000 3000 - 0x4000 33FF	IWDG		Section 15.4.5: IWDG register map on page 425
0x4000 2C00 - 0x4000 2FFF	WWDG		Section 16.6.4: WWDG register map on page 432
0x4000 2800 - 0x4000 2BFF	RTC & BKP Registers		Section 17.6.21: RTC register map on page 470
0x4000 0C00 - 0x4000 0FFF	TIM5		
0x4000 0800 - 0x4000 0BFF	TIM4		Section 13.4.21: TIMx register map on page 373
0x4000 0400 - 0x4000 07FF	TIM3		
0x4000 0000 - 0x4000 03FF	TIM2		

2.3.1 Embedded SRAM

STM32F411xC/E devices feature 128 Kbytes of system SRAM.

The embedded SRAM can be accessed as bytes, half-words (16 bits) or full words (32 bits). Read and write operations are performed at CPU speed with 0 wait state.

The CPU can access the embedded SRAM1 through the System Bus or through the I-Code/D-Code buses when boot from SRAM is selected or when physical remap is selected ([Section 7.2.1: SYSCFG memory remap register \(SYSCFG_MEMRMP\)](#) in the SYSCFG controller). To get the max performance on SRAM execution, physical remap should be selected (boot or software selection).

2.3.2 Flash memory overview

The Flash memory interface manages CPU AHB I-Code and D-Code accesses to the Flash memory. It implements the erase and program Flash memory operations and the read and write protection mechanisms. It accelerates code execution with a system of instruction prefetch and cache lines.

The Flash memory is organized as follows:

- A main memory block divided into sectors.
- System memory from which the device boots in System memory boot mode
- 512 OTP (one-time programmable) bytes for user data.
- Option bytes to configure read and write protection, BOR level, watchdog software/hardware and reset when the device is in Standby or Stop mode.

Refer to [Section 3: Embedded Flash memory interface](#) for more details.

2.3.3 Bit banding

The Cortex[®]-M4 with FPU memory map includes two bit-band regions. These regions map each word in an alias region of memory to a bit in a bit-band region of memory. Writing to a word in the alias region has the same effect as a read-modify-write operation on the targeted bit in the bit-band region.

In the STM32F4xx devices both the peripheral registers and the SRAM are mapped to a bit-band region, so that single bit-band write and read operations are allowed. The operations are only available for Cortex[®]-M4 with FPU accesses, and not from other bus masters (e.g. DMA).

A mapping formula shows how to reference each word in the alias region to a corresponding bit in the bit-band region. The mapping formula is:

$$bit_word_addr = bit_band_base + (byte_offset \times 32) + (bit_number \times 4)$$

where:

- *bit_word_addr* is the address of the word in the alias memory region that maps to the targeted bit
- *bit_band_base* is the starting address of the alias region
- *byte_offset* is the number of the byte in the bit-band region that contains the targeted bit
- *bit_number* is the bit position (0-7) of the targeted bit

Example

The following example shows how to map bit 2 of the byte located at SRAM address 0x20000300 to the alias region:

$$0x22006008 = 0x22000000 + (0x300 \times 32) + (2 \times 4)$$

Writing to address 0x22006008 has the same effect as a read-modify-write operation on bit 2 of the byte at SRAM address 0x20000300.

Reading address 0x22006008 returns the value (0x01 or 0x00) of bit 2 of the byte at SRAM address 0x20000300 (0x01: bit set; 0x00: bit reset).

For more information on bit-banding, please refer to the *Cortex®-M4 with FPU programming manual* (see [Related documents on page 1](#)).

2.4 Boot configuration

Due to its fixed memory map, the code area starts from address 0x0000 0000 (accessed through the ICode/DCode buses) while the data area (SRAM) starts from address 0x2000 0000 (accessed through the system bus). The Cortex®-M4 with FPU CPU always fetches the reset vector on the ICode bus, which implies to have the boot space available only in the code area (typically, Flash memory). STM32F4xx microcontrollers implement a special mechanism to be able to boot from other memories (like the internal SRAM).

In the STM32F4xx, three different boot modes can be selected through the BOOT[1:0] pins as shown in [Table 2](#).

Table 2. Boot modes

Boot mode selection pins		Boot mode	Aliasing
BOOT1	BOOT0		
x	0	Main Flash memory	Main Flash memory is selected as the boot space
0	1	System memory	System memory is selected as the boot space
1	1	Embedded SRAM	Embedded SRAM is selected as the boot space

The values on the BOOT pins are latched on the 4th rising edge of SYSCLK after a reset. It is up to the user to set the BOOT1 and BOOT0 pins after reset to select the required boot mode.

BOOT0 is a dedicated pin while BOOT1 is shared with a GPIO pin. Once BOOT1 has been sampled, the corresponding GPIO pin is free and can be used for other purposes.

The BOOT pins are also resampled when the device exits the Standby mode. Consequently, they must be kept in the required Boot mode configuration when the device is in the Standby mode. After this startup delay is over, the CPU fetches the top-of-stack value from address 0x0000 0000, then starts code execution from the boot memory starting from 0x0000 0004.

Note: *When the device boots from SRAM, in the application initialization code, you have to relocate the vector table in SRAM using the NVIC exception table and the offset register.*

Embedded bootloader

The embedded bootloader mode is used to reprogram the Flash memory using one of the following serial interfaces:

- USART1 (PA9/PA10)
- USART2 (PD5/PD6)
- I2C1 (PB6/PB7)
- I2C2 (PB10/PB3)
- I2C3 (PA8/PB4)
- SPI1 (PA4/PA5/PA6/PA7)
- SPI2 (PB12/PB13/PB14/PB15)
- SPI3 (PA15/PC10/PC11/PC12)
- USB OTG FS (PA11/12) in Device mode (DFU: device firmware upgrade).

The USART peripherals operate at the internal 16 MHz oscillator (HSI) frequency, while the USB OTG FS require an external clock (HSE) multiple of 1 MHz (ranging from 4 to 26 MHz).

The embedded bootloader code is located in system memory. It is programmed by ST during production. For additional information, refer to application note AN2606.

Physical remap in STM32F411xC/E

Once the boot pins are selected, the application software can modify the memory accessible in the code area (in this way the code can be executed through the ICode bus in place of the System bus). This modification is performed by programming the [Section 7.2.1: SYSCFG memory remap register \(SYSCFG_MEMRMP\)](#) in the SYSCFG controller.

The following memories can thus be remapped:

- Main Flash memory
- System memory
- Embedded SRAM1

Table 3. Memory mapping vs. Boot mode/physical remap in STM32F411xC/E

Addresses	Boot/Remap in main Flash memory	Boot/Remap in embedded SRAM	Boot/Remap in System memory
0x2000 0000 - 0x2002 0000	SRAM1 (128 KB)	SRAM1 (128KB)	SRAM1 (128KB)
0x1FFF 0000 - 0x1FFF 77FF	System memory	System memory	System memory
0x0804 0000 - 0x1FFE FFFF	Reserved	Reserved	Reserved
0x0800 0000 - 0x0807 FFFF	Flash memory	Flash memory	Flash memory
0x0400 000 - 0x07FF FFFF	Reserved	Reserved	Reserved
0x0000 0000 - 0x0007 FFFF ⁽¹⁾	Flash (512 KB) Aliased	SRAM1 (128 KB) Aliased	System memory (30 KB) Aliased

1. Even when aliased in the boot memory space, the related memory is still accessible at its original memory space.

3 Embedded Flash memory interface

3.1 Introduction

The Flash memory interface manages CPU AHB I-Code and D-Code accesses to the Flash memory. It implements the erase and program Flash memory operations and the read and write protection mechanisms.

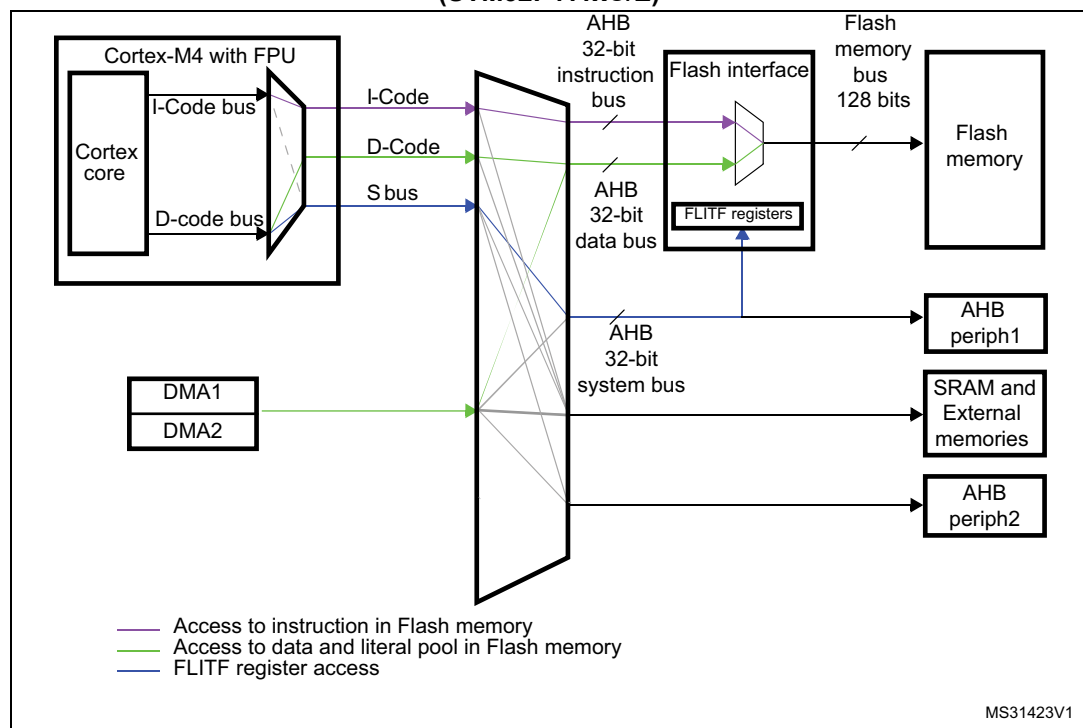
The Flash memory interface accelerates code execution with a system of instruction prefetch and cache lines.

3.2 Main features

- Flash memory read operations
- Flash memory program/erase operations
- Read / write protections
- Prefetch on I-Code
- 64 cache lines of 128 bits on I-Code
- 8 cache lines of 128 bits on D-Code

Figure 2 shows the Flash memory interface connection inside the system architecture.

Figure 2. Flash memory interface connection inside system architecture (STM32F411xC/E)



3.3 Embedded Flash memory in STM32F411xC/E

The Flash memory has the following main features:

- Capacity up to 512 KBytes for STM32F411xC/E
- 128 bits wide data read
- Byte, half-word, word and double word write
- Sector and mass erase
- Memory organization

The Flash memory is organized as follows:

- A main memory block divided into 4 sectors of 16 KBytes, 1 sector of 64 KBytes, 3 sectors of 128 Kbytes (STM32F411xC/E).
- System memory from which the device boots in System memory boot mode
- 512 OTP (one-time programmable) bytes for user data

The OTP area contains 16 additional bytes used to lock the corresponding OTP data block.

- Option bytes to configure read and write protection, BOR level, watchdog software/hardware and reset when the device is in Standby or Stop mode.
- Low-power modes (for details refer to the Power control (PWR) section of the reference manual)

Table 4. Flash module organization (STM32F411xC/E)

Block	Name	Block base addresses	Size
Main memory	Sector 0	0x0800 0000 - 0x0800 3FFF	16 Kbytes
	Sector 1	0x0800 4000 - 0x0800 7FFF	16 Kbytes
	Sector 2	0x0800 8000 - 0x0800 BFFF	16 Kbytes
	Sector 3	0x0800 C000 - 0x0800 FFFF	16 Kbytes
	Sector 4	0x0801 0000 - 0x0801 FFFF	64 Kbytes
	Sector 5	0x0802 0000 - 0x0803 FFFF	128 Kbytes
	Sector 6	0x0804 0000 - 0x0805 FFFF	128 Kbytes
	Sector 7	0x0806 0000 - 0x0807 FFFF	128 Kbytes
System memory		0x1FFF 0000 - 0x1FFF 77FF	30 Kbytes
OTP area		0x1FFF 7800 - 0x1FFF 7A0F	528 bytes
Option bytes		0x1FFF C000 - 0x1FFF C00F	16 bytes

3.4 Read interface

3.4.1 Relation between CPU clock frequency and Flash memory read time

To correctly read data from Flash memory, the number of wait states (LATENCY) must be correctly programmed in the Flash access control register (FLASH_ACR) according to the frequency of the CPU clock (HCLK) and the supply voltage of the device.

The prefetch buffer must be disabled when the supply voltage is below 2.1 V. The correspondence between wait states and CPU clock frequency is given in [Table 5](#).

- when $VOS[1:0] = 0x01$, the maximum value of $f_{HCLK} = 64$ MHz.

- when $VOS[1:0] = 0x10$, the maximum value of $f_{HCLK} = 84$ MHz.

- when $VOS[1:0] = 0x11$, the maximum value of $f_{HCLK} = 100$ MHz.

Table 5. Number of wait states according to CPU clock (HCLK) frequency

Wait states (WS) (LATENCY)	HCLK (MHz)			
	Voltage range 2.7 V - 3.6 V	Voltage range 2.4 V - 2.7 V	Voltage range 2.1 V - 2.4 V	Voltage range 1.71 V - 2.1 V
0 WS (1 CPU cycle)	$0 < HCLK \leq 30$	$0 < HCLK \leq 24$	$0 < HCLK \leq 18$	$0 < HCLK \leq 16$
1 WS (2 CPU cycles)	$30 < HCLK \leq 64$	$24 < HCLK \leq 48$	$18 < HCLK \leq 36$	$16 < HCLK \leq 32$
2 WS (3 CPU cycles)	$64 < HCLK \leq 90$	$48 < HCLK \leq 72$	$36 < HCLK \leq 54$	$32 < HCLK \leq 48$
3 WS (4 CPU cycles)	$90 < HCLK \leq 100$	$72 < HCLK \leq 96$	$54 < HCLK \leq 72$	$48 < HCLK \leq 64$
4 WS (5 CPU cycles)	-	$96 < HCLK \leq 100$	$72 < HCLK \leq 90$	$64 < HCLK \leq 80$
5 WS (6 CPU cycles)	-	-	$90 < HCLK \leq 100$	$80 < HCLK \leq 96$
6 WS (7 CPU cycles)	-	-	-	$96 < HCLK \leq 100$

After reset, the CPU clock frequency is 16 MHz and 0 wait state (WS) is configured in the FLASH_ACR register.

It is highly recommended to use the following software sequences to tune the number of wait states needed to access the Flash memory with the CPU frequency.

Increasing the CPU frequency

1. Program the new number of wait states to the LATENCY bits in the FLASH_ACR register
2. Check that the new number of wait states is taken into account to access the Flash memory by reading the FLASH_ACR register
3. Modify the CPU clock source by writing the SW bits in the RCC_CFGR register
4. If needed, modify the CPU clock prescaler by writing the HPRE bits in RCC_CFGR
5. Check that the new CPU clock source or/and the new CPU clock prescaler value is/are taken into account by reading the clock source status (SWS bits) or/and the AHB prescaler value (HPRE bits), respectively, in the RCC_CFGR register.

Decreasing the CPU frequency

1. Modify the CPU clock source by writing the SW bits in the RCC_CFGR register
2. If needed, modify the CPU clock prescaler by writing the HPRE bits in RCC_CFGR
3. Check that the new CPU clock source or/and the new CPU clock prescaler value is/are taken into account by reading the clock source status (SWS bits) or/and the AHB prescaler value (HPRE bits), respectively, in the RCC_CFGR register
4. Program the new number of wait states to the LATENCY bits in FLASH_ACR
5. Check that the new number of wait states is used to access the Flash memory by reading the FLASH_ACR register

Note: A change in CPU clock configuration or wait state (WS) configuration may not be effective straight away. To make sure that the current CPU clock frequency is the one you have configured, you can check the AHB prescaler factor and clock source status values. To make sure that the number of WS you have programmed is effective, you can read the FLASH_ACR register.

3.4.2 Adaptive real-time memory accelerator (ART Accelerator™)

The proprietary Adaptive real-time (ART) memory accelerator is optimized for STM32 industry-standard Arm® Cortex®-M4 with FPU processors. It balances the inherent performance advantage of the Arm® Cortex®-M4 with FPU over Flash memory technologies, which normally requires the processor to wait for the Flash memory at higher operating frequencies.

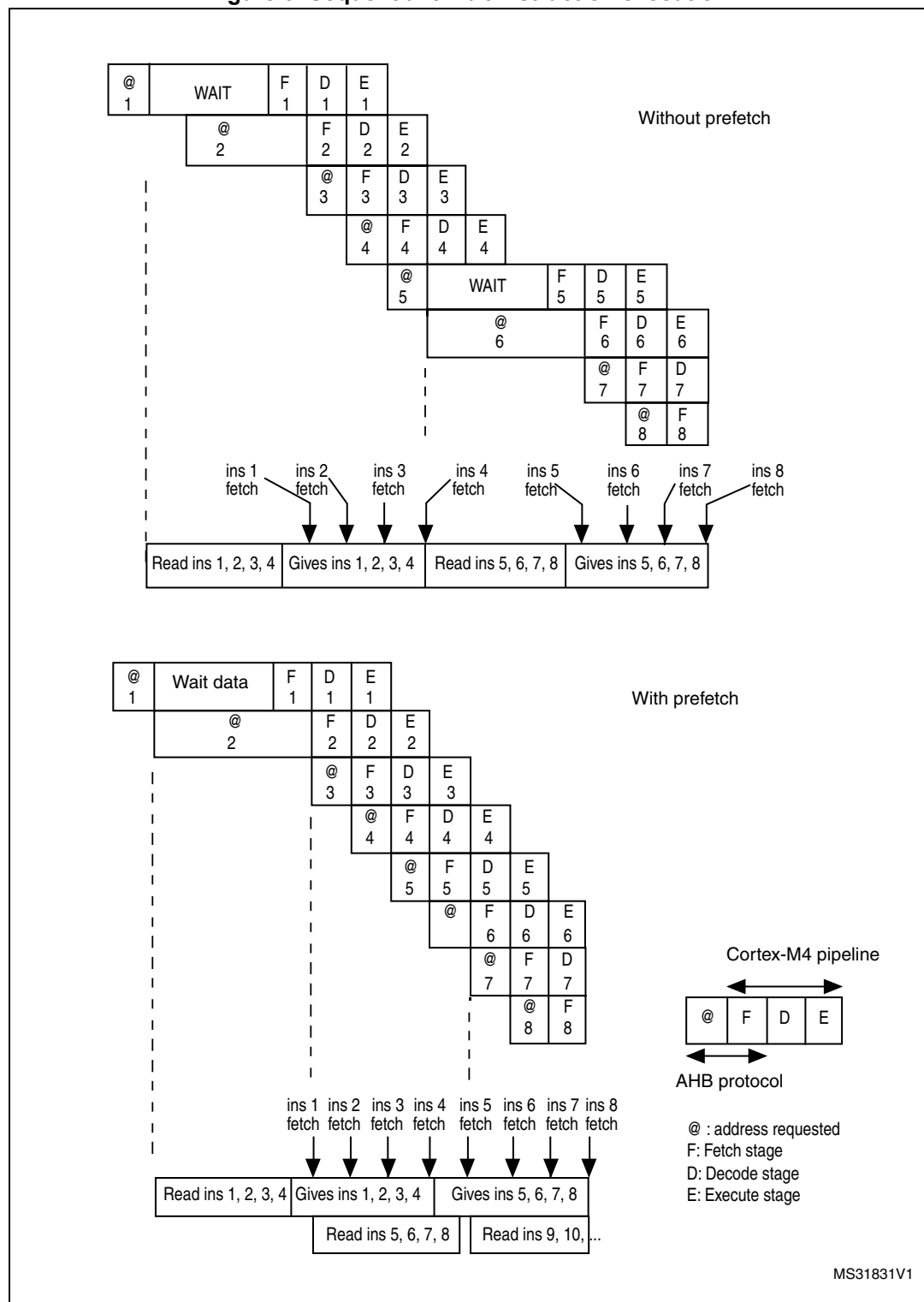
To release the processor full performance, the accelerator implements an instruction prefetch queue and branch cache which increases program execution speed from the 128-bit Flash memory. Based on CoreMark benchmark, the performance achieved thanks to the ART accelerator is equivalent to 0 wait state program execution from Flash memory at a CPU frequency up to 100 MHz.

Instruction prefetch

Each Flash memory read operation provides 128 bits from either four instructions of 32 bits or 8 instructions of 16 bits according to the program launched. So, in case of sequential code, at least four CPU cycles are needed to execute the previous read instruction line. Prefetch on the I-Code bus can be used to read the next sequential instruction line from the Flash memory while the current instruction line is being requested by the CPU. Prefetch is enabled by setting the PRFTEN bit in the FLASH_ACR register. This feature is useful if at least one wait state is needed to access the Flash memory.

Figure 3 shows the execution of sequential 32-bit instructions with and without prefetch when 3 WSs are needed to access the Flash memory.

Figure 3. Sequential 32-bit instruction execution



When the code is not sequential (branch), the instruction may not be present in the currently used instruction line or in the prefetched instruction line. In this case (miss), the penalty in terms of number of cycles is at least equal to the number of wait states.

Instruction cache memory

To limit the time lost due to jumps, it is possible to retain 64 lines of 128 bits in an instruction cache memory. This feature can be enabled by setting the instruction cache enable (ICEN) bit in the FLASH_ACR register. Each time a miss occurs (requested data not present in the currently used instruction line, in the prefetched instruction line or in the instruction cache memory), the line read is copied into the instruction cache memory. If some data contained in the instruction cache memory are requested by the CPU, they are provided without inserting any delay. Once all the instruction cache memory lines have been filled, the LRU (least recently used) policy is used to determine the line to replace in the instruction memory cache. This feature is particularly useful in case of code containing loops.

Data management

Literal pools are fetched from Flash memory through the D-Code bus during the execution stage of the CPU pipeline. The CPU pipeline is consequently stalled until the requested literal pool is provided. To limit the time lost due to literal pools, accesses through the AHB databus D-Code have priority over accesses through the AHB instruction bus I-Code.

If some literal pools are frequently used, the data cache memory can be enabled by setting the data cache enable (DCEN) bit in the FLASH_ACR register. This feature works like the instruction cache memory, but the retained data size is limited to 8 rows of 128 bits.

Note: Data in user configuration sector are not cacheable.

3.5 Erase and program operations

For any Flash memory program operation (erase or program), the CPU clock frequency (HCLK) must be at least 1 MHz. The contents of the Flash memory are not guaranteed if a device reset occurs during a Flash memory operation.

Any attempt to read the Flash memory on STM32F4xx while it is being written or erased, causes the bus to stall. Read operations are processed correctly once the program operation has completed. This means that code or data fetches cannot be performed while a write/erase operation is ongoing.

3.5.1 Unlocking the Flash control register

After reset, write is not allowed in the Flash control register (FLASH_CR) to protect the Flash memory against possible unwanted operations due, for example, to electric disturbances. The following sequence is used to unlock this register:

1. Write KEY1 = 0x45670123 in the Flash key register (FLASH_KEYR)
2. Write KEY2 = 0xCDEF89AB in the Flash key register (FLASH_KEYR)

Any wrong sequence will return a bus error and lock up the FLASH_CR register until the next reset.

The FLASH_CR register can be locked again by software by setting the LOCK bit in the FLASH_CR register.

Note: The FLASH_CR register is not accessible in write mode when the BSY bit in the FLASH_SR register is set. Any attempt to write to it with the BSY bit set will cause the AHB bus to stall until the BSY bit is cleared.

3.5.2 Program/erase parallelism

The Parallelism size is configured through the PSIZE field in the FLASH_CR register. It represents the number of bytes to be programmed each time a write operation occurs to the Flash memory. PSIZE is limited by the supply voltage and by whether the external V_{PP} supply is used or not. It must therefore be correctly configured in the FLASH_CR register before any programming/erasing operation.

A Flash memory erase operation can only be performed by sector, or for the whole Flash memory (mass erase). The erase time depends on PSIZE programmed value. For more details on the erase time, refer to the electrical characteristics section of the device datasheet.

[Table 6](#) provides the correct PSIZE values.

Table 6. Program/erase parallelism

	Voltage range 2.7 - 3.6 V with External V_{PP}	Voltage range 2.7 - 3.6 V	Voltage range 2.4 - 2.7 V	Voltage range 2.1 - 2.4 V	Voltage range 1.7 V - 2.1 V
Parallelism size	x64	x32	x16		x8
PSIZE(1:0)	11	10	01		00

Note: Any program or erase operation started with inconsistent program parallelism/voltage range settings may lead to unpredicted results. Even if a subsequent read operation indicates that the logical value was effectively written to the memory, this value may not be retained.

To use V_{PP} an external high-voltage supply (between 8 and 9 V) must be applied to the V_{PP} pad. The external supply must be able to sustain this voltage range even if the DC consumption exceeds 10 mA. It is advised to limit the use of V_{PP} to initial programming on the factory line. The V_{PP} supply must not be applied for more than an hour, otherwise the Flash memory might be damaged.

3.5.3 Erase

The Flash memory erase operation can be performed at sector level or on the whole Flash memory (Mass Erase). Mass Erase does not affect the OTP sector or the configuration sector.

Sector Erase

To erase a sector, follow the procedure below:

1. Check that no Flash memory operation is ongoing by checking the BSY bit in the FLASH_SR register
2. Set the SER bit and select the sector out of the 7 sectors (STM32F411xC/E) in the main memory block you wish to erase (SNB) in the FLASH_CR register
3. Set the STRT bit in the FLASH_CR register
4. Wait for the BSY bit to be cleared

Mass Erase

To perform Mass Erase, the following sequence is recommended:

1. Check that no Flash memory operation is ongoing by checking the BSY bit in the FLASH_SR register
2. Set the MER bit in the FLASH_CR register
3. Set the STRT bit in the FLASH_CR register
4. Wait for the BSY bit to be cleared

Note: If MERx and SER bits are both set in the FLASH_CR register, mass erase is performed.
If both MERx and SER bits are reset and the STRT bit is set, an unpredictable behavior may occur without generating any error flag. This condition should be forbidden.

3.5.4 Programming

Standard programming

The Flash memory programming sequence is as follows:

1. Check that no main Flash memory operation is ongoing by checking the BSY bit in the FLASH_SR register.
2. Set the PG bit in the FLASH_CR register
3. Perform the data write operation(s) to the desired memory address (inside main memory block or OTP area):
 - Byte access in case of x8 parallelism
 - Half-word access in case of x16 parallelism
 - Word access in case of x32 parallelism
 - Double word access in case of x64 parallelism
4. Wait for the BSY bit to be cleared.

Note: Successive write operations are possible without the need of an erase operation when changing bits from '1' to '0'. Writing '1' requires a Flash memory erase operation.
If an erase and a program operation are requested simultaneously, the erase operation is performed first.

Programming errors

It is not allowed to program data to the Flash memory that would cross the 128-bit row boundary. In such a case, the write operation is not performed and a program alignment error flag (PGAERR) is set in the FLASH_SR register.

The write access type (byte, half-word, word or double word) must correspond to the type of parallelism chosen (x8, x16, x32 or x64). If not, the write operation is not performed and a program parallelism error flag (PGPERR) is set in the FLASH_SR register.

If the standard programming sequence is not respected (for example, if there is an attempt to write to a Flash memory address when the PG bit is not set), the operation is aborted and a program sequence error flag (PGSERR) is set in the FLASH_SR register.

Programming and caches

If a Flash memory write access concerns some data in the data cache, the Flash write access modifies the data in the Flash memory and the data in the cache.

If an erase operation in Flash memory also concerns data in the data or instruction cache, you have to make sure that these data are rewritten before they are accessed during code execution. If this cannot be done safely, it is recommended to flush the caches by setting the DCRST and ICRST bits in the FLASH_CR register.

Note: The I/D cache should be flushed only when it is disabled (I/DCEN = 0).

3.5.5 Interrupts

Setting the end of operation interrupt enable bit (EOPIE) in the FLASH_CR register enables interrupt generation when an erase or program operation ends, that is when the busy bit (BSY) in the FLASH_SR register is cleared (operation completed, correctly or not). In this case, the end of operation (EOP) bit in the FLASH_SR register is set.

If an error occurs during a program, an erase, or a read operation request, one of the following error flags is set in the FLASH_SR register:

- PGAERR, PGPERR, PGSERR (Program error flags)
- WRPERR (Protection error flag)

In this case, if the error interrupt enable bit (ERRIE) is set in the FLASH_CR register, an interrupt is generated and the operation error bit (OPERR) is set in the FLASH_SR register.

Note: If several successive errors are detected (for example, in case of DMA transfer to the Flash memory), the error flags cannot be cleared until the end of the successive write requests.

Table 7. Flash interrupt request

Interrupt event	Event flag	Enable control bit
End of operation	EOP	EOPIE
Write protection error	WRPERR	ERRIE
Programming error	PGAERR, PGPERR, PGSERR	ERRIE

3.6 Option bytes

3.6.1 Description of user option bytes

The option bytes are configured by the end user depending on the application requirements. [Table 8](#) shows the organization of these bytes inside the user configuration sector.

Table 8. Option byte organization

Address	[63:16]	[15:0]
0x1FFF C000	Reserved	ROP & user option bytes (RDP & USER)
0x1FFF C008	Reserved	Write protection nWRP bits for sectors 0 to 7 (STM32F411xC/E)

Table 9. Description of the option bytes

Option bytes (word, address 0x1FFF C000)	
RDP: Read protection option byte. The read protection is used to protect the software code stored in Flash memory.	
Bit 15:8	0xAA: Level 0, no protection 0xCC: Level 2, chip protection (debug and boot from RAM features disabled) Others: Level 1, read protection of memories (debug features limited)
USER: User option byte This byte is used to configure the following features: Select the watchdog event: Hardware or software Reset event when entering the Stop mode Reset event when entering the Standby mode	
Bit 7	nRST_STDBY 0: Reset generated when entering the Standby mode 1: No reset generated
Bit 6	nRST_STOP 0: Reset generated when entering the Stop mode 1: No reset generated
Bit 5	WDG_SW 0: Hardware watchdog 1: Software watchdog
Bit 4	0x0: Not used. Always readout as "0".
Bits 3:2	BOR_LEV: BOR reset Level These bits contain the supply level threshold that activates/releases the reset. They can be written to program a new BOR level value into Flash memory. 00: BOR Level 3 (VBOR3), brownout threshold level 3 01: BOR Level 2 (VBOR2), brownout threshold level 2 10: BOR Level 1 (VBOR1), brownout threshold level 1 11: BOR off, POR/PDR reset threshold level is applied For full details on BOR characteristics, refer to the Electrical characteristics section of the product datasheet.
Bits 1:0	0x1: Not used
Option bytes (word, address 0x1FFF C008)	
Bit 15	SPRMOD: Selection of Protection Mode of nWPRi bits 0: nWPRi bits used for sector i write protection (Default) 1: nWPRi bits used for sector i PCROP protection (Sector)
Bits 14:8	Reserved

Table 9. Description of the option bytes (continued)

nWRP: Flash memory write protection option bytes Sector 0 to 7 (STM32F411xC/E) can be write protected.	
Bits 7:0	nWRPi If SPRMOD is reset (default value) : 0: Write protection active on sector i. 1: Write protection not active on sector i. If SPRMOD is set (active): 0: PCROP protection not active on sector i. 1: PCROP protection active on sector i.

3.6.2 Programming user option bytes

To run any operation on this sector, the option lock bit (OPTLOCK) in the Flash option control register (FLASH_OPTCR) must be cleared. To be allowed to clear this bit, you have to perform the following sequence:

1. Write OPTKEY1 = 0x0819 2A3B in the Flash option key register (FLASH_OPTKEYR)
2. Write OPTKEY2 = 0x4C5D 6E7F in the Flash option key register (FLASH_OPTKEYR)

The user option bytes can be protected against unwanted erase/program operations by setting the OPTLOCK bit by software.

Modifying user option bytes

To modify the user option value, follow the sequence below:

1. Check that no Flash memory operation is ongoing by checking the BSY bit in the FLASH_SR register
2. Write the desired option value in the FLASH_OPTCR register.
3. Set the option start bit (OPTSTRT) in the FLASH_OPTCR register
4. Wait for the BSY bit to be cleared.

Note: The value of an option is automatically modified by first erasing the user configuration sector and then programming all the option bytes with the values contained in the FLASH_OPTCR register.

3.6.3 Read protection (RDP)

The user area in the Flash memory can be protected against read operations by an entrusted code. Three read protection levels are defined:

- Level 0: no read protection
When the read protection level is set to Level 0 by writing 0xAA into the read protection option byte (RDP), all read/write operations (if no write protection is set) from/to the

Flash memory are possible in all boot configurations (Flash user boot, debug or boot from RAM).

- Level 1: read protection enabled

It is the default read protection level after option byte erase. The read protection Level 1 is activated by writing any value (except for 0xAA and 0xCC used to set Level 0 and Level 2, respectively) into the RDP option byte. When the read protection Level 1 is set:

- No access (read, erase, program) to Flash memory can be performed while the debug feature is connected or while booting from RAM or system memory bootloader. A bus error is generated in case of read request.
- When booting from Flash memory, accesses (read, erase, program) to Flash memory from user code are allowed.

When Level 1 is active, programming the protection option byte (RDP) to Level 0 causes the Flash memory to be mass-erased. As a result the user code area is cleared before the read protection is removed. The mass erase only erases the user code area. The other option bytes including write protections remain unchanged from before the mass-erase operation. The OTP area is not affected by mass erase and remains unchanged. Mass erase is performed only when Level 1 is active and Level 0 requested. When the protection level is increased (0->1, 1->2, 0->2) there is no mass erase.

- Level 2: debug/chip read protection disabled

The read protection Level 2 is activated by writing 0xCC to the RDP option byte. When the read protection Level 2 is set:

- All protections provided by Level 1 are active.
- Booting from RAM or system memory bootloader is no more allowed.
- JTAG, SWV (single-wire viewer), ETM, and boundary scan are disabled.
- User option bytes can no longer be changed.
- When booting from Flash memory, accesses (read, erase and program) to Flash memory from user code are allowed.

Memory read protection Level 2 is an irreversible operation. When Level 2 is activated, the level of protection cannot be decreased to Level 0 or Level 1.

Note: The JTAG port is permanently disabled when Level 2 is active (acting as a JTAG fuse). As a consequence, boundary scan cannot be performed. STMicroelectronics is not able to perform analysis on defective parts on which the Level 2 protection has been set.

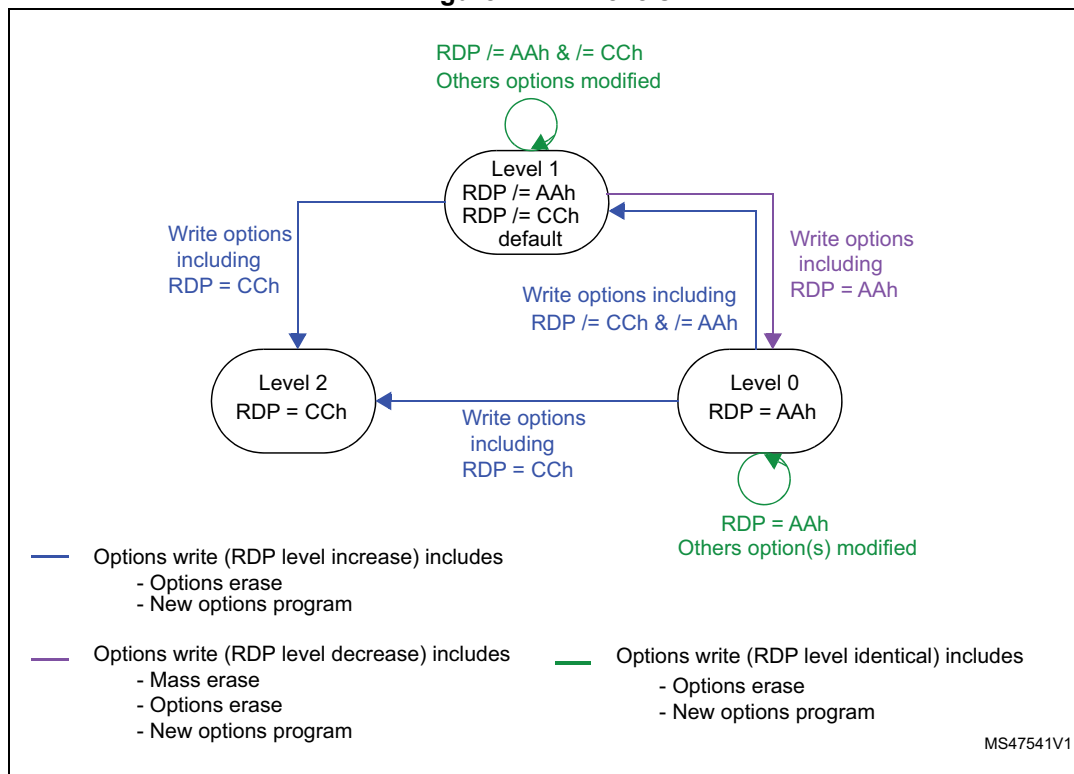
Table 10. Access versus read protection level

Memory area	Protection Level	Debug features, Boot from RAM or from System memory bootloader			Bootling from Flash memory		
		Read	Write	Erase	Read	Write	Erase
Main Flash Memory	Level 1	NO		NO ⁽¹⁾	YES		
	Level 2	NO			YES		
Option Bytes	Level 1	YES			YES		
	Level 2	NO			NO		
OTP	Level 1	NO		NA	YES		NA
	Level 2	NO		NA	YES		NA

1. The main Flash memory only erased when the RDP changes from level 1 to 0. The OTP area remains unchanged.

Figure 4 shows how to go from one RDP level to another.

Figure 4. RDP levels



3.6.4 Write protections

Up to 7 (STM32F411xC/E) user sectors in Flash memory can be protected against unwanted write operations due to loss of program counter contexts. When the non-write protection nWRPi bit ($0 \leq i \leq 7$) in the FLASH_OPTCR registers is low, the corresponding sector cannot be erased or programmed. Consequently, a mass erase cannot be performed if one of the sectors is write-protected.

If an erase/program operation to a write-protected part of the Flash memory is attempted (sector protected by write protection bit, OTP part locked or part of the Flash memory that can never be written like the ICP), the write protection error flag (WRPERR) is set in the FLASH_SR register.

Note: When the memory read protection level is selected (RDP level = 1), it is not possible to program or erase Flash memory sector *i* if the CPU debug features are connected (JTAG or single wire) or boot code is being executed from RAM, even if $nWRP_i = 1$.

Write protection error flag

If an erase/program operation to a write protected area of the Flash memory is performed, the Write Protection Error flag (WRPERR) is set in the FLASH_SR register.

If an erase operation is requested, the WRPERR bit is set when:

- Mass, bank, sector erase are configured (MER and SER = 1)
- A sector erase is requested and the Sector Number SNB field is not valid
- A mass erase is requested while at least one of the user sector is write protected by option bit (MER = 1 and $nWRP_i = 0$ with $0 \leq i \leq 7$ bits in the FLASH_OPTCRx register)
- A sector erase is requested on a write protected sector. (SER = 1, SNB = *i* and $nWRP_i = 0$ with $0 \leq i \leq 7$ bits in the FLASH_OPTCRx register)
- The Flash memory is readout protected and an intrusion is detected.

If a program operation is requested, the WRPERR bit is set when:

- A write operation is performed on system memory or on the reserved part of the user specific sector.
- A write operation is performed to the user configuration sector
- A write operation is performed on a sector write protected by option bit.
- A write operation is requested on an OTP area which is already locked
- The Flash memory is read protected and an intrusion is detected.

3.6.5 Proprietary code readout protection (PCROP)

Flash memory user sectors (0 to 7) can be protected against D-bus read accesses by using the proprietary readout protection (PCROP).

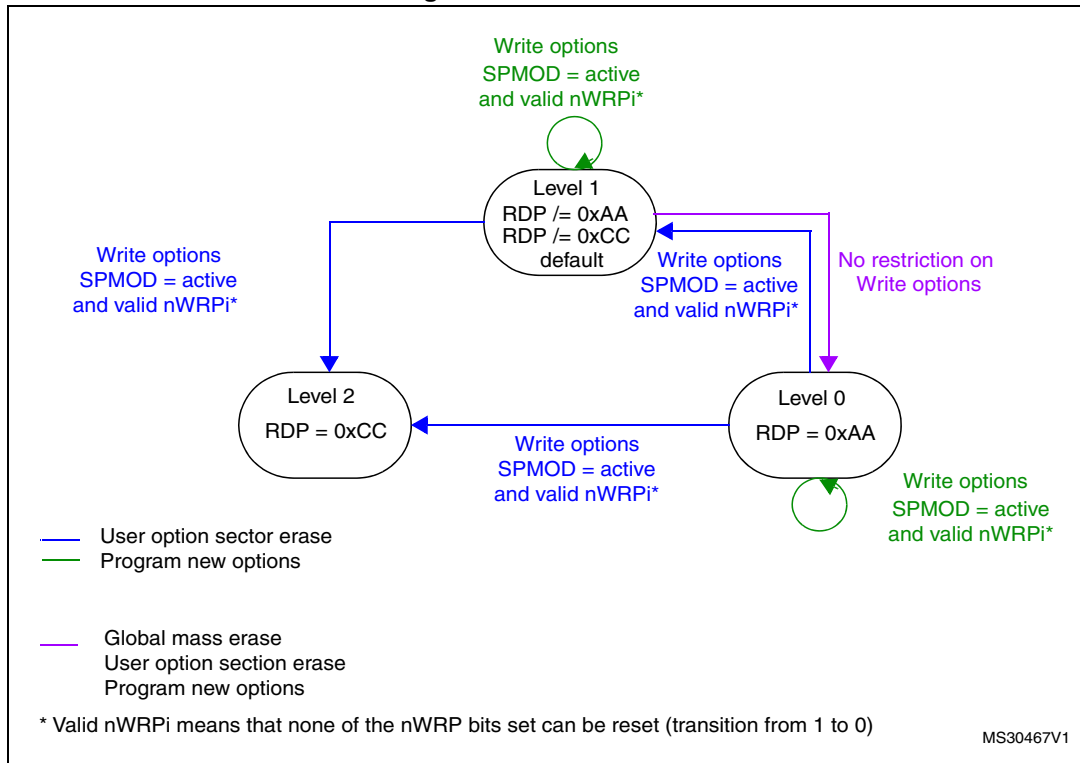
The PCROP protection is selected as follows, through the SPRMOD option bit in the FLASH_OPTCR register:

- SPRMOD = 0: $nWRP_i$ control the write protection of respective user sectors
- SPRMOD = 1: $nWRP_i$ control the read and write protection (PCROP) of respective user sectors.

When a sector is readout protected (PCROP mode activated), it can only be accessed for code fetch through ICODE Bus on Flash interface:

- Any read access performed through the D-bus triggers a RDERR flag error.
- Any program/erase operation on a PCROPed sector triggers a WRPERR flag error.

Figure 5. PCROP levels



The deactivation of the SPRMOD and/or the unprotection of PCROPed user sectors can only occur when, at the same time, the RDP level changes from 1 to 0. If this condition is not respected, the user option byte modification is cancelled and the write error WRPERR flag is set. The modification of the users option bytes (BOR_LEV, RST_STDBY, ..) is allowed since none of the active nWRP_i bits is reset and SPRMOD is kept active.

Note: The active value of nWRP_i bits is inverted when PCROP mode is active (SPRMOD = 1).
If SPRMOD = 1 and nWRP_i = 1, then user sector *i* of bank 1, respectively bank 2 is read/write protected (PCROP).

3.7 One-time programmable bytes

Table 11 shows the organization of the one-time programmable (OTP) part of the OTP area.

Table 11. OTP area organization

Block	[128:96]	[95:64]	[63:32]	[31:0]	Address byte 0
0	OTP0	OTP0	OTP0	OTP0	0x1FFF 7800
	OTP0	OTP0	OTP0	OTP0	0x1FFF 7810
1	OTP1	OTP1	OTP1	OTP1	0x1FFF 7820
	OTP1	OTP1	OTP1	OTP1	0x1FFF 7830
.	.				.
.	.				.
.	.				.
15	OTP15	OTP15	OTP15	OTP15	0x1FFF 79E0
	OTP15	OTP15	OTP15	OTP15	0x1FFF 79F0
Lock block	LOCKB15 ... LOCKB12	LOCKB11 ... LOCKB8	LOCKB7 ... LOCKB4	LOCKB3 ... LOCKB0	0x1FFF 7A00

The OTP area is divided into 16 OTP data blocks of 32 bytes and one lock OTP block of 16 bytes. The OTP data and lock blocks cannot be erased. The lock block contains 16 bytes LOCKBi ($0 \leq i \leq 15$) to lock the corresponding OTP data block (blocks 0 to 15). Each OTP data block can be programmed until the value 0x00 is programmed in the corresponding OTP lock byte. The lock bytes must only contain 0x00 and 0xFF values, otherwise the OTP bytes might not be taken into account correctly.

3.8 Flash interface registers

3.8.1 Flash access control register (FLASH_ACR)

The Flash access control register is used to enable/disable the acceleration features and control the Flash memory access time according to CPU frequency.

Address offset: 0x00

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			DCRST	ICRST	DCEN	ICEN	PRFTEN	Reserved				LATENCY			
			rw	w	rw	rw	rw					rw	rw	rw	rw

Bits 31:13 Reserved, must be kept cleared.

Bit 12 **DCRST**: Data cache reset

0: Data cache is not reset

1: Data cache is reset

This bit can be written only when the D cache is disabled.

Bit 11 **ICRST**: Instruction cache reset

0: Instruction cache is not reset

1: Instruction cache is reset

This bit can be written only when the I cache is disabled.

Bit 10 **DCEN**: Data cache enable

0: Data cache is disabled

1: Data cache is enabled

Bit 9 **ICEN**: Instruction cache enable

0: Instruction cache is disabled

1: Instruction cache is enabled

Bit 8 **PRFTEN**: Prefetch enable

0: Prefetch is disabled

1: Prefetch is enabled

Bits 7:4 Reserved, must be kept cleared.

Bits 3:0 **LATENCY**: Latency

These bits represent the ratio of the CPU clock period to the Flash memory access time.

0000: Zero wait state

0001: One wait state

0010: Two wait states

-

-

-

1110: Fourteen wait states

1111: Fifteen wait states

3.8.2 Flash key register (FLASH_KEYR)

The Flash key register is used to allow access to the Flash control register and so, to allow program and erase operations.

Address offset: 0x04

Reset value: 0x0000 0000

Access: no wait state, word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **FKEYR[31:0]**: FPEC key

The following values must be programmed consecutively to unlock the FLASH_CR register and allow programming/erasing it:

- a) KEY1 = 0x45670123
- b) KEY2 = 0xCDEF89AB

3.8.3 Flash option key register (FLASH_OPTKEYR)

The Flash option key register is used to allow program and erase operations in the user configuration sector.

Address offset: 0x08

Reset value: 0x0000 0000

Access: no wait state, word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OPTKEYR[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPTKEYR[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **OPTKEYR[31:0]**: Option byte key

The following values must be programmed consecutively to unlock the FLASH_OPTCR register and allow programming it:

- a) OPTKEY1 = 0x08192A3B
- b) OPTKEY2 = 0x4C5D6E7F

3.8.4 Flash status register (FLASH_SR)

The Flash status register gives information on ongoing program and erase operations.

Address offset: 0x0C

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															BSY
															r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							RDERR	PGSERR	PGPERR	PGAERR	WRPERR	Reserved	OPERR	EOP	
							rw	rc_w1	rc_w1	rc_w1	rc_w1		rc_w1	rc_w1	rc_w1

Bits 31:17 Reserved, must be kept cleared.

Bit 16 **BSY**: Busy

This bit indicates that a Flash memory operation is in progress. It is set at the beginning of a Flash memory operation and cleared when the operation finishes or an error occurs.

0: no Flash memory operation ongoing

1: Flash memory operation ongoing

Bits 15:9 Reserved, must be kept cleared.

Bit 8 **RDERR**: Read Protection Error (pcrop)

Set by hardware when an address to be read through the Dbus belongs to a read protected part of the flash.

Reset by writing 1.

Bit 7 **PGSERR**: Programming sequence error

Set by hardware when a write access to the Flash memory is performed by the code while the control register has not been correctly configured.

Cleared by writing 1.

Bit 6 **PGPERR**: Programming parallelism error

Set by hardware when the size of the access (byte, half-word, word, double word) during the program sequence does not correspond to the parallelism configuration PSIZE (x8, x16, x32, x64).

Cleared by writing 1.

Bit 5 **PGAERR**: Programming alignment error

Set by hardware when the data to program cannot be contained in the same 128-bit Flash memory row.

Cleared by writing 1.

Bit 4 **WRPERR**: Write protection error

Set by hardware when an address to be erased/programmed belongs to a write-protected part of the Flash memory.

Cleared by writing 1.

Bits 3:2 Reserved, must be kept cleared.

Bit 1 **OPERR**: Operation error

Set by hardware when a flash operation (programming / erase /read) request is detected and can not be run because of parallelism, alignment, or write protection error. This bit is set only if error interrupts are enabled (ERRIE = 1).

Bit 0 **EOP**: End of operation

Set by hardware when one or more Flash memory operations (program/erase) has/have completed successfully. It is set only if the end of operation interrupts are enabled (EOPIE = 1). Cleared by writing a 1.

3.8.5 Flash control register (FLASH_CR)

The Flash control register is used to configure and start Flash memory operations.

Address offset: 0x10

Reset value: 0x8000 0000

Access: no wait state when no Flash memory operation is ongoing, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCK	Reserved					ERRIE	EOPIE	Reserved							STRT
rs						rw	rw								rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved						PSIZE[1:0]		Res.	SNB[3:0]				MER	SER	PG
						rw	rw		rw	rw	rw	rw	rw	rw	rw

Bit 31 **LOCK**: Lock

Write to 1 only. When it is set, this bit indicates that the FLASH_CR register is locked. It is cleared by hardware after detecting the unlock sequence. In the event of an unsuccessful unlock operation, this bit remains set until the next reset.

Bits 30:26 Reserved, must be kept cleared.

Bit 25 **ERRIE**: Error interrupt enable

This bit enables the interrupt generation when the OPERR bit in the FLASH_SR register is set to 1.

0: Error interrupt generation disabled
1: Error interrupt generation enabled

Bit 24 **EOPIE**: End of operation interrupt enable

This bit enables the interrupt generation when the EOP bit in the FLASH_SR register goes to 1.

0: Interrupt generation disabled
1: Interrupt generation enabled

Bits 23:17 Reserved, must be kept cleared.

Bit 16 **STRT**: Start

This bit triggers an erase operation when set. It is set only by software and cleared when the BSY bit is cleared.

Bits 15:10 Reserved, must be kept cleared.

Bits 9:8 **PSIZE[1:0]**: Program size

These bits select the program parallelism.

00 program x8
01 program x16
10 program x32
11 program x64

Bit 7 Reserved, must be kept cleared.

Bits 6:3 **SNB[3:0]**: Sector number

These bits select the sector to erase.

0000 sector 0
0001 sector 1
...
0101 sector 5
0110 sector 6
0111 sector 7
1000 not allowed
...
1011 not allowed
1100 user specific sector
1101 user configuration sector
1110 not allowed
1111 not allowed

Bit 2 **MER**: Mass Erase

Erase activated for all user sectors.

Bit 1 **SER**: Sector Erase

Sector Erase activated.

Bit 0 **PG**: Programming

Flash programming activated.

3.8.6 Flash option control register (FLASH_OPTCR)

The FLASH_OPTCR register is used to modify the user option bytes.

Address offset: 0x14

Reset value: 0x0FFF AAED. The option bits are loaded with values from Flash memory at reset release.

Access: no wait state when no Flash memory operation is ongoing, word, half-word and byte access.

SPR MOD	Reserved							nWRP[7:0]							
rw								rw	rw	rw	rw	rw	rw	rw	rw
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDP[7:0]								nRST_STDBY	nRST_STOP	WDG_SW	Reserve d	BOR_LEV		OPTST_RT	OPTLOCK
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rs	rs

- Bit 31 **SPRMOD**: Selection of Protection Mode of nWPRi bits
0: PCROP disabled, nWPRi bits used for Write Protection on sector i
1: PCROP enabled, nWPRi bits used for PCROP Protection on sector i
- Bits 30:24 Reserved, must be kept cleared.
- Bits 23:16 **nWRP[7:0]**: Not write protect
These bits contain the value of the write-protection option bytes of sectors after reset. They can be written to program a new write protect value into Flash memory.
0: Write protection active on selected sector
1: Write protection not active on selected sector
These bits contain the value of the write-protection and read-protection (PCROP) option bytes for sectors 0 to 5 after reset. They can be written to program a new write-protect or PCROP value into Flash memory.
If SPRMOD is reset:
0: Write protection active on sector i
1: Write protection not active on sector i
If SPRMOD is set:
0: PCROP protection not active on sector i
1: PCROP protection active on sector i
- Bits 15:8 **RDP[7:0]**: Read protect
These bits contain the value of the read-protection option level after reset. They can be written to program a new read protection value into Flash memory.
0xAA: Level 0, read protection not active
0xCC: Level 2, chip read protection active
Others: Level 1, read protection of memories active
- Bits 7:5 **USER[2:0]**: User option bytes
These bits contain the value of the user option byte after reset. They can be written to program a new user option byte value into Flash memory.
Bit 7: nRST_STDBY
Bit 6: nRST_STOP
Bit 5: WDG_SW
Note: When changing the WDG mode from hardware to software or from software to hardware, a system reset is required to make the change effective.
- Bit 4 Reserved, must be kept cleared. Always read as "0".

Bits 3:2 BOR_LEV[1:0]: BOR reset Level

These bits contain the supply level threshold that activates/releases the reset. They can be written to program a new BOR level. By default, BOR is off. When the supply voltage (V_{DD}) drops below the selected BOR level, a device reset is generated.

00: BOR Level 3 (VBOR3), brownout threshold level 3

01: BOR Level 2 (VBOR2), brownout threshold level 2

10: BOR Level 1 (VBOR1), brownout threshold level 1

11: BOR off, POR/PDR reset threshold level is applied

Note: For full details about BOR characteristics, refer to the “Electrical characteristics” section in the device datasheet.

Bit 1 OPTSTRT: Option start

This bit triggers a user option operation when set. It is set only by software and cleared when the BSY bit is cleared.

Bit 0 OPTLOCK: Option lock

Write to 1 only. When this bit is set, it indicates that the FLASH_OPTCR register is locked. This bit is cleared by hardware after detecting the unlock sequence.

In the event of an unsuccessful unlock operation, this bit remains set until the next reset.

3.8.7 Flash interface register map

Table 12. Flash register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x00	FLASH_ACR	Reserved																			DCRST	ICRST	DCEN	ICEN	PRFTEN	Reserved			LATENCY							
	Reset value																				0	0	0	0	0				0	0	0	0				
0x04	FLASH_KEYR	KEY[31:16]															KEY[15:0]																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x08	FLASH_OPTKEYR	OPTKEYR[31:16]															OPTKEYR[15:0]																			
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x0C	FLASH_SR	Reserved															BSY	Reserved							RDERR	PGSERR	PGPERR	PGAERR	WRPERR	Reserved		OPERR	EOP			
	Reset value																0								0	0	0	0	0			0	0			
0x10	FLASH_CR	LOCK	Reserved						ERRIE	EOPIE	Reserved						STRT	Reserved					PSIZE[1:0]		Reserved	SNB[3:0]			MER	SER	PG					
	Reset value	1							0	0							0						0		0	0	0	0	0	0	0	0				
0x14	FLASH_OPTCR	SPRMOD	Reserved								nWRP[7:0]						RDP[7:0]						nRST_STDBY	nRST_STOP	WDG_SW	Reserved	BOR_LEV		OPTSTRT	OPTLOCK						
	Reset value	0									1						1						1						1	1	1		1		1	0

4 CRC calculation unit

4.1 CRC introduction

The CRC (cyclic redundancy check) calculation unit is used to get a CRC code from a 32-bit data word and a fixed generator polynomial.

Among other applications, CRC-based techniques are used to verify data transmission or storage integrity. In the scope of the EN/IEC 60335-1 standard, they offer a means of verifying the Flash memory integrity. The CRC calculation unit helps compute a signature of the software during runtime, to be compared with a reference signature generated at link-time and stored at a given memory location.

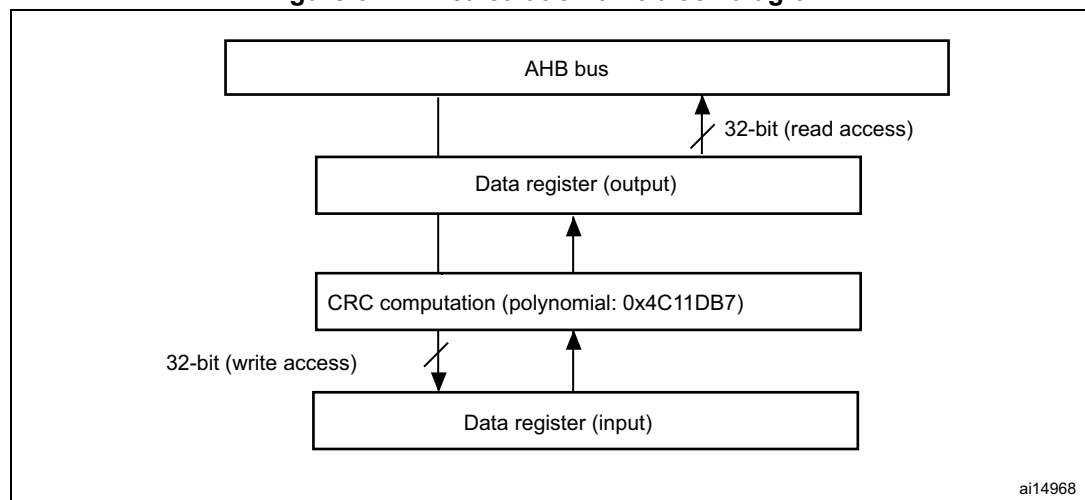
4.2 CRC main features

- Uses CRC-32 polynomial: 0x4C11DB7

$$- X^{32} + X^{26} + X^{23} + X^{22} + X^{16} + X^{12} + X^{11} + X^{10} + X^8 + X^7 + X^5 + X^4 + X^2 + X + 1$$
- Single input/output 32-bit data register
- CRC computation done in 4 AHB clock cycles (HCLK)
- General-purpose 8-bit register (can be used for temporary storage)

The block diagram is shown in [Figure 6](#).

Figure 6. CRC calculation unit block diagram



4.3 CRC functional description

The CRC calculation unit mainly consists of a single 32-bit data register, which:

- is used as an input register to enter new data in the CRC calculator (when writing into the register)
- holds the result of the previous CRC calculation (when reading the register)

Each write operation into the data register creates a combination of the previous CRC value and the new one (CRC computation is done on the whole 32-bit data word, and not byte per byte).

The write operation is stalled until the end of the CRC computation, thus allowing back-to-back write accesses or consecutive write and read accesses.

The CRC calculator can be reset to 0xFFFF FFFF with the RESET control bit in the CRC_CR register. This operation does not affect the contents of the CRC_IDR register.

4.4 CRC registers

The CRC calculation unit contains two data registers and a control register. The peripheral The CRC registers have to be accessed by words (32 bits).

4.4.1 Data register (CRC_DR)

Address offset: 0x00

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DR [31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR [15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **Data register bits**

Used as an input register when writing new data into the CRC calculator.

Holds the previous CRC calculation result when it is read.

4.4.2 Independent data register (CRC_IDR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								IDR[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **General-purpose 8-bit data register bits**

Can be used as a temporary storage location for one byte.

This register is not affected by CRC resets generated by the RESET bit in the CRC_CR register.

4.4.3 Control register (CRC_CR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														RESET	
														w	

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **RESET bit**

Resets the CRC calculation unit and sets the data register to 0xFFFF FFFF.

This bit can only be set, it is automatically cleared by hardware.

4.4.4 CRC register map

The following table provides the CRC register map and reset values.

Table 13. CRC calculation unit register map and reset values

Offset	Register	31-24	23-16	15-8	7	6	5	4	3	2	1	0
0x00	CRC_DR	Data register										
	Reset value	0xFFFF FFFF										
0x04	CRC_IDR	Reserved			Independent data register							
	Reset value				0x00							
0x08	CRC_CR	Reserved										RESET
	Reset value											0

5 Power controller (PWR)

5.1 Power supplies

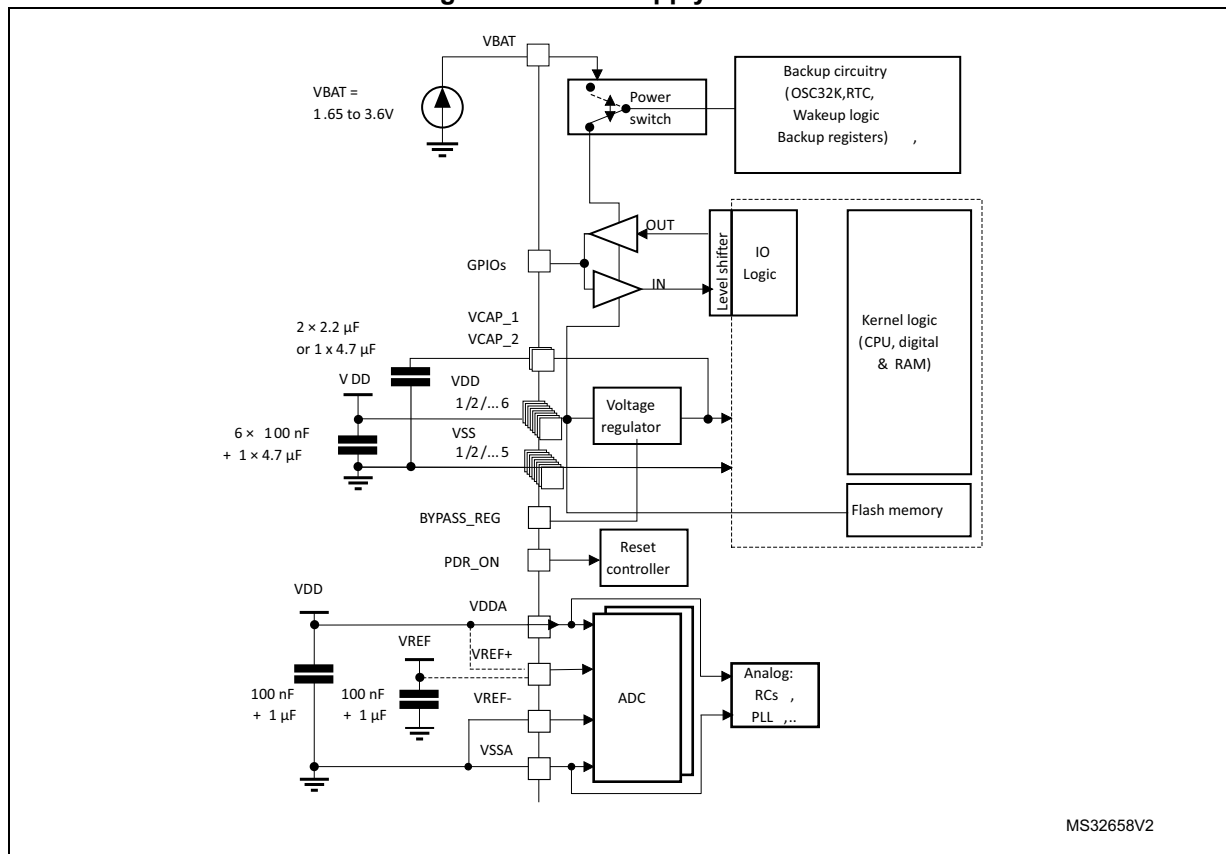
There are two main power supply schemes:

- VDD = 1.7 to 3.6 V: external power supply for I/Os with the internal regulator disabled, provided externally through VDD pins. Requires the use of an external power supply supervisor connected to the VDD and PDR_ON pins.
- VDD = 1.8 to 3.6 V: external power supply for I/Os and the internal regulator (when enabled), provided externally through VDD pins.

The real-time clock (RTC) and the RTC backup registers can be powered from the V_{BAT} voltage when the main V_{DD} supply is powered off.

Note: Depending on the operating power supply range, some peripheral may be used with limited functionality and performance. For more details refer to section “General operating conditions” in STM32F4xx datasheets.

Figure 7. Power supply overview



1. V_{DPA} and V_{SSA} must be connected to V_{DD} and V_{SS} , respectively.

5.1.1 Independent A/D converter supply and reference voltage

To improve conversion accuracy, the ADC has an independent power supply which can be separately filtered and shielded from noise on the PCB.

- The ADC voltage supply input is available on a separate V_{DDA} pin.
- An isolated supply ground connection is provided on pin V_{SSA} .

To ensure a better accuracy of low voltage inputs, the user can connect a separate external reference voltage ADC input on V_{REF} . The voltage on V_{REF} ranges from 1.7 V to V_{DDA} .

5.1.2 Battery backup domain

Backup domain description

To retain the content of the RTC backup registers and supply the RTC when V_{DD} is turned off, V_{BAT} pin can be connected to an optional standby voltage supplied by a battery or by another source.

To allow the RTC to operate even when the main digital supply (V_{DD}) is turned off, the V_{BAT} pin powers the following blocks:

- The RTC
- The LSE oscillator
- PC13 to PC15 I/Os

The switch to the V_{BAT} supply is controlled by the power-down reset embedded in the Reset block.

Warning: During $t_{RSTTEMPO}$ (temporization at V_{DD} startup) or after a PDR is detected, the power switch between V_{BAT} and V_{DD} remains connected to V_{BAT} .
 During the startup phase, if V_{DD} is established in less than $t_{RSTTEMPO}$ (Refer to the datasheet for the value of $t_{RSTTEMPO}$) and $V_{DD} > V_{BAT} + 0.6$ V, a current may be injected into V_{BAT} through an internal diode connected between V_{DD} and the power switch (V_{BAT}).
 If the power supply/battery connected to the V_{BAT} pin cannot support this current injection, it is strongly recommended to connect an external low-drop diode between this power supply and the V_{BAT} pin.

If no external battery is used in the application, it is recommended to connect the V_{BAT} pin to V_{DD} with a 100 nF external decoupling ceramic capacitor in parallel.

When the backup domain is supplied by V_{DD} (analog switch connected to V_{DD}), the following functions are available:

- PC14 and PC15 can be used as either GPIO or LSE pins
- PC13 can be used as a GPIO or additional functions can be configured (refer to [Table 25: RTC additional functions](#) for more details about this pin configuration)

Note: Due to the fact that the switch only sinks a limited amount of current (3 mA), the use of PC13 to PC15 GPIOs in output mode is restricted: the speed has to be limited to 2 MHz with

a maximum load of 30 pF and these I/Os must not be used as a current source (e.g. to drive an LED).

When the backup domain is supplied by V_{BAT} (analog switch connected to V_{BAT} because V_{DD} is not present), the following functions are available:

- PC14 and PC15 can be used as LSE pins only
- PC13 can be used as the RTC additional function pin (refer to [Table 25: RTC additional functions](#) for more details about this pin configuration)

Backup domain access

After reset, the backup domain (RTC registers and RTC backup register) is protected against possible unwanted write accesses. To enable access to the backup domain, proceed as follows:

- Access to the RTC and RTC backup registers
1. Enable the power interface clock by setting the PWREN bits in the RCC_APB1ENR register (see [Section 6.3.11: RCC APB1 peripheral clock enable register \(RCC_APB1ENR\)](#))
 2. Set the DBP bit in the [Section 5.4.1](#) to enable access to the backup domain
 3. Select the RTC clock source: see [Section 7.2.8: RTC/AWU clock](#)
 4. Enable the RTC clock by programming the RTCEN [15] bit in the [Section 7.3.20: RCC Backup domain control register \(RCC_BDCR\)](#)

RTC and RTC backup registers

The real-time clock (RTC) is an independent BCD timer/counter. The RTC provides a time-of-day clock/calendar, two programmable alarm interrupts, and a periodic programmable wakeup flag with interrupt capability. The RTC contains 20 backup data registers (80 bytes) which are reset when a tamper detection event occurs. For more details refer to [Section 17: Real-time clock \(RTC\)](#).

5.1.3 Voltage regulator

An embedded linear voltage regulator supplies all the digital circuitries except for the backup domain and the Standby circuitry. The regulator output voltage is around 1.2 V.

This voltage regulator requires one or two external capacitors to be connected to one or two dedicated pins, V_{CAP_1} and for some packages V_{CAP_2} . Specific pins must be connected either to V_{SS} or V_{DD} to activate or deactivate the voltage regulator. These pins depend on the package.

When activated by software, the voltage regulator is always enabled after Reset. It works in three different modes depending on the application modes.

- In **Run mode**, the regulator supplies full power to the 1.2 V domain (core, memories and digital peripherals). In this mode, the regulator output voltage (around 1.2 V) can be scaled by software to different voltage values:
 Scale 1, scale 2, or scale 3 can be configured through the VOS[1:0] bits of the PWR_CR register. After reset the VOS register is set to scale 2. When the PLL is OFF, the voltage regulator is set to scale 3 independently of the VOS register

content. The VOS register content is only taken into account once the PLL is activated and the HSI or HSE is selected as clock source.

The voltage scaling allows optimizing the power consumption when the device is clocked below the maximum system frequency.

- In **Stop mode**, the main regulator or the low-power regulator supplies low power to the 1.2 V domain, thus preserving the content of registers and internal SRAM. The voltage regulator can be put either in main regulator mode (MR) or in low-power mode (LPR). The programmed voltage scale remains the same during Stop mode:

Voltage scale 3 is automatically selected when the microcontroller enters Stop mode (see [Section 5.4.1: PWR power control register \(PWR_CR\)](#)).

- In **Standby mode**, the regulator is powered down. The content of the registers and SRAM are lost except for the Standby circuitry and the backup domain.

Note: For more details, refer to the voltage regulator section in the STM32F411xC/E datasheet.

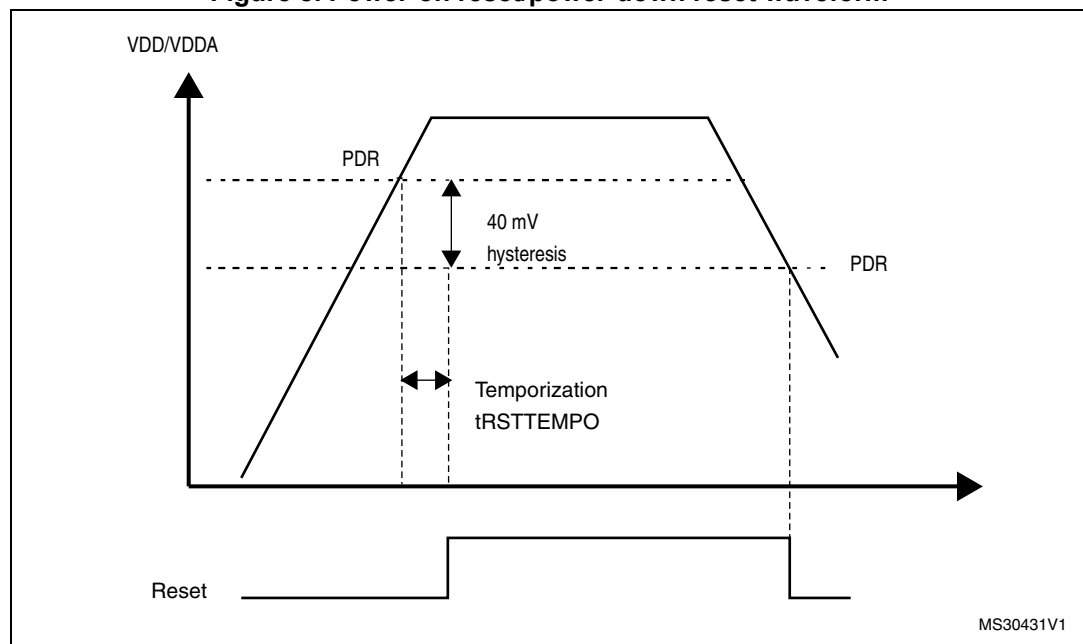
5.2 Power supply supervisor

5.2.1 Power-on reset (POR)/power-down reset (PDR)

The device has an integrated POR/PDR circuitry that allows proper operation starting from 1.8 V.

To use the device below 1.8 V, the internal power supervisor must be switched off using the PDR_ON pin (please refer to section Power supply supervisor of the STM32F411xC/E datasheet). The device remains in Reset mode when V_{DD}/V_{DDA} is below a specified threshold, $V_{POR/PDR}$, without the need for an external reset circuit. For more details concerning the power on/power-down reset threshold, refer to the electrical characteristics of the datasheet.

Figure 8. Power-on reset/power-down reset waveform



5.2.2 Brownout reset (BOR)

During power on, the Brownout reset (BOR) keeps the device under reset until the supply voltage reaches the specified V_{BOR} threshold.

V_{BOR} is configured through device option bytes. By default, BOR is off. 3 programmable V_{BOR} threshold levels can be selected:

- BOR Level 3 (VBOR3). Brownout threshold level 3.
- BOR Level 2 (VBOR2). Brownout threshold level 2.
- BOR Level 1 (VBOR1). Brownout threshold level 1.

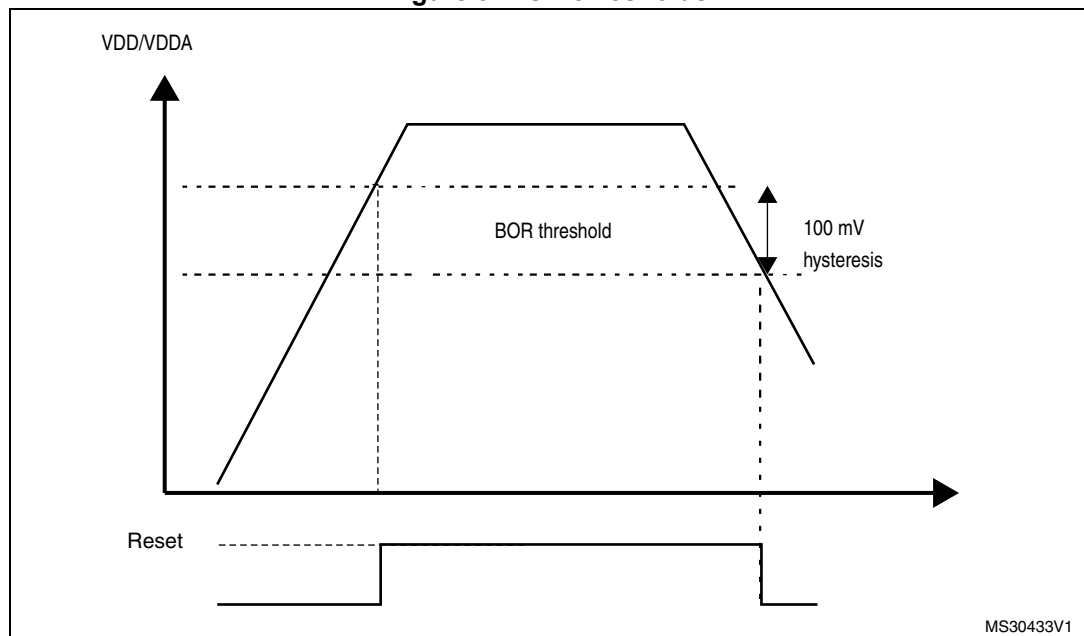
Note: For full details about BOR characteristics, refer to the "Electrical characteristics" section in the device datasheet.

When the supply voltage (V_{DD}) drops below the selected V_{BOR} threshold, a device reset is generated.

The BOR can be disabled by programming the device option bytes. In this case, the power-on and power-down is then monitored by the POR/ PDR or by an external power supervisor if the PDR is switched off through the PDR_ON pin (see [Section 5.2.1: Power-on reset \(POR\)/power-down reset \(PDR\)](#)).

The BOR threshold hysteresis is ~100 mV (between the rising and the falling edge of the supply voltage).

Figure 9. BOR thresholds



MS30433V1

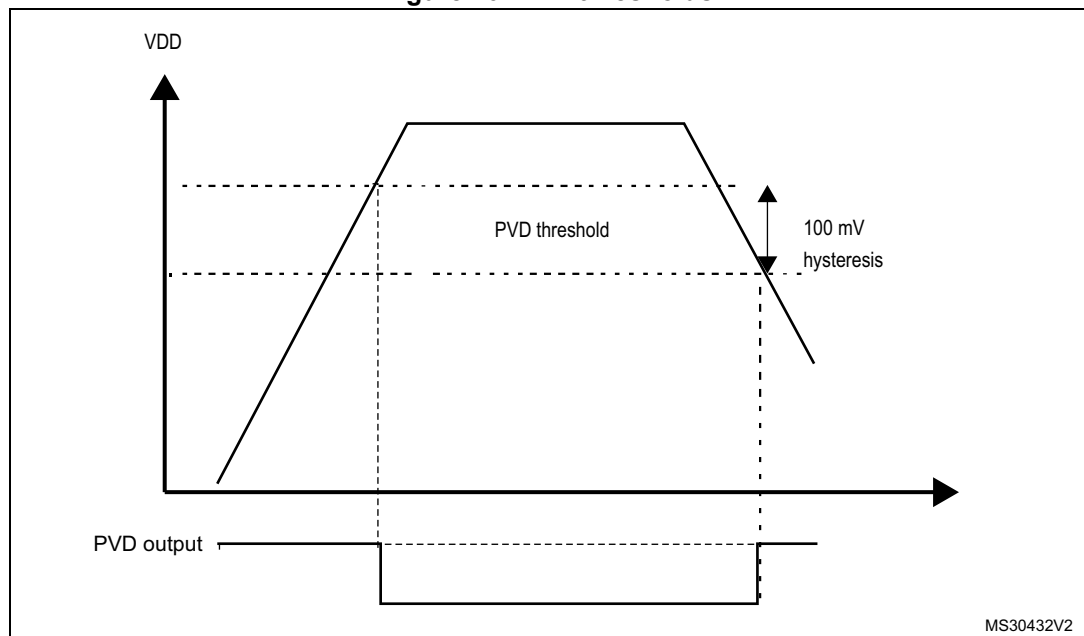
5.2.3 Programmable voltage detector (PVD)

You can use the PVD to monitor the V_{DD} power supply by comparing it to a threshold selected by the PLS[2:0] bits in the *PWR power control register (PWR_CR)*.

The PVD is enabled by setting the PVDE bit.

A PVDO flag is available, in the *PWR power control/status register (PWR_CSR)*, to indicate if V_{DD} is higher or lower than the PVD threshold. This event is internally connected to the EXTI line16 and can generate an interrupt if enabled through the EXTI registers. The PVD output interrupt can be generated when V_{DD} drops below the PVD threshold and/or when V_{DD} rises above the PVD threshold depending on EXTI line16 rising/falling edge configuration. As an example the service routine could perform emergency shutdown tasks.

Figure 10. PVD thresholds



5.3 Low-power modes

By default, the microcontroller is in Run mode after a system or a power-on reset. In Run mode the CPU is clocked by HCLK and the program code is executed. Several low-power modes are available to save power when the CPU does not need to be kept running, for example when waiting for an external event. It is up to the user to select the mode that gives the best compromise between low-power consumption, short startup time and available wakeup sources.

The devices feature three low-power modes:

- Sleep mode (Cortex®-M4 with FPU core stopped, peripherals kept running)
- Stop mode (all clocks are stopped)
- Standby mode (1.2 V domain powered off)

In addition, the power consumption in Run mode can be reduced by one of the following means:

- Slowing down the system clocks
- Gating the clocks to the APBx and AHBx peripherals when they are unused.

Entering low-power mode

Low-power modes are entered by the MCU by executing the WFI (Wait For Interrupt), or WFE (Wait for Event) instructions, or when the SLEEPONEXIT bit in the Cortex®-M4 with FPU System Control register is set on Return from ISR.

Entering Low-power mode through WFI or WFE will be executed only if no interrupt is pending or no event is pending.

Exiting low-power mode

The MCU exits from Sleep and Stop modes low-power mode depending on the way the low-power mode was entered:

- If the WFI instruction or Return from ISR was used to enter the low-power mode, any peripheral interrupt acknowledged by the NVIC can wake up the device.
- If the WFE instruction is used to enter the low-power mode, the MCU exits the low-power mode as soon as an event occurs. The wakeup event can be generated either by:

- NVIC IRQ interrupt:

When SEVONPEND = 0 in the Cortex®-M4 with FPU System Control register: by enabling an interrupt in the peripheral control register and in the NVIC. When the MCU resumes from WFE, the peripheral interrupt pending bit and the NVIC peripheral IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared. Only NVIC interrupts with sufficient priority will wakeup and interrupt the MCU.

When SEVONPEND = 1 in the Cortex®-M4 with FPU System Control register: by enabling an interrupt in the peripheral control register and optionally in the NVIC. When the MCU resumes from WFE, the peripheral interrupt pending bit and when enabled the NVIC peripheral IRQ channel pending bit (in the NVIC interrupt clear pending register) have to be cleared. All NVIC interrupts will wakeup the MCU, even the disabled ones. Only enabled NVIC interrupts with sufficient priority will wakeup and interrupt the MCU.

- Event

This is done by configuring a EXTI line in event mode. When the CPU resumes from WFE, it is not necessary to clear the EXTI peripheral interrupt pending bit or the NVIC IRQ channel pending bit as the pending bits corresponding to the event line is not set. It may be necessary to clear the interrupt flag in the peripheral.

The MCU exits from Standby low-power mode through an external reset (NRST pin), an IWDG reset, a rising edge on one of the enabled WKUPx pins or a RTC event occurs (see [Figure 159: RTC block diagram](#)).

After waking up from Standby mode, program execution restarts in the same way as after a Reset (boot pin sampling, option bytes loading, reset vector is fetched, etc.).

Only enabled NVIC interrupts with sufficient priority will wakeup and interrupt the MCU.

Table 14. Low-power mode summary

Mode name	Entry	Wakeup	Effect on 1.2 V domain clocks	Effect on V _{DD} domain clocks	Voltage regulator
Sleep (Sleep now or Sleep-on-exit)	WFI or Return from ISR	Any interrupt	CPU CLK OFF no effect on other clocks or analog clock sources	None	ON
	WFE	Wakeup event			
Stop	SLEEPDEEP bit + WFI, Return from ISR or WFE	Any EXTI line (configured in the EXTI registers, internal and external lines)	All 1.2 V domain clocks OFF	HSI and HSE oscillators OFF	Main regulator or Low-Power regulator (depends on PWR power control register (PWR_CR))
Standby	PDDS bit + SLEEPDEEP bit + WFI, Return from ISR or WFE	WKUP pin rising edge, RTC alarm (Alarm A or Alarm B), RTC Wakeup event, RTC tamper events, RTC time stamp event, external reset in NRST pin, IWDG reset			OFF

5.3.1 Slowing down system clocks

In Run mode the speed of the system clocks (SYSCLK, HCLK, PCLK1, PCLK2) can be reduced by programming the prescaler registers. These prescalers can also be used to slow down peripherals before entering Sleep mode.

For more details refer to [Section 7.3.3: RCC clock configuration register \(RCC_CFGR\)](#).

5.3.2 Peripheral clock gating

In Run mode, the HCLKx and PCLKx for individual peripherals and memories can be stopped at any time to reduce power consumption.

To further reduce power consumption in Sleep mode the peripheral clocks can be disabled prior to executing the WFI or WFE instructions.

Peripheral clock gating is controlled by the AHB1 peripheral clock enable register (RCC_AHB1ENR), AHB2 peripheral clock enable register (RCC_AHB2ENR) (see [Section 7.3.10: RCC AHB1 peripheral clock enable register \(RCC_AHB1ENR\)](#), [Section 7.3.11: RCC AHB2 peripheral clock enable register \(RCC_AHB2ENR\)](#)).

Disabling the peripherals clocks in Sleep mode can be performed automatically by resetting the corresponding bit in RCC_AHBxLPENR and RCC_APBxLPENR registers.

5.3.3 Sleep mode

Entering Sleep mode

The Sleep mode is entered according to [Section : Entering low-power mode](#), when the SLEEPDEEP bit in the Cortex[®]-M4 with FPU System Control register is cleared.

Refer to [Table 15](#) and [Table 16](#) for details on how to enter Sleep mode.

Note: All interrupt pending bits must be cleared before the sleep mode entry.

Exiting Sleep mode

The Sleep mode is exited according to [Section : Exiting low-power mode](#).

Refer to [Table 15](#) and [Table 16](#) for more details on how to exit Sleep mode.

Table 15. Sleep-now entry and exit

Sleep-now mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP = 0, and – No interrupt (for WFI) or event (for WFE) is pending. Refer to the Cortex [®] -M4 with FPU System Control register.
	On Return from ISR while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 1, – No interrupt is pending. Refer to the Cortex [®] -M4 with FPU System Control register.
Mode exit	If WFI or Return from ISR was used for entry: Interrupt: Refer to Table 37: Vector table for STM32F411xC/E If WFE was used for entry and SEVONPEND = 0 Wakeup event: Refer to Section 10.2.3: Wakeup event management If WFE was used for entry and SEVONPEND = 1 Interrupt even when disabled in NVIC: refer to Table 37: Vector table for STM32F411xC/E or Wakeup event (see Section 10.2.3: Wakeup event management).
Wakeup latency	None

Table 16. Sleep-on-exit entry and exit

Sleep-on-exit	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP = 0, and – No interrupt (for WFI) or event (for WFE) is pending. Refer to the Cortex [®] -M4 with FPU System Control register.
	On Return from ISR while: – SLEEPDEEP = 0, and – SLEEPONEXIT = 1, and – No interrupt is pending. Refer to the Cortex [®] -M4 with FPU System Control register.

Table 16. Sleep-on-exit entry and exit (continued)

Sleep-on-exit	Description
Mode exit	Interrupt: refer to Table 37: Vector table for STM32F411xC/E
Wakeup latency	None

5.3.4 Stop mode

The Stop mode is based on the Cortex[®]-M4 with FPU deepsleep mode combined with peripheral clock gating. The voltage regulator can be configured either in normal or low-power mode. In Stop mode, all clocks in the 1.2 V domain are stopped, the PLLs, the HSI and the HSE RC oscillators are disabled. Internal SRAM and register contents are preserved.

Some settings in the PWR_CR register allow to further reduce the power consumption. When the Flash memory is in power-down mode, an additional startup delay is incurred when waking up from Stop mode (see [Table 17: Stop operating modes](#) and [Section 5.4.1: PWR power control register \(PWR_CR\)](#)).

Table 17. Stop operating modes

	Stop mode	MRLV bit	LPLV bit	FPDS bit	LPDS bit	Wakeup latency
Normal mode	STOP MR	0	-	0	0	HSI RC startup time
	STOP MRFPD	0	-	1	0	HSI RC startup time + Flash wakeup time from Deep Power Down mode
	STOP LP	0	0	0	1	HSI RC startup time + regulator wakeup time from LP mode
	STOP LPFPD	-	0	1	1	HSI RC startup time + Flash wakeup time from Deep Power Down mode + regulator wakeup time from LP mode
	STOP MRLV	1	-	-	0	HSI RC startup time + Flash wakeup time from Deep Power Down mode + Main regulator from low voltage mode
	STOP LPLV	-	1	-	1	HSI RC startup time + Flash wakeup time from Deep Power Down mode + regulator wakeup time from Low Voltage LP mode

Entering Stop mode

The Stop mode is entered according to [Section : Entering low-power mode](#), when the SLEEPDEEP bit in the Cortex[®]-M4 with FPU System Control register is set.

Refer to [Table 18](#) for details on how to enter the Stop mode.

To further reduce power consumption in Stop mode, the internal voltage regulator can be put in low-power mode. This is configured by the LPDS bit of the [PWR power control register \(PWR_CR\)](#).

If Flash memory programming is ongoing, the Stop mode entry is delayed until the memory access is finished.

If an access to the APB domain is ongoing, The Stop mode entry is delayed until the APB access is finished.

In Stop mode, the following features can be selected by programming individual control bits:

- Independent watchdog (IWDG): the IWDG is started by writing to its Key register or by hardware option. Once started it cannot be stopped except by a Reset. See [Section 15.3](#) in [Section 15: Independent watchdog \(IWDG\)](#).
- Real-time clock (RTC): this is configured by the RTCEN bit in the [Section 7.3.20: RCC Backup domain control register \(RCC_BDCR\)](#)
- Internal RC oscillator (LSI RC): this is configured by the LSION bit in the [Section 7.3.21: RCC clock control & status register \(RCC_CSR\)](#).
- External 32.768 kHz oscillator (LSE OSC): this is configured by the LSEON bit in the [Section 7.3.20: RCC Backup domain control register \(RCC_BDCR\)](#).

The ADC can also consume power during the Stop mode, unless it is disabled before entering it. To disable it, the ADON bit in the ADC_CR2 register must be written to 0.

Note: *If the application needs to disable the external clock before entering Stop mode, the HSEON bit must first be disabled and the system clock switched to HSI.*

Otherwise, if the HSEON bit is kept enabled while the external clock (external oscillator) can be removed before entering stop mode, the clock security system (CSS) feature must be enabled to detect any external oscillator failure and avoid a malfunction behavior when entering stop mode.

Exiting Stop mode

The Stop mode is exited according to [Section : Exiting low-power mode](#).

Refer to [Table 18](#) for more details on how to exit Stop mode.

When exiting Stop mode by issuing an interrupt or a wakeup event, the HSI RC oscillator is selected as system clock.

When the voltage regulator operates in low-power mode, an additional startup delay is incurred when waking up from Stop mode. By keeping the internal regulator ON during Stop mode, the consumption is higher although the startup time is reduced.

Table 18. Stop mode entry and exit

Stop mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – No interrupt (for WFI) or event (for WFE) is pending, – SLEEPDEEP bit is set in Cortex[®]-M4 with FPU System Control register, – PDDS bit is cleared in Power Control register (PWR_CR), – Select the voltage regulator mode by configuring LPDS bit in PWR_CR.
	On Return from ISR: – No interrupt is pending, – SLEEPDEEP bit is set in Cortex[®]-M4 with FPU System Control register, – SLEEPONEXIT = 1, – PDDS bit is cleared in Power Control register (PWR_CR).
	<i>Note: To enter Stop mode, all EXTI Line pending bits (in Pending register (EXTI_PR)), all peripheral interrupts pending bits, the RTC Alarm (Alarm A and Alarm B), RTC wakeup, RTC tamper, and RTC time stamp flags, must be reset. Otherwise, the Stop mode entry procedure is ignored and program execution continues.</i>
Mode exit	If WFI or Return from ISR was used for entry: Any EXTI lines configured in Interrupt mode (the corresponding EXTI Interrupt vector must be enabled in the NVIC). The interrupt source can be external interrupts or peripherals with wakeup capability. Refer to Table 37: Vector table for STM32F411xC/E. If WFE was used for entry and SEVONPEND = 0 Any EXTI lines configured in event mode. Refer to Section 10.2.3: Wakeup event management on page 205. If WFE was used for entry and SEVONPEND = 1: – Any EXTI lines configured in Interrupt mode (even if the corresponding EXTI Interrupt vector is disabled in the NVIC). The interrupt source can be an external interrupt or a peripheral with wakeup capability. Refer to Table 37: Vector table for STM32F411xC/E. – Wakeup event: refer to Section 10.2.3: Wakeup event management on page 205.
Wakeup latency	Table 17: Stop operating modes

5.3.5 Standby mode

The Standby mode allows to achieve the lowest power consumption. It is based on the Cortex[®]-M4 with FPU deepsleep mode, with the voltage regulator disabled. The 1.2 V domain is consequently powered off. The PLLs, the HSI oscillator and the HSE oscillator are also switched off. SRAM and register contents are lost except for registers in the backup domain (RTC registers and RTC backup register), and Standby circuitry (see [Figure 7](#)).

Entering Standby mode

The Standby mode is entered according to [Section : Entering low-power mode](#), when the SLEEPDEEP bit in the Cortex[®]-M4 with FPU System Control register is set.

Refer to [Table 19](#) for more details on how to enter Standby mode.

In Standby mode, the following features can be selected by programming individual control bits:

- Independent watchdog (IWDG): the IWDG is started by writing to its Key register or by hardware option. Once started it cannot be stopped except by a reset. See [Section 15.3](#) in [Section 15: Independent watchdog \(IWDG\)](#).
- Real-time clock (RTC): this is configured by the RTCEN bit in the backup domain control register (RCC_BDCR)
- Internal RC oscillator (LSI RC): this is configured by the LSION bit in the Control/status register (RCC_CSR).
- External 32.768 kHz oscillator (LSE OSC): this is configured by the LSEON bit in the backup domain control register (RCC_BDCR)

Exiting Standby mode

The Standby mode is exited according to [Section : Exiting low-power mode](#). The SBF status flag in PWR_CR (see [Section 5.4.2: PWR power control/status register \(PWR_CSR\)](#)) indicates that the MCU was in Standby mode. All registers are reset after wakeup from Standby except for PWR_CR.

Refer to [Table 19](#) for more details on how to exit Standby mode.

Table 19. Standby mode entry and exit

Standby mode	Description
Mode entry	WFI (Wait for Interrupt) or WFE (Wait for Event) while: – SLEEPDEEP is set in Cortex[®]-M4 with FPU System Control register, – PDDS bit is set in Power Control register (PWR_CR), – No interrupt (for WFI) or event (for WFE) is pending, – WUF bit is cleared in Power Control register (PWR_CR), – the RTC flag corresponding to the chosen wakeup source (RTC Alarm A, RTC Alarm B, RTC wakeup, Tamper or Timestamp flags) is cleared
	On return from ISR while: – SLEEPDEEP bit is set in Cortex[®]-M4 with FPU System Control register, and – SLEEPONEXIT = 1, and – PDDS bit is set in Power Control register (PWR_CR), and – No interrupt is pending, – WUF bit is cleared in Power Control/Status register (PWR_SR), – The RTC flag corresponding to the chosen wakeup source (RTC Alarm A, RTC Alarm B, RTC wakeup, Tamper or Timestamp flags) is cleared.
Mode exit	WKUP pin rising edge, RTC alarm (Alarm A and Alarm B), RTC wakeup, tamper event, time stamp event, external reset in NRST pin, IWDG reset.
Wakeup latency	Reset phase.

I/O states in Standby mode

In Standby mode, all I/O pins are high impedance except for:

- Reset pad (still available)
- RTC_AF1 pin (PC13) if configured for tamper, time stamp, RTC Alarm out, or RTC clock calibration out
- WKUP pin (PA0), if enabled

Debug mode

By default, the debug connection is lost if the application puts the MCU in Stop or Standby mode while the debug features are used. This is due to the fact that the Cortex®-M4 with FPU core is no longer clocked.

However, by setting some configuration bits in the DBGMCU_CR register, the software can be debugged even when using the low-power modes extensively. For more details, refer to [Section 23.16.1: Debug support for low-power modes](#).

5.3.6 Programming the RTC alternate functions to wake up the device from the Stop and Standby modes

The MCU can be woken up from a low-power mode by an RTC alternate function.

The RTC alternate functions are the RTC alarms (Alarm A and Alarm B), RTC wakeup, RTC tamper event detection and RTC time stamp event detection.

These RTC alternate functions can wake up the system from the Stop and Standby low-power modes.

The system can also wake up from low-power modes without depending on an external interrupt (Auto-wakeup mode), by using the RTC alarm or the RTC wakeup events.

The RTC provides a programmable time base for waking up from the Stop or Standby mode at regular intervals.

For this purpose, two of the three alternate RTC clock sources can be selected by programming the RTCSEL[1:0] bits in the [Section 7.3.20: RCC Backup domain control register \(RCC_BDCR\)](#):

- Low-power 32.768 kHz external crystal oscillator (LSE OSC)
This clock source provides a precise time base with a very low-power consumption (additional consumption of less than 1 µA under typical conditions)
- Low-power internal RC oscillator (LSI RC)
This clock source has the advantage of saving the cost of the 32.768 kHz crystal. This internal RC oscillator is designed to use minimum power.

RTC alternate functions to wake up the device from the Stop mode

- To wake up the device from the Stop mode with an RTC alarm event, it is necessary to:
 - a) Configure the EXTI Line 17 to be sensitive to rising edges (Interrupt or Event modes)
 - b) Enable the RTC Alarm Interrupt in the RTC_CR register
 - c) Configure the RTC to generate the RTC alarm
- To wake up the device from the Stop mode with an RTC tamper or time stamp event, it is necessary to:
 - a) Configure the EXTI Line 21 to be sensitive to rising edges (Interrupt or Event modes)
 - b) Enable the RTC time stamp Interrupt in the RTC_CR register or the RTC tamper interrupt in the RTC_TAFCR register
 - c) Configure the RTC to detect the tamper or time stamp event
- To wake up the device from the Stop mode with an RTC wakeup event, it is necessary to:
 - a) Configure the EXTI Line 22 to be sensitive to rising edges (Interrupt or Event modes)
 - b) Enable the RTC wakeup interrupt in the RTC_CR register
 - c) Configure the RTC to generate the RTC Wakeup event

RTC alternate functions to wake up the device from the Standby mode

- To wake up the device from the Standby mode with an RTC alarm event, it is necessary to:
 - a) Enable the RTC alarm interrupt in the RTC_CR register
 - b) Configure the RTC to generate the RTC alarm
- To wake up the device from the Standby mode with an RTC tamper or time stamp event, it is necessary to:
 - a) Enable the RTC time stamp interrupt in the RTC_CR register or the RTC tamper interrupt in the RTC_TAFCR register
 - b) Configure the RTC to detect the tamper or time stamp event
- To wake up the device from the Standby mode with an RTC wakeup event, it is necessary to:
 - a) Enable the RTC wakeup interrupt in the RTC_CR register
 - b) Configure the RTC to generate the RTC wakeup event

Safe RTC alternate function wakeup flag clearing sequence

If the selected RTC alternate function is set before the PWR wakeup flag (WUTF) is cleared, it will not be detected on the next event as detection is made once on the rising edge.

To avoid bouncing on the pins onto which the RTC alternate functions are mapped, and exit correctly from the Stop and Standby modes, it is recommended to follow the sequence below before entering the Standby mode:

- When using RTC alarm to wake up the device from the low-power modes:
 - a) Disable the RTC alarm interrupt (ALRAIE or ALRBIE bits in the RTC_CR register)
 - b) Clear the RTC alarm (ALRAF/ALRBF) flag

- c) Clear the PWR Wakeup (WUF) flag
 - d) Enable the RTC alarm interrupt
 - e) Re-enter the low-power mode
- When using RTC wakeup to wake up the device from the low-power modes:
 - a) Disable the RTC Wakeup interrupt (WUTIE bit in the RTC_CR register)
 - b) Clear the RTC Wakeup (WUTF) flag
 - c) Clear the PWR Wakeup (WUF) flag
 - d) Enable the RTC Wakeup interrupt
 - e) Re-enter the low-power mode
- When using RTC tamper to wake up the device from the low-power modes:
 - a) Disable the RTC tamper interrupt (TAMPIE bit in the RTC_TAFCR register)
 - b) Clear the Tamper (TAMP1F/TSF) flag
 - c) Clear the PWR Wakeup (WUF) flag
 - d) Enable the RTC tamper interrupt
 - e) Re-enter the low-power mode
- When using RTC time stamp to wake up the device from the low-power modes:
 - a) Disable the RTC time stamp interrupt (TSIE bit in RTC_CR)
 - b) Clear the RTC time stamp (TSF) flag
 - c) Clear the PWR Wakeup (WUF) flag
 - d) Enable the RTC TimeStamp interrupt
 - e) Re-enter the low-power mode

5.4 Power control registers

5.4.1 PWR power control register (PWR_CR)

Address offset: 0x00

Reset value: 0x0000 8000 (reset by wakeup from Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved										FISSR	FMSSR	Reserved			
										rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VOS		ADCD1	Res	MRLV DS	LPLV DS	FPDS	DBP	PLS[2:0]			PVDE	CSBF	CWUF	PDDS	LPDS
rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	w	w	rw	rw

Bits 31:22 Reserved, must be kept at reset value.

Bit 21 **FISSR**: Flash Interface Stop while System Run

0: Flash Interface clock run (Default value).

1: Flash Interface clock off.

Note: This bit could not be set while executing with the Flash itself. It should be done with specific routine executed from RAM.

Bit 20 **FMSSR**: Flash Memory Sleep System Run.

0: Flash standard mode (Default value)

1: Flash forced to be in STOP or DeepPower Down mode (depending of **FPDS** value bit) by hardware.

Note: This bit could not be set while executing with the Flash itself. It should be done with specific routine executed from RAM.

Bits 19:16 Reserved, must be kept at reset value.

Bits 15:14 **VOS[1:0]**: Regulator voltage scaling output selection

These bits control the main internal voltage regulator output voltage to achieve a trade-off between performance and power consumption when the device does not operate at the maximum frequency (refer to the corresponding datasheet for more details).

These bits can be modified only when the PLL is OFF. The new value programmed is active only when the PLL is ON. When the PLL is OFF, the voltage regulator is set to scale 3 independently of the VOS register content.

00: Reserved (Scale 3 mode selected)

01: Scale 3 mode <= 64 MHz

10: Scale 2 mode (reset value) <= 84 MHz

11: Scale 1 mode <= 100 MHz

Bit 13 **ADCD1**:

0: No effect.

1: Refer to AN4073 for details on how to use this bit.

Note: This bit can only be set when operating at supply voltage range 2.7 to 3.6V and when the Prefetch is OFF.

Bit 12 Reserved, must be kept at reset value.

Bit 11 **MRLVDS**: Main regulator Low Voltage in Deep Sleep

0: Main regulator in Voltage scale 3 when the device is in Stop mode.

1: Main regulator in Low Voltage and Flash memory in Deep Sleep mode when the device is in Stop mode.

Bit 10 **LPLVDS**: Low-power regulator Low Voltage in Deep Sleep

0: Low-power regulator on if LPDS bit is set when the device is in Stop mode.

1: Low-power regulator in Low Voltage and Flash memory in Deep Sleep mode if LPDS bit is set when device is in Stop mode.

Bit 9 **FPDS**: Flash power-down in Stop mode

When set, the Flash memory enters power-down mode when the device enters Stop mode.

This allows to achieve a lower consumption in stop mode but a longer restart time.

0: Flash memory not in power-down when the device is in Stop mode or the Flash Memory is in Stop mode (FMSSR bit)

1: Flash memory in power-down when the device is in Stop mode or the Flash Memory is in Stop mode (FMSSR bit)

Bit 8 **DBP**: Disable backup domain write protection

In reset state, the RCC_BDCR register, the RTC registers (including the backup registers), and the BRE bit of the PWR_CSR register, are protected against parasitic write access. This bit must be set to enable write access to these registers.

0: Access to RTC and RTC Backup registers.

1: Access to RTC and RTC Backup registers.

Bits 7:5 **PLS[2:0]**: PVD level selection

These bits are written by software to select the voltage threshold detected by the Power Voltage Detector

000: 2.2 V

001: 2.3 V

010: 2.4 V

011: 2.5 V

100: 2.6 V

101: 2.7 V

110: 2.8 V

111: 2.9 V

Note: Refer to the electrical characteristics of the datasheet for more details.

Bit 4 **PVDE**: Power voltage detector enable

This bit is set and cleared by software.

0: PVD disabled

1: PVD enabled

Bit 3 **CSBF**: Clear standby flag

This bit is always read as 0.

0: No effect.

1: Clear the SBF Standby Flag (write).

Bit 2 **CWUF**: Clear wakeup flag

This bit is always read as 0.

0: No effect.

1: Clear the WUF Wakeup Flag after 2 System clock cycles.

Bit 1 **PDDS**: Power-down deepsleep

This bit is set and cleared by software. It works together with the LPDS bit.

0: Enter Stop mode when the CPU enters deepsleep. The regulator status depends on the LPDS bit.

1: Enter Standby mode when the CPU enters deepsleep.

Bit 0 **LPDS**: Low-power deepsleep

This bit is set and cleared by software. It works together with the PDDS bit.

0: Voltage regulator on during Stop mode.

1: Low-power Voltage regulator on during Stop mode.

5.4.2 PWR power control/status register (PWR_CSR)

Address offset: 0x04

Reset value: 0x0000 0000 (not reset by wakeup from Standby mode)

Additional APB cycles are needed to read this register versus a standard APB read.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	VOS RDY	Reserved				BRE	EWUP	Reserved				BRR	PVDO	SBF	WUF
	r					rw	rw					r	r	r	r

Bits 31:15 Reserved, must be kept at reset value.

Bit 14 **VOSRDY**: Regulator voltage scaling output selection ready bit

0: Not ready

1: Ready

Bits 13:10 Reserved, must be kept at reset value.

Bit 9 **BRE**: Backup regulator enable

When set, the Backup regulator (used to maintain the backup domain content) is enabled. If BRE is reset, the backup regulator is switched off. Once set, the application must wait that the Backup Regulator Ready flag (BRR) is set to indicate that the data written into the backup registers will be maintained in the Standby and V_{BAT} modes.

0: Backup regulator disabled

1: Backup regulator enabled

Note: This bit is not reset when the device wakes up from Standby mode, by a system reset, or by a power reset.

Bit 8 **EWUP**: Enable WKUP pin

This bit is set and cleared by software.

0: WKUP pin is used for general purpose I/O. An event on the WKUP pin does not wakeup the device from Standby mode.

1: WKUP pin is used for wakeup from Standby mode and forced in input pull down configuration (rising edge on WKUP pin wakes-up the system from Standby mode).

Note: This bit is reset by a system reset.

Bits 7:4 Reserved, must be kept at reset value.

Bit 3 **BRR**: Backup regulator ready

Set by hardware to indicate that the Backup Regulator is ready.

0: Backup Regulator not ready

1: Backup Regulator ready

Note: This bit is not reset when the device wakes up from Standby mode or by a system reset or power reset.

Bit 2 **PVDO**: PVD output

This bit is set and cleared by hardware. It is valid only if PVD is enabled by the PVDE bit.

0: V_{DD} is higher than the PVD threshold selected with the PLS[2:0] bits.

1: V_{DD} is lower than the PVD threshold selected with the PLS[2:0] bits.

Note: The PVD is stopped by Standby mode. For this reason, this bit is equal to 0 after Standby or reset until the PVDE bit is set.

Bit 1 **SBF**: Standby flag

This bit is set by hardware and cleared only by a POR/PDR (power-on reset/power-down reset) or by setting the CSBF bit in the PWR_CR register.

0: Device has not been in Standby mode

1: Device has been in Standby mode

Bit 0 **WUF**: Wakeup flag

This bit is set by hardware and cleared either by a system reset or by setting the CWUF bit in the PWR_CR register.

0: No wakeup event occurred

1: A wakeup event was received from the WKUP pin or from the RTC alarm (Alarm A or Alarm B), RTC Tamper event, RTC TimeStamp event or RTC Wakeup).

Note: An additional wakeup event is detected if the WKUP pin is enabled (by setting the EWUP bit) when the WKUP pin level is already high.

5.5 PWR register map

The following table summarizes the PWR registers.

Table 20. PWR - register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x000	PWR_CR	Reserved											FISSR	FMSR	Reser ved			VOS[1:0]	ADCD1		MRLVDS	LPLVDS	FPDS	DBP	PLS[2:0]			PVDE	CSBF	CWUF	PDDS	LPDS		
	Reset value												0	0					1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004	PWR_CSR	Reserved																		VOSRDY	Reserved					BRE	EWUP	Reserved			BRR	PVDO	SBF	WUF
	Reset value																			0						0	0				0	0	0	0

Refer to [Table 3 on page 42](#) for the register boundary addresses.

6 Reset and clock control (RCC) for STM32F411xC/E

6.1 Reset

There are three types of reset, defined as system Reset, power Reset and backup domain Reset.

6.1.1 System reset

A system reset sets all registers to their reset values except the reset flags in the clock controller CSR register and the registers in the Backup domain.

A system reset is generated when one of the following events occurs:

1. A low level on the NRST pin (external reset)
2. Window watchdog end of count condition (WWDG reset)
3. Independent watchdog end of count condition (IWDG reset)
4. A software reset (SW reset) (see [Software reset](#))
5. Low-power management reset (see [Low-power management reset](#))

Software reset

The reset source can be identified by checking the reset flags in the [RCC clock control & status register \(RCC_CSR\)](#).

The SYSRESETREQ bit in Cortex[®]-M4 with FPU Application Interrupt and Reset Control Register must be set to force a software reset on the device. Refer to the Cortex[®]-M4 with FPU technical reference manual for more details.

Low-power management reset

There are two ways of generating a low-power management reset:

1. Reset generated when entering the Standby mode:
This type of reset is enabled by resetting the nRST_STDBY bit in the user option bytes. In this case, whenever a Standby mode entry sequence is successfully executed, the device is reset instead of entering the Standby mode.
2. Reset when entering the Stop mode:
This type of reset is enabled by resetting the nRST_STOP bit in the user option bytes. In this case, whenever a Stop mode entry sequence is successfully executed, the device is reset instead of entering the Stop mode.

For further information on the user option bytes, refer to the STM32F411xC/E Flash programming manual available from your ST sales office.

6.1.2 Power reset

A power reset is generated when one of the following events occurs:

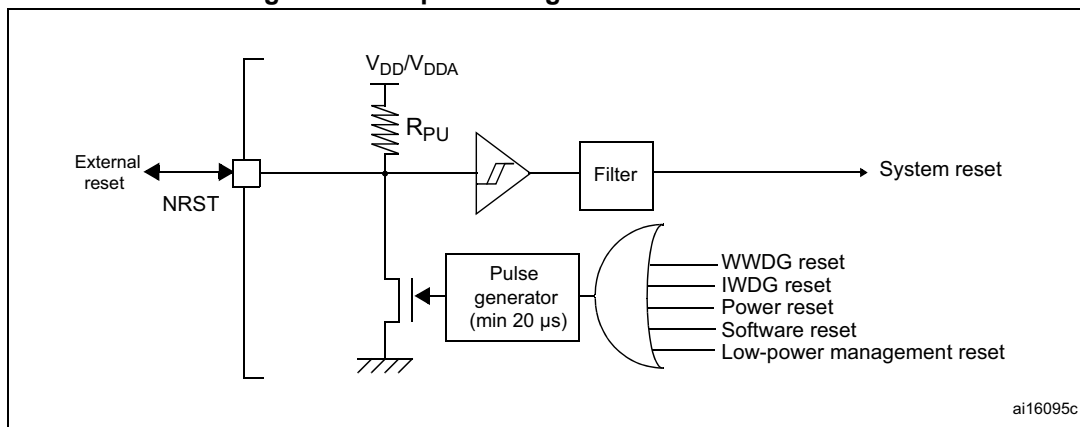
1. Power-on/power-down reset (POR/PDR reset) or brownout (BOR) reset
2. When exiting the Standby mode

A power reset sets all registers to their reset values except the Backup domain.

These sources act on the NRST pin and it is always kept low during the delay phase. The RESET service routine vector is fixed at address 0x0000_0004 in the memory map.

The system reset signal provided to the device is output on the NRST pin. The pulse generator guarantees a minimum reset pulse duration of 20 μ s for each internal reset source. In case of an external reset, the reset pulse is generated while the NRST pin is asserted low.

Figure 11. Simplified diagram of the reset circuit



6.1.3 Backup domain reset

The backup domain reset sets all RTC registers and the RCC_BDCR register to their reset values.

A backup domain reset is generated when one of the following events occurs:

1. Software reset, triggered by setting the BDRST bit in the *RCC Backup domain control register (RCC_BDCR)*.
2. V_{DD} or V_{BAT} power on, if both supplies have previously been powered off.

6.2 Clocks

Three different clock sources can be used to drive the system clock (SYSCLK):

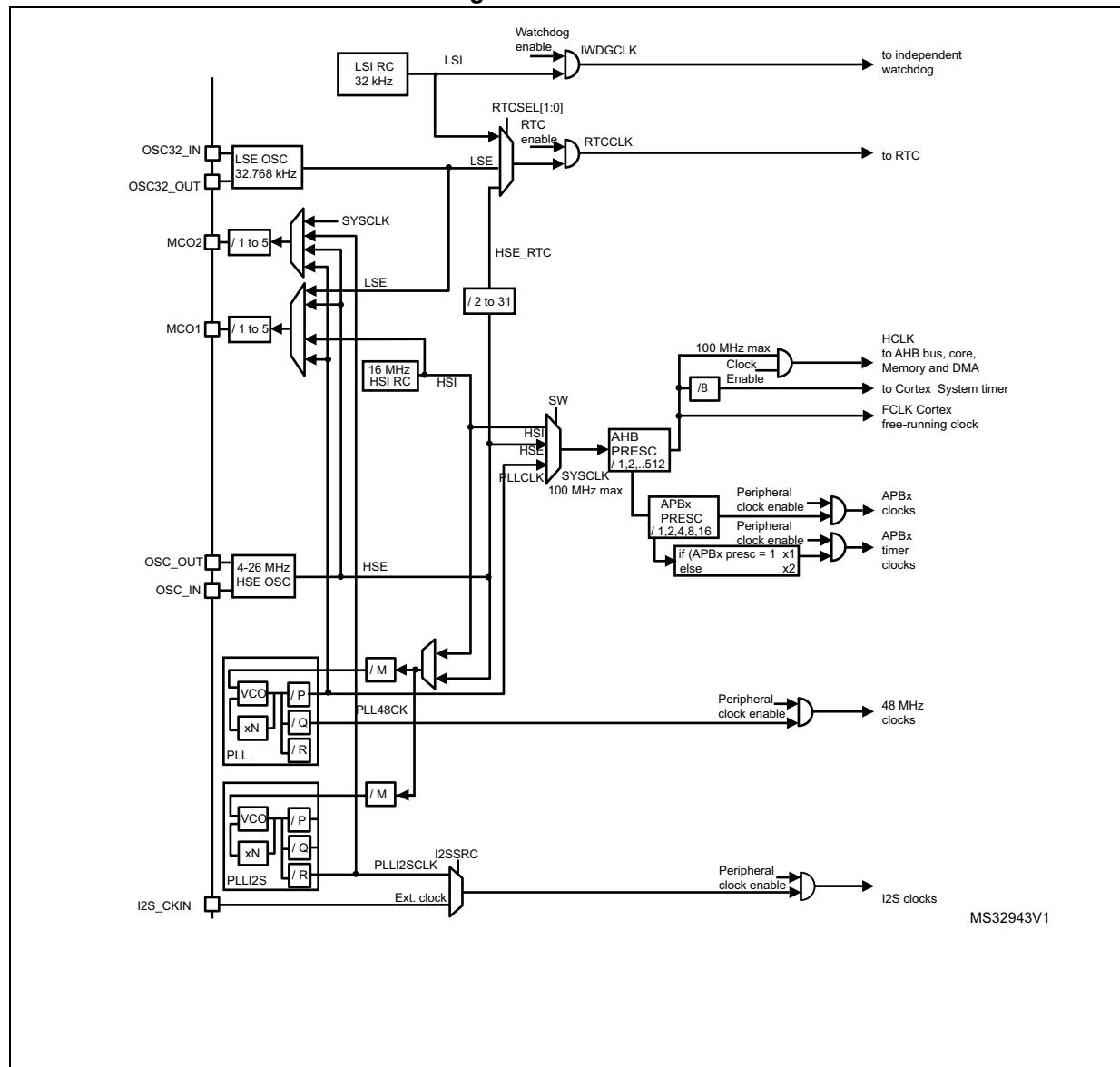
- HSI oscillator clock
- HSE oscillator clock
- Main PLL (PLL) clock

The devices have the two following secondary clock sources:

- 32 kHz low-speed internal RC (LSI RC) which drives the independent watchdog and, optionally, the RTC used for Auto-wakeup from the Stop/Standby mode.
- 32.768 kHz low-speed external crystal (LSE crystal) which optionally drives the RTC clock (RTCCLK)

Each clock source can be switched on or off independently when it is not used, to optimize power consumption.

Figure 12. Clock tree



1. For full details about the internal and external clock source characteristics, refer to the Electrical characteristics section in the device datasheet.

The clock controller provides a high degree of flexibility to the application in the choice of the external crystal or the oscillator to run the core and peripherals at the highest frequency and, guarantee the appropriate frequency for peripherals that need a specific clock like USB OTG FS, I2S and SDIO.

Several prescalers are used to configure the AHB frequency, the high-speed APB (APB2) and the low-speed APB (APB1) domains. The maximum frequency of the AHB domain is 100 MHz. The maximum allowed frequency of the high-speed APB2 domain is 100 MHz. The maximum allowed frequency of the low-speed APB1 domain is 50 MHz

All peripheral clocks are derived from the system clock (SYSCLK) except for:

- The USB OTG FS clock (48 MHz) and the SDIO clock (≤ 48 MHz) which are coming from a specific output of PLL (PLL48CLK)
- The I2S clock
To achieve high-quality audio performance, the I2S clock can be derived either from a specific PLL (PLL12S) or from an external clock mapped on the I2S_CKIN pin. For more information about I2S clock frequency and precision, refer to [Section 20.4.4: Clock generator](#).

The RCC feeds the external clock of the Cortex System Timer (SysTick) with the AHB clock (HCLK) divided by 8. The SysTick can work either with this clock or with the Cortex clock (HCLK), configurable in the SysTick control and status register.

The timer clock frequencies for STM32F411xC/E are automatically set by hardware. There are two cases:

1. If the APB prescaler is 1, the timer clock frequencies are set to the same frequency as that of the APB domain to which the timers are connected.
2. Otherwise, they are set to twice ($\times 2$) the frequency of the APB domain to which the timers are connected.

The timer clock frequencies are automatically set by hardware. There are two cases depending on the value of TIMPRE bit in RCC_DCKCFGR register:

- If TIMPRE bit is reset:
If the APB prescaler is configured to a division factor of 1, the timer clock frequencies (TIMxCLK) are set to HCLK. Otherwise, the timer clock frequencies are twice the frequency of the APB domain to which the timers are connected: $TIMxCLK = 2 \times PCLKx$.
- If TIMPRE bit is set:
If the APB prescaler is configured to a division factor of 1 or 2, the timer clock frequencies (TIMxCLK) are set to HCLK. Otherwise, the timer clock frequencies is four times the frequency of the APB domain to which the timers are connected: $TIMxCLK = 4 \times PCLKx$.

FCLK acts as Cortex[®]-M4 with FPU free-running clock. For more details, refer to the Cortex[®]-M4 with FPU technical reference manual.

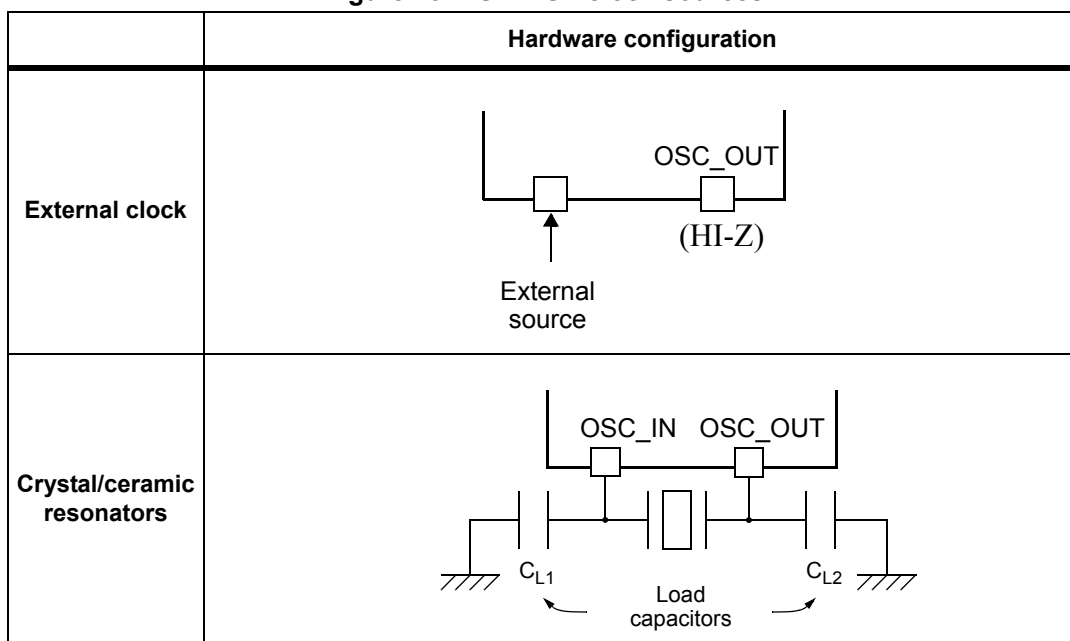
6.2.1 HSE clock

The high speed external clock signal (HSE) can be generated from two possible clock sources:

- HSE external crystal/ceramic resonator
- HSE external user clock

The resonator and the load capacitors have to be placed as close as possible to the oscillator pins in order to minimize output distortion and startup stabilization time. The loading capacitance values must be adjusted according to the selected oscillator.

Figure 13. HSE/ LSE clock sources



External source (HSE bypass)

In this mode, an external clock source must be provided. You select this mode by setting the HSEBYP and HSEON bits in the [RCC clock control register \(RCC_CR\)](#). The external clock signal (square, sinus or triangle) with ~50% duty cycle has to drive the OSC_IN pin while the OSC_OUT pin should be left HI-Z. See [Figure 13](#).

External crystal/ceramic resonator (HSE crystal)

The HSE has the advantage of producing a very accurate rate on the main clock.

The associated hardware configuration is shown in [Figure 13](#). Refer to the electrical characteristics section of the *datasheet* for more details.

The HSERDY flag in the [RCC clock control register \(RCC_CR\)](#) indicates if the high-speed external oscillator is stable or not. At startup, the clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock interrupt register \(RCC_CIR\)](#).

The HSE Crystal can be switched on and off using the HSEON bit in the [RCC clock control register \(RCC_CR\)](#).

6.2.2 HSI clock

The HSI clock signal is generated from an internal 16 MHz RC oscillator and can be used directly as a system clock, or used as PLL input.

The HSI RC oscillator has the advantage of providing a clock source at low cost (no external components). It also has a faster startup time than the HSE crystal oscillator however, even with calibration the frequency is less accurate than an external crystal oscillator or ceramic resonator.

Calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations, this is why each device is factory calibrated by ST for 1% accuracy at $T_A = 25\text{ }^{\circ}\text{C}$.

After reset, the factory calibration value is loaded in the HSICAL[7:0] bits in the [RCC clock control register \(RCC_CR\)](#).

If the application is subject to voltage or temperature variations this may affect the RC oscillator speed. You can trim the HSI frequency in the application using the HSITRIM[4:0] bits in the [RCC clock control register \(RCC_CR\)](#).

The HSIRDY flag in the [RCC clock control register \(RCC_CR\)](#) indicates if the HSI RC is stable or not. At startup, the HSI RC output clock is not released until this bit is set by hardware.

The HSI RC can be switched on and off using the HSION bit in the [RCC clock control register \(RCC_CR\)](#).

The HSI signal can also be used as a backup source (Auxiliary clock) if the HSE crystal oscillator fails. Refer to [Section 6.2.7: Clock security system \(CSS\) on page 98](#).

6.2.3 PLL configuration

The STM32F411xC/E devices feature two PLLs:

- A main PLL (PLL) clocked by the HSE or HSI oscillator and featuring two different output clocks:
 - The first output is used to generate the high speed system clock (up to 100 MHz)
 - The second output is used to generate the clock for the USB OTG FS (48 MHz) and the SDIO (≤ 50 MHz).
- A dedicated PLL (PLLI2S) used to generate an accurate clock to achieve high-quality audio performance on the I2S interface.

Since the main-PLL configuration parameters cannot be changed once PLL is enabled, it is recommended to configure PLL before enabling it (selection of the HSI or HSE oscillator as PLL clock source, and configuration of division factors M, P, Q and multiplication factor N).

The PLLI2S uses the same input clock as the main PLL (HSI or HSE). However, the PLLI2S has dedicated enable/disable and division factors configuration bits. Refer to [Section 6.3.1: RCC clock control register \(RCC_CR\)](#), [Section 6.3.2: RCC PLL configuration register \(RCC_PLLCFGR\)](#) and [Section 6.3.20: RCC PLLI2S configuration register \(RCC_PLLI2SCFGR\)](#). Once the PLLI2S is enabled, the configuration parameters cannot be changed.

The two PLLs are disabled by hardware when entering Stop and Standby modes, or when an HSE failure occurs when HSE or PLL (clocked by HSE) are used as system clock. [RCC PLL configuration register \(RCC_PLLCFGR\)](#) and [RCC clock configuration register](#)

([RCC_CFGR](#)) can be used to configure PLL and PLLI2S, respectively.

6.2.4 LSE clock

The LSE clock is generated using a 32.768kHz low speed external crystal or ceramic resonator. It has the advantage providing a low-power but highly accurate clock source to the real-time clock peripheral (RTC) for clock/calendar or other timing functions.

The LSE oscillator is switched on and off using the LSEON bit in [RCC Backup domain control register \(RCC_BDCR\)](#).

The LSERDY flag in the [RCC Backup domain control register \(RCC_BDCR\)](#) indicates if the LSE crystal is stable or not. At startup, the LSE crystal output clock signal is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock interrupt register \(RCC_CIR\)](#).

External source (LSE bypass)

In this mode, an external clock source must be provided. It must have a frequency up to 1 MHz. You select this mode by setting the LSEBYP and LSEON bits in the [RCC Backup domain control register \(RCC_BDCR\)](#). The external clock signal (square, sinus or triangle) with ~50% duty cycle has to drive the OSC32_IN pin while the OSC32_OUT pin should be left HI-Z. See [Figure 13](#).

6.2.5 LSI clock

The LSI RC acts as an low-power clock source that can be kept running in Stop and Standby mode for the independent watchdog (IWDG) and Auto-wakeup unit (AWU). The clock frequency is around 32 kHz. For more details, refer to the electrical characteristics section of the datasheets.

The LSI RC can be switched on and off using the LSION bit in the [RCC clock control & status register \(RCC_CSR\)](#).

The LSIRDY flag in the [RCC clock control & status register \(RCC_CSR\)](#) indicates if the low-speed internal oscillator is stable or not. At startup, the clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock interrupt register \(RCC_CIR\)](#).

6.2.6 System clock (SYSCLK) selection

After a system reset, the HSI oscillator is selected as the system clock. When a clock source is used directly or through PLL as the system clock, it is not possible to stop it.

A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source that is not yet ready is selected, the switch occurs when the clock source is ready. Status bits in the [RCC clock control register \(RCC_CR\)](#) indicate which clock(s) is (are) ready and which clock is currently used as the system clock.

6.2.7 Clock security system (CSS)

The clock security system can be activated by software. In this case, the clock detector is enabled after the HSE oscillator startup delay, and disabled when this oscillator is stopped.

If a failure is detected on the HSE clock, this oscillator is automatically disabled, a clock failure event is sent to the break inputs of advanced-control timer TIM1, and an interrupt is generated to inform the software about the failure (clock security system interrupt CSSI), allowing the MCU to perform rescue operations. The CSSI is linked to the Cortex®-M4 with FPU NMI (non-maskable interrupt) exception vector.

Note: When the CSS is enabled, if the HSE clock happens to fail, the CSS generates an interrupt, which causes the automatic generation of an NMI. The NMI is executed indefinitely unless the CSS interrupt pending bit is cleared. As a consequence, the application has to clear the CSS interrupt in the NMI ISR by setting the CSSC bit in the Clock interrupt register (RCC_CIR).

If the HSE oscillator is used directly or indirectly as the system clock (indirectly meaning that it is directly used as PLL input clock, and that PLL clock is the system clock) and a failure is detected, then the system clock switches to the HSI oscillator and the HSE oscillator is disabled.

If the HSE oscillator clock was the clock source of PLL used as the system clock when the failure occurred, PLL is also disabled. In this case, if the PLLI2S was enabled, it is also disabled when the HSE fails.

6.2.8 RTC/AWU clock

Once the RTCCLK clock source has been selected, the only possible way of modifying the selection is to reset the power domain.

The RTCCLK clock source can be either the HSE 1 MHz (HSE divided by a programmable prescaler), the LSE or the LSI clock. This is selected by programming the RTCSEL[1:0] bits in the [RCC Backup domain control register \(RCC_BDCR\)](#) and the RTCPRE[4:0] bits in [RCC clock configuration register \(RCC_CFGR\)](#). This selection cannot be modified without resetting the Backup domain.

If the LSE is selected as the RTC clock, the RTC will work normally if the backup or the system supply disappears. If the LSI is selected as the AWU clock, the AWU state is not guaranteed if the system supply disappears. If the HSE oscillator divided by a value between 2 and 31 is used as the RTC clock, the RTC state is not guaranteed if the backup or the system supply disappears.

The LSE clock is in the Backup domain, whereas the HSE and LSI clocks are not. As a consequence:

- If LSE is selected as the RTC clock:
 - The RTC continues to work even if the V_{DD} supply is switched off, provided the V_{BAT} supply is maintained.
- If LSI is selected as the Auto-wakeup unit (AWU) clock:
 - The AWU state is not guaranteed if the V_{DD} supply is powered off. Refer to [Section 6.2.5: LSI clock on page 97](#) for more details on LSI calibration.
- If the HSE clock is used as the RTC clock:
 - The RTC state is not guaranteed if the V_{DD} supply is powered off or if the internal voltage regulator is powered off (removing power from the 1.2 V domain).

Note: To read the RTC calendar register when the APB1 clock frequency is less than seven times the RTC clock frequency ($f_{APB1} < 7 \times f_{RTCLK}$), the software must read the calendar time and date registers twice. The data are correct if the second read access to `RTC_TR` gives the same result than the first one. Otherwise a third read access must be performed.

6.2.9 Watchdog clock

If the independent watchdog (IWDG) is started by either hardware option or software access, the LSI oscillator is forced ON and cannot be disabled. After the LSI oscillator temporization, the clock is provided to the IWDG.

6.2.10 Clock-out capability

Two microcontroller clock output (MCO) pins are available:

- MCO1
You can output four different clock sources onto the MCO1 pin (PA8) using the configurable prescaler (from 1 to 5):
 - HSI clock
 - LSE clock
 - HSE clock
 - PLL clockThe desired clock source is selected using the `MCO1PRE[2:0]` and `MCO1[1:0]` bits in the [RCC clock configuration register \(RCC_CFGR\)](#).
- MCO2
You can output four different clock sources onto the MCO2 pin (PC9) using the configurable prescaler (from 1 to 5):
 - HSE clock
 - PLL clock
 - System clock (SYSCLK)
 - PLLI2S clockThe desired clock source is selected using the `MCO2PRE[2:0]` and `MCO2` bits in the [RCC clock configuration register \(RCC_CFGR\)](#).

For the different MCO pins, the corresponding GPIO port has to be programmed in alternate function mode.

The selected clock to output onto MCO must not exceed 100 MHz (the maximum I/O speed).

6.2.11 Internal/external clock measurement using TIM5/TIM11

It is possible to indirectly measure the frequencies of all on-board clock source generators by means of the input capture of TIM5 channel4 and TIM11 channel1 as shown in [Figure 14](#) and [Figure 15](#).

Internal/external clock measurement using TIM5 channel4

TIM5 has an input multiplexer which allows choosing whether the input capture is triggered by the I/O or by an internal clock. This selection is performed through the `TI4_RMP[1:0]` bits in the `TIM5_OR` register.

The primary purpose of having the LSE connected to the channel4 input capture is to be able to precisely measure the HSI (this requires to have the HSI used as the system clock source). The number of HSI clock counts between consecutive edges of the LSE signal provides a measurement of the internal clock period. Taking advantage of the high precision of LSE crystals (typically a few tens of ppm) we can determine the internal clock frequency with the same resolution, and trim the source to compensate for manufacturing-process and/or temperature- and voltage-related frequency deviations.

The HSI oscillator has dedicated, user-accessible calibration bits for this purpose.

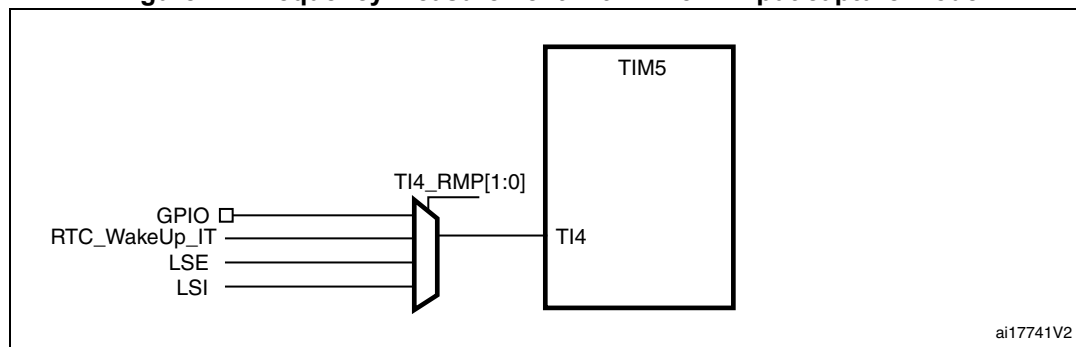
The basic concept consists in providing a relative measurement (e.g. HSI/LSE ratio): the precision is therefore tightly linked to the ratio between the two clock sources. The greater the ratio, the better the measurement.

It is also possible to measure the LSI frequency: this is useful for applications that do not have a crystal. The ultralow-power LSI oscillator has a large manufacturing process deviation: by measuring it versus the HSI clock source, it is possible to determine its frequency with the precision of the HSI. The measured value can be used to have more accurate RTC time base timeouts (when LSI is used as the RTC clock source) and/or an IWDG timeout with an acceptable accuracy.

Use the following procedure to measure the LSI frequency:

1. Enable the TIM5 timer and configure channel4 in Input capture mode.
2. Set the TI4_RMP bits in the TIM5_OR register to 0x01 to connect the LSI clock internally to TIM5 channel4 input capture for calibration purposes.
3. Measure the LSI clock frequency using the TIM5 capture/compare 4 event or interrupt.
4. Use the measured LSI frequency to update the prescaler of the RTC depending on the desired time base and/or to compute the IWDG timeout.

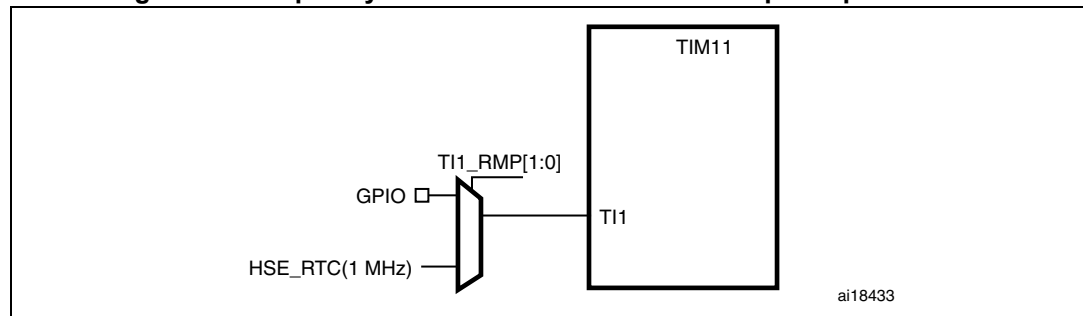
Figure 14. Frequency measurement with TIM5 in Input capture mode



Internal/external clock measurement using TIM11 channel1

TIM11 has an input multiplexer which allows choosing whether the input capture is triggered by the I/O or by an internal clock. This selection is performed through TI1_RMP [1:0] bits in the TIM11_OR register. The HSE_RTC clock (HSE divided by a programmable prescaler) is connected to channel 1 input capture to have a rough indication of the external crystal frequency. This requires that the HSI is the system clock source. This can be useful for instance to ensure compliance with the IEC 60730/IEC 61335 standards which require to be able to determine harmonic or subharmonic frequencies ($-50/+100\%$ deviations).

Figure 15. Frequency measurement with TIM11 in Input capture mode



6.3 RCC registers

Refer to [Section 1.1: List of abbreviations for registers](#) for a list of abbreviations used in register descriptions.

6.3.1 RCC clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX81 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLI2S RDY	PLLI2S ON	PLL RDY	PLL ON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
				r	rw	r	rw					rw	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **PLLI2SRDY**: PLLI2S clock ready flag

Set by hardware to indicate that the PLLI2S is locked.

0: PLLI2S unlocked

1: PLLI2S locked

Bit 26 **PLLI2SON**: PLLI2S enable

Set and cleared by software to enable PLLI2S.

Cleared by hardware when entering Stop or Standby mode.

0: PLLI2S OFF

1: PLLI2S ON

Bit 25 **PLLRDY**: Main PLL (PLL) clock ready flag

Set by hardware to indicate that PLL is locked.

0: PLL unlocked

1: PLL locked

Bit 24 **PLLON**: Main PLL (PLL) enable

Set and cleared by software to enable PLL.

Cleared by hardware when entering Stop or Standby mode. This bit cannot be reset if PLL clock is used as the system clock.

0: PLL OFF

1: PLL ON

Bits 23:20 Reserved, must be kept at reset value.

Bit 19 **CSSON**: Clock security system enable

Set and cleared by software to enable the clock security system. When CSSON is set, the clock detector is enabled by hardware when the HSE oscillator is ready, and disabled by hardware if an oscillator failure is detected.

0: Clock security system OFF (Clock detector OFF)

1: Clock security system ON (Clock detector ON if HSE oscillator is stable, OFF if not)

Bit 18 HSEBYP: HSE clock bypass

Set and cleared by software to bypass the oscillator with an external clock. The external clock must be enabled with the HSEON bit, to be used by the device.
The HSEBYP bit can be written only if the HSE oscillator is disabled.

0: HSE oscillator not bypassed

1: HSE oscillator bypassed with an external clock

Bit 17 HSERDY: HSE clock ready flag

Set by hardware to indicate that the HSE oscillator is stable. After the HSEON bit is cleared, HSERDY goes low after 6 HSE oscillator clock cycles.

0: HSE oscillator not ready

1: HSE oscillator ready

Bit 16 HSEON: HSE clock enable

Set and cleared by software.

Cleared by hardware to stop the HSE oscillator when entering Stop or Standby mode. This bit cannot be reset if the HSE oscillator is used directly or indirectly as the system clock.

0: HSE oscillator OFF

1: HSE oscillator ON

Bits 15:8 HSICAL[7:0]: Internal high-speed clock calibration

These bits are initialized automatically at startup.

Bits 7:3 HSITRIM[4:0]: Internal high-speed clock trimming

These bits provide an additional user-programmable trimming value that is added to the HSICAL[7:0] bits. It can be programmed to adjust to variations in voltage and temperature that influence the frequency of the internal HSI RC.

Bit 2 Reserved, must be kept at reset value.**Bit 1 HSIRDY:** Internal high-speed clock ready flag

Set by hardware to indicate that the HSI oscillator is stable. After the HSION bit is cleared, HSIRDY goes low after 6 HSI clock cycles.

0: HSI oscillator not ready

1: HSI oscillator ready

Bit 0 HSION: Internal high-speed clock enable

Set and cleared by software.

Set by hardware to force the HSI oscillator ON when leaving the Stop or Standby mode or in case of a failure of the HSE oscillator used directly or indirectly as the system clock. This bit cannot be cleared if the HSI is used directly or indirectly as the system clock.

0: HSI oscillator OFF

1: HSI oscillator ON

6.3.2 RCC PLL configuration register (RCC_PLLCFGR)

Address offset: 0x04

Reset value: 0x2400 3010

Access: no wait state, word, half-word and byte access.

This register is used to configure the PLL clock outputs according to the formulas:

- $f_{(VCO \text{ clock})} = f_{(PLL \text{ clock input})} \times (PLL_N / PLL_M)$
- $f_{(PLL \text{ general clock output})} = f_{(VCO \text{ clock})} / PLL_P$
- $f_{(USB \text{ OTG FS, SDIO})} = f_{(VCO \text{ clock})} / PLL_Q$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				PLLQ3	PLLQ2	PLLQ1	PLLQ0	Reserved	PLLSRC	Reserved				PLL1P1	PLL1P0
				rw	rw	rw	rw		rw					rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	PLL_N									PLL_M5	PLL_M4	PLL_M3	PLL_M2	PLL_M1	PLL_M0
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31:28 Reserved, must be kept at reset value.

Bits 27:24 **PLLQ**: Main PLL (PLL) division factor for USB OTG FS, and SDIO clocks

Set and cleared by software to control the frequency of USB OTG FS clock, and the SDIO clock. These bits should be written only if PLL is disabled.

Caution: The USB OTG FS requires a 48 MHz clock to work correctly. The SDIO need a frequency lower than or equal to 48 MHz to work correctly.

USB OTG FS clock frequency = VCO frequency / PLLQ with $2 \leq PLLQ \leq 15$

0000: PLLQ = 0, wrong configuration

0001: PLLQ = 1, wrong configuration

0010: PLLQ = 2

0011: PLLQ = 3

0100: PLLQ = 4

...

1111: PLLQ = 15

Bit 23 Reserved, must be kept at reset value.

Bit 22 **PLLSRC**: Main PLL(PLL) and audio PLL (PLL12S) entry clock source

Set and cleared by software to select PLL and PLL12S clock source. This bit can be written only when PLL and PLL12S are disabled.

0: HSI clock selected as PLL and PLL12S clock entry

1: HSE oscillator clock selected as PLL and PLL12S clock entry

Bits 21:18 Reserved, must be kept at reset value.

Bits 17:16 **PLL_P**: Main PLL (PLL) division factor for main system clock

Set and cleared by software to control the frequency of the general PLL output clock. These bits can be written only if PLL is disabled.

Caution: The software has to set these bits correctly not to exceed 100 MHz on this domain.

PLL output clock frequency = VCO frequency / PLL_P with PLL_P = 2, 4, 6, or 8

00: PLL_P = 2

01: PLL_P = 4

10: PLL_P = 6

11: PLL_P = 8

Bits 14:6 **PLL_N**: Main PLL (PLL) multiplication factor for VCO

Set and cleared by software to control the multiplication factor of the VCO. These bits can be written only when PLL is disabled. Only half-word and word accesses are allowed to write these bits.

Caution: The software has to set these bits correctly to ensure that the VCO output frequency is between 100 and 432 MHz. (check also [Section 6.3.20: RCC PLLI2S configuration register \(RCC_PLLI2SCFGR\)](#))

VCO output frequency = VCO input frequency × PLL_N with $50 \leq \text{PLL}_N \leq 432$

000000000: PLL_N = 0, wrong configuration

000000001: PLL_N = 1, wrong configuration

...

000110010: PLL_N = 50

...

001100011: PLL_N = 99

001100100: PLL_N = 100

...

110110000: PLL_N = 432

110110001: PLL_N = 433, wrong configuration

...

111111111: PLL_N = 511, wrong configuration

Note: Multiplication factors possible for VCO input frequency higher than 1 MHz but care must be taken to fulfill the minimum VCO output frequency as specified above.

Bits 5:0 **PLL_M**: Division factor for the main PLL (PLL) input clock

Set and cleared by software to divide the PLL input clock before the VCO. These bits can be written only when the PLL is disabled.

Caution: The software has to set these bits correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.

VCO input frequency = PLL input clock frequency / PLL_M with $2 \leq \text{PLL}_M \leq 63$

000000: PLL_M = 0, wrong configuration

000001: PLL_M = 1, wrong configuration

000010: PLL_M = 2

000011: PLL_M = 3

000100: PLL_M = 4

...

111110: PLL_M = 62

111111: PLL_M = 63

6.3.3 RCC clock configuration register (RCC_CFGR)

Address offset: 0x08

Reset value: 0x0000 0000

Access: $0 \leq \text{wait state} \leq 2$, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during a clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2		MCO2 PRE[2:0]			MCO1 PRE[2:0]			I2SSC R	MCO1		RTCPRE[4:0]				
rw		rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PPRE2[2:0]			PPRE1[2:0]			Reserved		HPRE[3:0]				SWS1	SWS0	SW1	SW0
rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	r	r	rw	rw

Bits 31:30 **MCO2[1:0]**: Microcontroller clock output 2

Set and cleared by software. Clock source selection may generate glitches on MCO2. It is highly recommended to configure these bits only after reset before enabling the external oscillators and the PLLs.

00: System clock (SYSCLK) selected

01: PLLI2S clock selected

10: HSE oscillator clock selected

11: PLL clock selected

Bits 29:27 **MCO2PRE**: MCO2 prescaler

Set and cleared by software to configure the prescaler of the MCO2. Modification of this prescaler may generate glitches on MCO2. It is highly recommended to change this prescaler only after reset before enabling the external oscillators and the PLLs.

0xx: no division

100: division by 2

101: division by 3

110: division by 4

111: division by 5

Bits 26:24 **MCO1PRE**: MCO1 prescaler

Set and cleared by software to configure the prescaler of the MCO1. Modification of this prescaler may generate glitches on MCO1. It is highly recommended to change this prescaler only after reset before enabling the external oscillators and the PLL.

0xx: no division

100: division by 2

101: division by 3

110: division by 4

111: division by 5

Bit 23 **I2SSRC**: I2S clock selection

Set and cleared by software. This bit allows to select the I2S clock source between the PLLI2S clock and the external clock. It is highly recommended to change this bit only after reset and before enabling the I2S module.

0: PLLI2S clock used as I2S clock source

1: External clock mapped on the I2S_CKIN pin used as I2S clock source

Bits 22:21 **MCO1**: Microcontroller clock output 1

Set and cleared by software. Clock source selection may generate glitches on MCO1. It is highly recommended to configure these bits only after reset before enabling the external oscillators and PLL.

00: HSI clock selected

01: LSE oscillator selected

10: HSE oscillator clock selected

11: PLL clock selected

Bits 20:16 **RTCPRE**: HSE division factor for RTC clock

Set and cleared by software to divide the HSE clock input clock to generate a 1 MHz clock for RTC.

Caution: The software has to set these bits correctly to ensure that the clock supplied to the RTC is 1 MHz. These bits must be configured if needed before selecting the RTC clock source.

00000: no clock

00001: no clock

00010: HSE/2

00011: HSE/3

00100: HSE/4

...

11110: HSE/30

11111: HSE/31

Bits 15:13 **PPRE2**: APB high-speed prescaler (APB2)

Set and cleared by software to control APB high-speed clock division factor.

Caution: The software has to set these bits correctly not to exceed 100 MHz on this domain. The clocks are divided with the new prescaler factor from 1 to 16 AHB cycles after PPRE2 write.

0xx: AHB clock not divided

100: AHB clock divided by 2

101: AHB clock divided by 4

110: AHB clock divided by 8

111: AHB clock divided by 16

Bits 12:10 **PPRE1**: APB Low speed prescaler (APB1)

Set and cleared by software to control APB low-speed clock division factor.

Caution: The software has to set these bits correctly not to exceed 50 MHz on this domain. The clocks are divided with the new prescaler factor from 1 to 16 AHB cycles after PPRE1 write.

0xx: AHB clock not divided

100: AHB clock divided by 2

101: AHB clock divided by 4

110: AHB clock divided by 8

111: AHB clock divided by 16

Bits 9:8 Reserved, must be kept at reset value.

Bits 7:4 **HPRE**: AHB prescaler

Set and cleared by software to control AHB clock division factor.

Caution: The clocks are divided with the new prescaler factor from 1 to 16 AHB cycles after HPRE write.

0xxx: system clock not divided
1000: system clock divided by 2
1001: system clock divided by 4
1010: system clock divided by 8
1011: system clock divided by 16
1100: system clock divided by 64
1101: system clock divided by 128
1110: system clock divided by 256
1111: system clock divided by 512

Bits 3:2 **SWS**: System clock switch status

Set and cleared by hardware to indicate which clock source is used as the system clock.

00: HSI oscillator used as the system clock
01: HSE oscillator used as the system clock
10: PLL used as the system clock
11: not applicable

Bits 1:0 **SW**: System clock switch

Set and cleared by software to select the system clock source.

Set by hardware to force the HSI selection when leaving the Stop or Standby mode or in case of failure of the HSE oscillator used directly or indirectly as the system clock.

00: HSI oscillator selected as system clock
01: HSE oscillator selected as system clock
10: PLL selected as system clock
11: not allowed

6.3.4 RCC clock interrupt register (RCC_CIR)

Address offset: 0x0C

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CSSC	Reserved	PLLI2S RDYC	PLL RDYC	HSE RDYC	HSI RDYC	LSE RDYC	LSI RDYC
								w		w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	PLLI2S RDYIE	PLL RDYIE	HSE RDYIE	HSI RDYIE	LSE RDYIE	LSI RDYIE	CSSF	Reserved	PLLI2S RDYF	PLL RDYF	HSE RDYF	HSI RDYF	LSE RDYF	LSI RDYF	
	rw	rw	rw	rw	rw	rw			r	r	r	r	r	r	

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **CSSC**: Clock security system interrupt clear

This bit is set by software to clear the CSSF flag.

0: No effect

1: Clear CSSF flag

Bit 22 Reserved, must be kept at reset value.

Bit 21 **PLLI2SRDYC**: PLLI2S ready interrupt clear

This bit is set by software to clear the PLLI2SRDYF flag.

0: No effect

1: PLLI2SRDYF cleared

Bit 20 **PLLRDYC**: Main PLL(PLL) ready interrupt clear

This bit is set by software to clear the PLLRDYF flag.

0: No effect

1: PLLRDYF cleared

Bit 19 **HSERDYC**: HSE ready interrupt clear

This bit is set by software to clear the HSERDYF flag.

0: No effect

1: HSERDYF cleared

Bit 18 **HSIRDYC**: HSI ready interrupt clear

This bit is set software to clear the HSIRDYF flag.

0: No effect

1: HSIRDYF cleared

Bit 17 **LSERDYC**: LSE ready interrupt clear

This bit is set by software to clear the LSERDYF flag.

0: No effect

1: LSERDYF cleared

Bit 16 **LSIRDYC**: LSI ready interrupt clear

This bit is set by software to clear the LSIRDYF flag.

0: No effect

1: LSIRDYF cleared

Bits 15:14 Reserved, must be kept at reset value.

Bit 13 **PLLI2SRDYIE**: PLLI2S ready interrupt enable

Set and cleared by software to enable/disable interrupt caused by PLLI2S lock.

0: PLLI2S lock interrupt disabled

1: PLLI2S lock interrupt enabled

Bit 12 **PLLRDYIE**: Main PLL (PLL) ready interrupt enable

Set and cleared by software to enable/disable interrupt caused by PLL lock.

0: PLL lock interrupt disabled

1: PLL lock interrupt enabled

Bit 11 **HSERDYIE**: HSE ready interrupt enable

Set and cleared by software to enable/disable interrupt caused by the HSE oscillator stabilization.

0: HSE ready interrupt disabled

1: HSE ready interrupt enabled

Bit 10 **HSIRDYIE**: HSI ready interrupt enable

Set and cleared by software to enable/disable interrupt caused by the HSI oscillator stabilization.

0: HSI ready interrupt disabled

1: HSI ready interrupt enabled

Bit 9 **LSERDYIE**: LSE ready interrupt enable

Set and cleared by software to enable/disable interrupt caused by the LSE oscillator stabilization.

0: LSE ready interrupt disabled

1: LSE ready interrupt enabled

Bit 8 **LSIRDYIE**: LSI ready interrupt enable

Set and cleared by software to enable/disable interrupt caused by LSI oscillator stabilization.

0: LSI ready interrupt disabled

1: LSI ready interrupt enabled

Bit 7 **CSSF**: Clock security system interrupt flag

Set by hardware when a failure is detected in the HSE oscillator.

Cleared by software setting the CSSC bit.

0: No clock security interrupt caused by HSE clock failure

1: Clock security interrupt caused by HSE clock failure

Bit 6 Reserved, must be kept at reset value.

Bit 5 **PLLI2SRDYF**: PLLI2S ready interrupt flag

Set by hardware when the PLLI2S locks and PLLI2SRDYIE is set.

Cleared by software setting the PLLRI2SDYC bit.

0: No clock ready interrupt caused by PLLI2S lock

1: Clock ready interrupt caused by PLLI2S lock

Bit 4 **PLLRDYF**: Main PLL (PLL) ready interrupt flag

Set by hardware when PLL locks and PLLRDYIE is set.

Cleared by software setting the PLLRDYC bit.

0: No clock ready interrupt caused by PLL lock

1: Clock ready interrupt caused by PLL lock

Bit 3 **HSERDYF**: HSE ready interrupt flag

Set by hardware when External High Speed clock becomes stable and HSERDYDIE is set.

Cleared by software setting the HSERDYC bit.

0: No clock ready interrupt caused by the HSE oscillator

1: Clock ready interrupt caused by the HSE oscillator

Bit 2 **HSIRDYF**: HSI ready interrupt flag

Set by hardware when the Internal High Speed clock becomes stable and HSIRDYDIE is set.

Cleared by software setting the HSIRDYC bit.

0: No clock ready interrupt caused by the HSI oscillator

1: Clock ready interrupt caused by the HSI oscillator

Bit 1 **LSERDYF**: LSE ready interrupt flag

Set by hardware when the External Low Speed clock becomes stable and LSERDYDIE is set.

Cleared by software setting the LSERDYC bit.

0: No clock ready interrupt caused by the LSE oscillator

1: Clock ready interrupt caused by the LSE oscillator

Bit 0 **LSIRDYF**: LSI ready interrupt flag

Set by hardware when the internal low speed clock becomes stable and LSIRDYDIE is set.

Cleared by software setting the LSIRDYC bit.

0: No clock ready interrupt caused by the LSI oscillator

1: Clock ready interrupt caused by the LSI oscillator

6.3.5 RCC AHB1 peripheral reset register (RCC_AHB1RSTR)

Address offset: 0x10

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2 RST	DMA1 RST	Reserved				
									rw	rw					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCRST	Reserved				GPIOH RST	Reserved		GPIOE RST	GIPOD RST	GPIOC RST	GPIOB RST	GPIOA RST
			rw					rw			rw	rw	rw	rw	

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **DMA2RST**: DMA2 reset

Set and cleared by software.

0: does not reset DMA2

1: resets DMA2

Bit 21 **DMA1RST**: DMA1 reset

Set and cleared by software.

0: does not reset DMA1

1: resets DMA1

Bits 20:13 Reserved, must be kept at reset value.

Bit 12 **CRCRST**: CRC reset

Set and cleared by software.

0: does not reset CRC

1: resets CRC

Bits 11:8 Reserved, must be kept at reset value.

Bit 7 **GPIOHRST**: IO port H reset

Set and cleared by software.

0: does not reset IO port H

1: resets IO port H

Bits 6:5 Reserved, must be kept at reset value.

Bit 4 **GPIOERST**: IO port E reset

Set and cleared by software.

0: does not reset IO port E

1: resets IO port E

Bit 3 **GPIODRST**: IO port D reset

Set and cleared by software.

0: does not reset IO port D

1: resets IO port D

Bit 2 **GPIOCRST**: IO port C reset

Set and cleared by software.

0: does not reset IO port C

1: resets IO port C

Bit 1 **GPIOBRST**: IO port B reset

Set and cleared by software.

0: does not reset IO port B

1: resets IO port B

Bit 0 **GPIOARST**: IO port A reset

Set and cleared by software.

0: does not reset IO port A

1: resets IO port A

6.3.6 RCC AHB2 peripheral reset register (RCC_AHB2RSTR)

Address offset: 0x14

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								OTGFS RST	Reserved						
								rw							

Bits 31:8 Reserved, must be kept at reset value.

Bit 6:0 Reserved, must be kept at reset value.

6.3.7 RCC APB1 peripheral reset register for (RCC_APB1RSTR)

Address offset: 0x20

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR RST	Reserved				I2C3 RST	I2C2 RST	I2C1 RST	Reserved			USART2 RST	Reser- ved
			rw					rw	rw	rw				rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 RST	SPI2 RST	Reserved		WWDG RST	Reserved						TIM5 RST	TIM4 RST	TIM3 RST	TIM2 RST	
rw	rw			rw							rw	rw	rw	rw	

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **PWRRST**: Power interface reset

Set and cleared by software.

0: does not reset the power interface

1: resets the power interface

Bits 27:24 Reserved, must be kept at reset value.

Bit 23 **I2C3RST**: I2C3 reset

Set and cleared by software.

0: does not reset I2C3

1: resets I2C3

Bit 22 **I2C2RST**: I2C2 reset

Set and cleared by software.

0: does not reset I2C2

1: resets I2C2

- Bit 21 **I2C1RST**: I2C1 reset
Set and cleared by software.
0: does not reset I2C1
1: resets I2C1
- Bits 20:18 Reserved, must be kept at reset value.
- Bit 17 **USART2RST**: USART2 reset
Set and cleared by software.
0: does not reset USART2
1: resets USART2
- Bit 16 Reserved, must be kept at reset value.
- Bit 15 **SPI3RST**: SPI3 reset
Set and cleared by software.
0: does not reset SPI3
1: resets SPI3
- Bit 14 **SPI2RST**: SPI2 reset
Set and cleared by software.
0: does not reset SPI2
1: resets SPI2
- Bits 13:12 Reserved, must be kept at reset value.
- Bit 11 **WWDGRST**: Window watchdog reset
Set and cleared by software.
0: does not reset the window watchdog
1: resets the window watchdog
- Bits 10:4 Reserved, must be kept at reset value.
- Bit 3 **TIM5RST**: TIM5 reset
Set and cleared by software.
0: does not reset TIM5
1: resets TIM5
- Bit 2 **TIM4RST**: TIM4 reset
Set and cleared by software.
0: does not reset TIM4
1: resets TIM4
- Bit 1 **TIM3RST**: TIM3 reset
Set and cleared by software.
0: does not reset TIM3
1: resets TIM3
- Bit 0 **TIM2RST**: TIM2 reset
Set and cleared by software.
0: does not reset TIM2
1: resets TIM2

6.3.8 RCC APB2 peripheral reset register (RCC_APB2RSTR)

Address offset: 0x24

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SPI5 RST	Reser- ved	TIM11 RST	TIM10 RST	TIM9 RST
											rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reser- ved	SYSCFG RST	SPI4 RST	SPI1 RST	SDIO RST	Reserved			Reserved			USART6 RST	USART1 RST	Reserved		TIM1 RST
	rw	rw	rw	rw							rw	rw			rw

Bits 31:21 Reserved, must be kept at reset value.

Bit 20 **SPI5RST**: SPI5RST

This bit is set and cleared by software.

0: does not reset SPI5

1: resets SPI5

Bit 19 Reserved, must be kept at reset value.

Bit 18 **TIM11RST**: TIM11 reset

Set and cleared by software.

0: does not reset TIM11

1: resets TIM11

Bit 17 **TIM10RST**: TIM10 reset

Set and cleared by software.

0: does not reset TIM10

1: resets TIM10

Bit 16 **TIM9RST**: TIM9 reset

Set and cleared by software.

0: does not reset TIM9

1: resets TIM9

Bit 15 Reserved, must be kept at reset value.

Bit 14 **SYSCFGRST**: System configuration controller reset

Set and cleared by software.

0: does not reset the System configuration controller

1: resets the System configuration controller

Bit 13 **SPI4RST**: SPI4 reset

Set and reset by software.

0: does not reset SPI4

1: resets SPI4

Bit 12 **SPI1RST**: SPI1 reset

Set and cleared by software.

0: does not reset SPI1

1: resets SPI1

Bit 11 **SDIORST**: SDIO reset

Set and cleared by software.

0: does not reset the SDIO module

1: resets the SDIO module

Bits 10:9 Reserved, must be kept at reset value.

Bit 8 **ADC1RST**: ADC interface reset

Set and cleared by software.

0: does not reset the ADC interface

1: resets the ADC interface

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **USART6RST**: USART6 reset

Set and cleared by software.

0: does not reset USART6

1: resets USART6

Bit 4 **USART1RST**: USART1 reset

Set and cleared by software.

0: does not reset USART1

1: resets USART1

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **TIM1RST**: TIM1 reset

Set and cleared by software.

0: does not reset TIM1

1: resets TIM1

6.3.9 RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2EN	DMA1EN	Reserved				
									rw	rw					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCEN	Reserved				GPIOH EN	Reserved		GPIOEEN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw					rw			rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **DMA2EN**: DMA2 clock enable

Set and cleared by software.

0: DMA2 clock disabled

1: DMA2 clock enabled

Bit 21 **DMA1EN**: DMA1 clock enable

Set and cleared by software.

0: DMA1 clock disabled

1: DMA1 clock enabled

Bits 20:13 Reserved, must be kept at reset value.

Bit 12 **CRCEN**: CRC clock enable

Set and cleared by software.

0: CRC clock disabled

1: CRC clock enabled

Bits 11:8 Reserved, must be kept at reset value.

Bit 7 **GPIOHEN**: IO port H clock enable

Set and reset by software.

0: IO port H clock disabled

1: IO port H clock enabled

Bits 6:5 Reserved, must be kept at reset value.

Bit 4 **GPIOEEN**: IO port E clock enable

Set and cleared by software.

0: IO port E clock disabled

1: IO port E clock enabled

Bit 3 **GPIODEN**: IO port D clock enable

Set and cleared by software.

0: IO port D clock disabled

1: IO port D clock enabled

Bit 2 **GPIOCEN**: IO port C clock enable

Set and cleared by software.

0: IO port C clock disabled

1: IO port C clock enabled

Bit 1 **GPIOBEN**: IO port B clock enable

Set and cleared by software.

0: IO port B clock disabled

1: IO port B clock enabled

Bit 0 **GPIOAEN**: IO port A clock enable

Set and cleared by software.

0: IO port A clock disabled

1: IO port A clock enabled

6.3.10 RCC AHB2 peripheral clock enable register (RCC_AHB2ENR)

Address offset: 0x34

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								OTGFS EN	Reserved						
								rw							

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **OTGFSEN**: USB OTG FS clock enable

Set and cleared by software.

0: USB OTG FS clock disabled

1: USB OTG FS clock enabled

Bits 6:0 Reserved, must be kept at reset value.

6.3.11 RCC APB1 peripheral clock enable register (RCC_APB1ENR)

Address offset: 0x40

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR EN	Reserved				I2C3 EN	I2C2 EN	I2C1 EN	Reserved			USART2 EN	Reser- ved
			rw					rw	rw	rw				rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 EN	SPI2 EN	Reserved		WWDG EN	Reserved						TIM5 EN	TIM4 EN	TIM3 EN	TIM2 EN	
rw	rw			rw							rw	rw	rw	rw	

Bits 31:29 Reserved, must be kept at reset value.

- Bit 28 **PWREN**: Power interface clock enable
Set and cleared by software.
0: Power interface clock disabled
1: Power interface clock enable

Bits 27:24 Reserved, must be kept at reset value.

- Bit 23 **I2C3EN**: I2C3 clock enable
Set and cleared by software.
0: I2C3 clock disabled
1: I2C3 clock enabled

- Bit 22 **I2C2EN**: I2C2 clock enable
Set and cleared by software.
0: I2C2 clock disabled
1: I2C2 clock enabled

- Bit 21 **I2C1EN**: I2C1 clock enable
Set and cleared by software.
0: I2C1 clock disabled
1: I2C1 clock enabled

Bits 20:18 Reserved, must be kept at reset value.

- Bit 17 **USART2EN**: USART2 clock enable
Set and cleared by software.
0: USART2 clock disabled
1: USART2 clock enabled

Bit 16 Reserved, must be kept at reset value.

- Bit 15 **SPI3EN**: SPI3 clock enable
Set and cleared by software.
0: SPI3 clock disabled
1: SPI3 clock enabled

- Bit 14 **SPI2EN**: SPI2 clock enable
Set and cleared by software.
0: SPI2 clock disabled
1: SPI2 clock enabled

Bits 13:12 Reserved, must be kept at reset value.

- Bit 11 **WWDGEN**: Window watchdog clock enable
Set and cleared by software.
0: Window watchdog clock disabled
1: Window watchdog clock enabled

Bits 10:4 Reserved, must be kept at reset value.

- Bit 3 **TIM5EN**: TIM5 clock enable
Set and cleared by software.
0: TIM5 clock disabled
1: TIM5 clock enabled

- Bit 2 **TIM4EN**: TIM4 clock enable
Set and cleared by software.
0: TIM4 clock disabled
1: TIM4 clock enabled
- Bit 1 **TIM3EN**: TIM3 clock enable
Set and cleared by software.
0: TIM3 clock disabled
1: TIM3 clock enabled
- Bit 0 **TIM2EN**: TIM2 clock enable
Set and cleared by software.
0: TIM2 clock disabled
1: TIM2 clock enabled

6.3.12 RCC APB2 peripheral clock enable register (RCC_APB2ENR)

Address offset: 0x44

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SPI5EN	Reser- ved	TIM11 EN	TIM10 EN	TIM9 EN
											rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reser- ved	SYSCF G EN	SPI4EN	SPI1 EN	SDIO EN	Reserved			Reserved		ADC1 EN	USART6 EN	USART1 EN	Reserved		TIM1 EN
	rw	rw	rw	rw						rw	rw	rw			rw

Bits 31:21 Reserved, must be kept at reset value.

Bit 20 **SPI5EN**: SPI5 clock enable

This bit is set and cleared by software

0: SPI5 clock disabled

1: SPI5 clock enabled

Bit 19 Reserved, must be kept at reset value.

Bit 18 **TIM11EN**: TIM11 clock enable

Set and cleared by software.

0: TIM11 clock disabled

1: TIM11 clock enabled

Bit 17 **TIM10EN**: TIM10 clock enable

Set and cleared by software.

0: TIM10 clock disabled

1: TIM10 clock enabled

Bit 16 **TIM9EN**: TIM9 clock enable

Set and cleared by software.

0: TIM9 clock disabled

1: TIM9 clock enabled

Bit 15 Reserved, must be kept at reset value.

Bit 14 **SYSCFGEN**: System configuration controller clock enable

Set and cleared by software.

0: System configuration controller clock disabled

1: System configuration controller clock enabled

Bit 13 **SPI4EN**: SPI4 clock enable

Set and reset by software.

0: SPI4 clock disabled

1: SPI4 clock enable

Bit 12 **SPI1EN**: SPI1 clock enable

Set and cleared by software.

0: SPI1 clock disabled

1: SPI1 clock enabled

Bit 11 **SDIOEN**: SDIO clock enable
Set and cleared by software.
0: SDIO module clock disabled
1: SDIO module clock enabled

Bit 8 **ADC1EN**: ADC1 clock enable
Set and cleared by software.
0: ADC1 clock disabled
1: ADC1 clock disabled

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **USART6EN**: USART6 clock enable
Set and cleared by software.
0: USART6 clock disabled
1: USART6 clock enabled

Bit 4 **USART1EN**: USART1 clock enable
Set and cleared by software.
0: USART1 clock disabled
1: USART1 clock enabled

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **TIM1EN**: TIM1 clock enable
Set and cleared by software.
0: TIM1 clock disabled
1: TIM1 clock enabled

6.3.13 RCC AHB1 peripheral clock enable in low power mode register (RCC_AHB1LPENR)

Address offset: 0x50

Reset value: 0x0061 900F

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved									DMA2 LPEN	DMA1 LPEN	Reserved				SRAM1 LPEN
									rw	rw					rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLITF LPEN	Reserved		CRC LPEN	Reserved				GPIOH LPEN	Reserved		GPIOE LPEN	GPIOD LPEN	GPIOC LPEN	GPIOB LPEN	GPIOA LPEN
rw			rw					rw			rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **DMA2LPEN**: DMA2 clock enable during Sleep mode

Set and cleared by software.

0: DMA2 clock disabled during Sleep mode

1: DMA2 clock enabled during Sleep mode

Bit 21 **DMA1LPEN**: DMA1 clock enable during Sleep mode

Set and cleared by software.

0: DMA1 clock disabled during Sleep mode

1: DMA1 clock enabled during Sleep mode

Bits 20:17 Reserved, must be kept at reset value.

Bit 16 **SRAM1LPEN**: SRAM1 interface clock enable during Sleep mode

Set and cleared by software.

0: SRAM1 interface clock disabled during Sleep mode

1: SRAM1 interface clock enabled during Sleep mode

Bit 15 **FLITFLPEN**: Flash interface clock enable during Sleep mode

Set and cleared by software.

0: Flash interface clock disabled during Sleep mode

1: Flash interface clock enabled during Sleep mode

Bits 14:13 Reserved, must be kept at reset value.

Bit 12 **CRCLPEN**: CRC clock enable during Sleep mode

Set and cleared by software.

0: CRC clock disabled during Sleep mode

1: CRC clock enabled during Sleep mode

Bits 11:8 Reserved, must be kept at reset value.

Bit 7 **GPIOHLPEN**: IO port H clock enable during sleep mode

Set and reset by software.

0: IO port H clock disabled during sleep mode

1: IO port H clock enabled during sleep mode

Bits 6:5 Reserved, must be kept at reset value.

Bit 4 **GPIOELPEN**: IO port E clock enable during Sleep mode

Set and cleared by software.

0: IO port E clock disabled during Sleep mode

1: IO port E clock enabled during Sleep mode

Bit 3 **GPIODLPEN**: IO port D clock enable during Sleep mode

Set and cleared by software.

0: IO port D clock disabled during Sleep mode

1: IO port D clock enabled during Sleep mode

Bit 2 **GPIOCLPEN**: IO port C clock enable during Sleep mode

Set and cleared by software.

0: IO port C clock disabled during Sleep mode

1: IO port C clock enabled during Sleep mode

Bit 1 **GPIOBLPEN**: IO port B clock enable during Sleep mode

Set and cleared by software.

0: IO port B clock disabled during Sleep mode

1: IO port B clock enabled during Sleep mode

Bit 0 **GPIOALPEN**: IO port A clock enable during sleep mode

Set and cleared by software.

0: IO port A clock disabled during Sleep mode

1: IO port A clock enabled during Sleep mode

6.3.14 RCC AHB2 peripheral clock enable in low power mode register (RCC_AHB2LPENR)

Address offset: 0x54

Reset value: 0x0000 0080

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								OTGFS LPEN	Reserved						
								rw							

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **OTGFSLPEN**: USB OTG FS clock enable during Sleep mode

Set and cleared by software.

0: USB OTG FS clock disabled during Sleep mode

1: USB OTG FS clock enabled during Sleep mode

Bits 6:0 Reserved, must be kept at reset value.

6.3.15 RCC APB1 peripheral clock enable in low power mode register (RCC_APB1LPENR)

Address offset: 0x60

Reset value: 0x10E2 C80F

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved			PWR LPEN	Reserved				I2C3 LPEN	I2C2 LPEN	I2C1 LPEN	Reserved			USART2 LPEN	Reserved
			rw					rw	rw	rw				rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3 LPEN	SPI2 LPEN	Reserved			WWDG LPEN	Reserved						TIM5 LPEN	TIM4 LPEN	TIM3 LPEN	TIM2 LPEN
rw	rw				rw							rw	rw	rw	rw

Bits 31:29 Reserved, must be kept at reset value.

Bit 28 **PWRLPEN**: Power interface clock enable during Sleep mode

Set and cleared by software.

0: Power interface clock disabled during Sleep mode

1: Power interface clock enabled during Sleep mode

Bits 27:24 Reserved, must be kept at reset value.

Bit 23 **I2C3LPEN**: I2C3 clock enable during Sleep mode

Set and cleared by software.

0: I2C3 clock disabled during Sleep mode

1: I2C3 clock enabled during Sleep mode

Bit 22 **I2C2LPEN**: I2C2 clock enable during Sleep mode

Set and cleared by software.

0: I2C2 clock disabled during Sleep mode

1: I2C2 clock enabled during Sleep mode

Bit 21 **I2C1LPEN**: I2C1 clock enable during Sleep mode

Set and cleared by software.

0: I2C1 clock disabled during Sleep mode

1: I2C1 clock enabled during Sleep mode

Bits 20:18 Reserved, must be kept at reset value.

Bit 17 **USART2LPEN**: USART2 clock enable during Sleep mode

Set and cleared by software.

0: USART2 clock disabled during Sleep mode

1: USART2 clock enabled during Sleep mode

Bit 16 Reserved, must be kept at reset value.

Bit 15 **SPI3LPEN**: SPI3 clock enable during Sleep mode

Set and cleared by software.

0: SPI3 clock disabled during Sleep mode

1: SPI3 clock enabled during Sleep mode

Bit 14 **SPI2LPEN**: SPI2 clock enable during Sleep mode

Set and cleared by software.

0: SPI2 clock disabled during Sleep mode

1: SPI2 clock enabled during Sleep mode

Bits 13:12 Reserved, must be kept at reset value.

Bit 11 **WWDGLPEN**: Window watchdog clock enable during Sleep mode

Set and cleared by software.

0: Window watchdog clock disabled during sleep mode

1: Window watchdog clock enabled during sleep mode

Bits 10:4 Reserved, must be kept at reset value.

Bit 3 **TIM5LPEN**: TIM5 clock enable during Sleep mode

Set and cleared by software.

0: TIM5 clock disabled during Sleep mode

1: TIM5 clock enabled during Sleep mode

Bit 2 **TIM4LPEN**: TIM4 clock enable during Sleep mode

Set and cleared by software.

0: TIM4 clock disabled during Sleep mode

1: TIM4 clock enabled during Sleep mode

Bit 1 **TIM3LPEN**: TIM3 clock enable during Sleep mode

Set and cleared by software.

0: TIM3 clock disabled during Sleep mode

1: TIM3 clock enabled during Sleep mode

Bit 0 **TIM2LPEN**: TIM2 clock enable during Sleep mode

Set and cleared by software.

0: TIM2 clock disabled during Sleep mode

1: TIM2 clock enabled during Sleep mode

6.3.16 RCC APB2 peripheral clock enabled in low power mode register (RCC_APB2LPENR)

Address offset: 0x64

Reset value: 0x0007 7930

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved											SPI5 LPEN	Reser- ved	TIM11 LPEN	TIM10 LPEN	TIM9 LPEN
											rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reser- ved	SYSC FG LPEN	SPI4LP EN	SPI1 LPEN	SDIO LPEN	Reserved			ADC1 LPEN	Reserved		USART6 LPEN	USART1 LPEN	Reserved		TIM1 LPEN
	rw	rw	rw	rw				rw			rw	rw			rw

Bits 31:21 Reserved, must be kept at reset value.

Bit 20 **SPI5LPEN**: SPI5 clock enable during Sleep mode

This bit is set and cleared by software

0: SPI5 clock disabled during Sleep mode

1: SPI5 clock enabled during Sleep mode

Bit 19 Reserved, must be kept at reset value.

Bit 18 **TIM11LPEN**: TIM11 clock enable during Sleep mode

Set and cleared by software.

0: TIM11 clock disabled during Sleep mode

1: TIM11 clock enabled during Sleep mode

Bit 17 **TIM10LPEN**: TIM10 clock enable during Sleep mode

Set and cleared by software.

0: TIM10 clock disabled during Sleep mode

1: TIM10 clock enabled during Sleep mode

Bit 16 **TIM9LPEN**: TIM9 clock enable during sleep mode

Set and cleared by software.

0: TIM9 clock disabled during Sleep mode

1: TIM9 clock enabled during Sleep mode

Bit 15 Reserved, must be kept at reset value.

Bit 14 **SYSCFGLPEN**: System configuration controller clock enable during Sleep mode

Set and cleared by software.

0: System configuration controller clock disabled during Sleep mode

1: System configuration controller clock enabled during Sleep mode

Bit 13 **SPI4LPEN**: SPI4 clock enable during sleep mode

Set and reset by software.

0: SPI4 clock disabled during sleep mode

1: SPI4 clock enabled during sleep mode

- Bit 12 **SPI1LPEN**: SPI1 clock enable during Sleep mode
Set and cleared by software.
0: SPI1 clock disabled during Sleep mode
1: SPI1 clock enabled during Sleep mode
- Bit 11 **SDIOLPEN**: SDIO clock enable during Sleep mode
Set and cleared by software.
0: SDIO module clock disabled during Sleep mode
1: SDIO module clock enabled during Sleep mode
- Bits 10:9 Reserved, must be kept at reset value.
- Bit 8 **ADC1LPEN**: ADC1 clock enable during Sleep mode
Set and cleared by software.
0: ADC1 clock disabled during Sleep mode
1: ADC1 clock disabled during Sleep mode
- Bits 7:6 Reserved, must be kept at reset value.
- Bit 5 **USART6LPEN**: USART6 clock enable during Sleep mode
Set and cleared by software.
0: USART6 clock disabled during Sleep mode
1: USART6 clock enabled during Sleep mode
- Bit 4 **USART1LPEN**: USART1 clock enable during Sleep mode
Set and cleared by software.
0: USART1 clock disabled during Sleep mode
1: USART1 clock enabled during Sleep mode
- Bits 3:1 Reserved, must be kept at reset value.
- Bit 0 **TIM1LPEN**: TIM1 clock enable during Sleep mode
Set and cleared by software.
0: TIM1 clock disabled during Sleep mode
1: TIM1 clock enabled during Sleep mode

6.3.17 RCC Backup domain control register (RCC_BDCR)

Address offset: 0x70

Reset value: 0x0000 0000, reset by Backup domain reset.

Access: $0 \leq \text{wait state} \leq 3$, word, half-word and byte access

Wait states are inserted in case of successive accesses to this register.

The LSEON, LSEBYP, RTCSEL and RTCEN bits in the *RCC Backup domain control register (RCC_BDCR)* are in the Backup domain. As a result, after Reset, these bits are write-protected and the DBP bit in the *PWR power control register (PWR_CR)* has to be set before these can be modified. Refer to [Section 5.1.2 on page 71](#) for further information. These bits are only reset after a Backup domain Reset (see [Section 6.1.3: Backup domain reset](#)). Any internal or external Reset will not have any effect on these bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															BDRST
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTCEN	Reserved					RTCSEL[1:0]		Reserved				LSEMOD	LSEBYP	LSERDY	LSEON
rw						rw	rw						rw		

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **BDRST**: Backup domain software reset

Set and cleared by software.

0: Reset not activated

1: Resets the entire Backup domain

Bit 15 **RTCEN**: RTC clock enable

Set and cleared by software.

0: RTC clock disabled

1: RTC clock enabled

Bits 14:10 Reserved, must be kept at reset value.

Bits 9:8 **RTCSEL[1:0]**: RTC clock source selection

Set by software to select the clock source for the RTC. Once the RTC clock source has been selected, it cannot be changed anymore unless the Backup domain is reset. The BDRST bit can be used to reset them.

00: No clock

01: LSE oscillator clock used as the RTC clock

10: LSI oscillator clock used as the RTC clock

11: HSE oscillator clock divided by a programmable prescaler (selection through the RTCPRE[4:0] bits in the RCC clock configuration register (RCC_CFGR)) used as the RTC clock

Bits 7:4 Reserved, must be kept at reset value.

Bit 3 **LSEMOD**: External low-speed oscillator bypass

Set and reset by software to select crystal mode for low speed oscillator. Two power modes are available.

0: LSE oscillator "low power" mode selection

1: LSE oscillator "high drive" mode selection

Bit 2 **LSEBYP**: External low-speed oscillator bypass

Set and cleared by software to bypass oscillator in debug mode. This bit can be written only when the LSE clock is disabled.

0: LSE oscillator not bypassed
1: LSE oscillator bypassed

Bit 1 **LSERDY**: External low-speed oscillator ready

Set and cleared by hardware to indicate when the external 32 kHz oscillator is stable. After the LSEON bit is cleared, LSERDY goes low after 6 external low-speed oscillator clock cycles.

0: LSE clock not ready
1: LSE clock ready

Bit 0 **LSEON**: External low-speed oscillator enable

Set and cleared by software.

0: LSE clock OFF
1: LSE clock ON

6.3.18 RCC clock control & status register (RCC_CSR)

Address offset: 0x74

Reset value: 0x0E00 0000, reset by system reset, except reset flags by power reset only.

Access: $0 \leq \text{wait state} \leq 3$, word, half-word and byte access

Wait states are inserted in case of successive accesses to this register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPWR RSTF	WWDG RSTF	IWDG RSTF	SFT RSTF	POR RSTF	PIN RSTF	BORRS TF	RMVF	Reserved							
r	r	r	r	r	r	r	rt_w								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved													LSIRDY	LSION	
													r	rw	

Bit 31 **LPWRRSTF**: Low-power reset flag

Set by hardware when a Low-power management reset occurs.
Cleared by writing to the RMVF bit.

0: No Low-power management reset occurred
1: Low-power management reset occurred

For further information on Low-power management reset, refer to [Low-power management reset](#).

Bit 30 **WWDGRSTF**: Window watchdog reset flag

Set by hardware when a window watchdog reset occurs.
Cleared by writing to the RMVF bit.

0: No window watchdog reset occurred
1: Window watchdog reset occurred

Bit 29 **IWDGRSTF**: Independent watchdog reset flag

Set by hardware when an independent watchdog reset from V_{DD} domain occurs.
Cleared by writing to the RMVF bit.

0: No watchdog reset occurred
1: Watchdog reset occurred

- Bit 28 **SFTRSTF**: Software reset flag
Set by hardware when a software reset occurs.
Cleared by writing to the RMVF bit.
0: No software reset occurred
1: Software reset occurred
- Bit 27 **PORRSTF**: POR/PDR reset flag
Set by hardware when a POR/PDR reset occurs.
Cleared by writing to the RMVF bit.
0: No POR/PDR reset occurred
1: POR/PDR reset occurred
- Bit 26 **PINRSTF**: PIN reset flag
Set by hardware when a reset from the NRST pin occurs.
Cleared by writing to the RMVF bit.
0: No reset from NRST pin occurred
1: Reset from NRST pin occurred
- Bit 25 **BORRSTF**: BOR reset flag
Cleared by software by writing the RMVF bit.
Set by hardware when a POR/PDR or BOR reset occurs.
0: No POR/PDR or BOR reset occurred
1: POR/PDR or BOR reset occurred
- Bit 24 **RMVF**: Remove reset flag
Set by software to clear the reset flags.
0: No effect
1: Clear the reset flags
- Bits 23:2 Reserved, must be kept at reset value.
- Bit 1 **LSIRDY**: Internal low-speed oscillator ready
Set and cleared by hardware to indicate when the internal RC 40 kHz oscillator is stable.
After the LSION bit is cleared, LSIRDY goes low after 3 LSI clock cycles.
0: LSI RC oscillator not ready
1: LSI RC oscillator ready
- Bit 0 **LSION**: Internal low-speed oscillator enable
Set and cleared by software.
0: LSI RC oscillator OFF
1: LSI RC oscillator ON

6.3.19 RCC spread spectrum clock generation register (RCC_SSCGR)

Address offset: 0x80

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

The spread spectrum clock generation is available only for the main PLL.

The RCC_SSCGR register must be written either before the main PLL is enabled or after the main PLL disabled.

Note: For full details about PLL spread spectrum clock generation (SSCG) characteristics, refer to the “Electrical characteristics” section in your device datasheet.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SSCG EN	SPR EAD SEL	Reserved		INCSTEP											
rw	rw			rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
INCSTEP				MODPER											
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **SSCGEN**: Spread spectrum modulation enable

Set and cleared by software.

0: Spread spectrum modulation DISABLE. (To write after clearing CR[24]=PLLON bit)

1: Spread spectrum modulation ENABLE. (To write before setting CR[24]=PLLON bit)

Bit 30 **SPREADSEL**: Spread Select

Set and cleared by software.

To write before to set CR[24]=PLLON bit.

0: Center spread

1: Down spread

Bits 29:28 Reserved, must be kept at reset value.

Bits 27:13 **INCSTEP**: Incrementation step

Set and cleared by software. To write before setting CR[24]=PLLON bit.

Configuration input for modulation profile amplitude.

Bits 12:0 **MODPER**: Modulation period

Set and cleared by software. To write before setting CR[24]=PLLON bit.

Configuration input for modulation profile period.

6.3.20 RCC PLLI2S configuration register (RCC_PLLI2SCFGR)

Address offset: 0x84

Reset value: 0x2400 3000

Access: no wait state, word, half-word and byte access.

This register is used to configure the PLLI2S clock outputs according to the formulas:

- $f_{(VCO \text{ clock})} = f_{(PLLI2S \text{ clock input})} \times (PLLI2SN / PLLI2SM)$
- $f_{(PLL \text{ I2S clock output})} = f_{(VCO \text{ clock})} / PLLI2SR$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	PLLI2S R2	PLLI2S R1	PLLI2S R0	Reserved											
	rw	rw	rw												
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	PLLI2SN[8:0]										PLLI2SM[5:0]				
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:28 **PLLI2SR**: PLLI2S division factor for I2S clocks

Set and cleared by software to control the I2S clock frequency. These bits should be written only if the PLLI2S is disabled. The factor must be chosen in accordance with the prescaler values inside the I2S peripherals, to reach 0.3% error when using standard crystals and 0% error with audio crystals. For more information about I2S clock frequency and precision, refer to [Section 20.4.4: Clock generator](#) in the I2S chapter.

Caution: The I2Ss requires a frequency lower than or equal to 192 MHz to work correctly.

I2S clock frequency = VCO frequency / PLLR with $2 \leq PLLR \leq 7$

000: PLLR = 0, wrong configuration

001: PLLR = 1, wrong configuration

010: PLLR = 2

...

111: PLLR = 7

Bits 27:15 Reserved, must be kept at reset value.

Bits 14:6 **PLLI2SN**: PLLI2S multiplication factor for VCO

Set and cleared by software to control the multiplication factor of the VCO. These bits can be written only when the PLLI2S is disabled. Only half-word and word accesses are allowed to write these bits.

Caution: The software has to set these bits correctly to ensure that the VCO output frequency is between 100 and 432 MHz. With VCO input frequency ranges from 1 to 2 MHz (refer to [Figure 13](#) and divider factor M of the [RCC PLL configuration register \(RCC_PLLCFGR\)](#))

VCO output frequency = VCO input frequency × PLLI2SN with $50 \leq \text{PLLI2SN} \leq 432$

000000000: PLLI2SN = 0, wrong configuration

000000001: PLLI2SN = 1, wrong configuration

...

001100010: PLLI2SN = 50

...

001100011: PLLI2SN = 99

001100100: PLLI2SN = 100

001100101: PLLI2SN = 101

001100110: PLLI2SN = 102

...

110110000: PLLI2SN = 432

110110000: PLLI2SN = 433, wrong configuration

...

111111111: PLLI2SN = 511, wrong configuration

Note: Between 50 and 99 multiplication factors are possible for VCO input frequency higher than 1 MHz. However care must be taken to fulfill the minimum VCO output frequency as specified above.

Bits 5:0 **PLLI2SM**: Division factor for the audio PLL (PLLI2S) input clock

Set and cleared by software to divide the PLLI2S input clock before the VCO. These bits can be written only when the PLLI2S is disabled.

Caution: The software has to set these bits correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.

VCO input frequency = PLL input clock frequency / PLLI2SM with $2 \leq \text{PLLI2SM} \leq 63$

000000: PLLI2SM = 0, wrong configuration

000001: PLLI2SM = 1, wrong configuration...

000010: PLLI2SM = 2

000011: PLLI2SM = 3

000100: PLLI2SM = 4

.....

111110: PLLI2SM = 62

111111: PLLI2SM = 63

6.3.21 RCC Dedicated Clocks Configuration Register (RCC_DCKCFGR)

Address offset: 0x8C

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved							TIMPRE	Reserved							
							rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **TIMPRE**: Timers clocks prescalers selection

Set and reset by software to control the clock frequency of all the timers connected to APB1 and APB2 domain.

0: If the APB prescaler (PPRE1, PPRE2 in the RCC_CFGR register) is configured to a division factor of 1, $TIMxCLK = HCLK$. Otherwise, the timer clock frequencies are set to twice to the frequency of the APB domain to which the timers are connected:
 $TIMxCLK = 2 \times PCLKx$.

1: If the APB prescaler (PPRE1, PPRE2 in the RCC_CFGR register) is configured to a division factor of 1 or 2, $TIMxCLK = HCLK$. Otherwise, the timer clock frequencies are set to four times to the frequency of the APB domain to which the timers are connected:
 $TIMxCLK = 4 \times PCLKx$.

Bits 23: 0 Reserved, must be kept at reset value.

6.3.22 RCC register map

Table 21 gives the register map and reset values

Table 21. RCC register map and reset values for STM32F411xC/E

Addr. offset	Register name																																

Table 21. RCC register map and reset values for STM32F411xC/E (continued)

Addr. offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x40	RCC_APB1ENR	Reserved			PWREN	Reserved				I2C3EN	I2C2EN	I2C1EN		Reserved		USART2EN	Reserved	SPI3EN	SPI2EN	Reserved		WWDGEN	Reserved								TIM5EN	TIM4EN	TIM3EN	TIM2EN
0x44	RCC_APB2ENR	Reserved											SPI5EN	Reserved	TIM11EN	TIM10EN	Reserved	TIM9EN	Reserved	SPI3EN	SYSCFG	SPI4EN	SPI1EN	SDIOEN	Reserved	ADC1EN	Reserved	Reserved	USART6EN	USART1EN		Reserved		TIM1EN
0x48	Reserved	Reserved																																
0x4C	Reserved	Reserved																																
0x50	RCC_AHB1LPENR	Reserved									DMA2LPEN	DMA1LPEN	Reserved				SRAM1LPEN	FLITFLPEN	Reserved	CRCLPEN	Reserved				GPIOLPEN	Reserved	GPIOLPEN	GPIODLPEN	GPIODLPEN	GPIODLPEN	GPIODLPEN	GPIODLPEN	GPIODLPEN	
0x54	RCC_AHB2LPENR	Reserved																								OTGFSLPEN	Reserved							
0x58	Reserved	Reserved																																
0x5C	Reserved	Reserved																																
0x60	RCC_APB1LPENR	Reserved		PWRLPEN	Reserved				I2C3LPEN	I2C2LPEN	I2C1LPEN		Reserved		USART2LPEN	Reserved	SPI3LPEN	SPI2LPEN	Reserved	Reserved	WWDGLPEN	Reserved						TIM5LPEN	TIM4LPEN	TIM3LPEN	TIM2LPEN			
0x64	RCC_APB2LPENR	Reserved												SPI5LPEN	Reserved	TIM11LPEN	TIM10LPEN	TIM9LPEN	Reserved	SYSCFG	SPI4LPEN	SPI1LPEN	SDIOLPEN	Reserved	ADC1LPEN	Reserved	Reserved	USART6LPEN	USART1LPEN	Reserved		TIM1LPEN		
0x68	Reserved	Reserved																																
0x6C	Reserved	Reserved																																
0x70	RCC_BDCR	Reserved															BDRST	RTCEN	Reserved				RTCSEL 1	RTCSEL 0	Reserved				LSEMOD	LSEBYP	LSERDY	LSEON		
0x74	RCC_CSR	LPWRRSTF	WWDGRSTF	WDGRSTF	SFTRSTF	PORRSTF	PADRSTF	BORRSTF	RMVF	Reserved																							LSIRDY	LSION
0x78	Reserved	Reserved																																
0x7C	Reserved	Reserved																																
0x80	RCC_SSCGR	SSCGEN	SPREADSEL	Reserved	INCSTEP												MODPER																	
0x84	RCC_PLLI2SCFGR	Reserved	PLLI2SRx	Reserved												PLLI2SNx								PLLI2SMx										

Table 21. RCC register map and reset values for STM32F411xC/E (continued)

Addr. offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x88	Reserved																																
0x8C	RCC_DCKCFGR	Reserved							TIMPRE	Reserved																							

Refer to [Table 1](#) for the register boundary addresses.



7 System configuration controller (SYSCFG)

The system configuration controller is mainly used to remap the memory accessible in the code area and manage the external interrupt line connection to the GPIOs.

7.1 I/O compensation cell

By default the I/O compensation cell is not used. However when the I/O output buffer speed is configured in 50 MHz or 100 MHz mode, it is recommended to use the compensation cell for slew rate control on I/O $t_{f(I/O)out}/t_{r(I/O)out}$ commutation to reduce the I/O noise on power supply.

When the compensation cell is enabled, a READY flag is set to indicate that the compensation cell is ready and can be used. The I/O compensation cell can be used only when the supply voltage ranges from 2.4 to 3.6 V.

7.2 SYSCFG registers

7.2.1 SYSCFG memory remap register (SYSCFG_MEMRMP)

This register is used for specific configurations on memory remap:

- Two bits are used to configure the type of memory accessible at address 0x0000 0000. These bits are used to select the physical remap by software and so, bypass the BOOT pins.
- After reset these bits take the value selected by the BOOT pins. When booting from main Flash memory with BOOT0 pin set to 0 this register takes the value 0x00.

In remap mode, the CPU can access the external memory via ICode bus instead of System bus which boosts up the performance.

Address offset: 0x00

Reset value: 0x0000 000X (X is the memory mode selected by the BOOT pins)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved														MEM_MODE	
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bits 1:0 **MEM_MODE**: Memory mapping selection

Set and cleared by software. This bit controls the memory internal mapping at address 0x0000 0000. After reset these bits take the value selected by the Boot pins .

00: Main Flash memory mapped at 0x0000 0000

01: System Flash memory mapped at 0x0000 0000

11: Embedded SRAM mapped at 0x0000 0000

Note: Refer to [Section 2.3: Memory map](#) for details about the memory mapping at address 0x0000 0000.

7.2.2 SYSCFG peripheral mode configuration register (SYSCFG_PMC)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	
Reserved															ADC1D C2	
															rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
Reserved																

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **ADCxDC2**:

0: No effect.

1: Refer to AN4073 on how to use this bit .

Note: These bits can be set only if the following conditions are met:

- ADC clock higher or equal to 30 MHz.
- Only one ADCxDC2 bit must be selected if ADC conversions do not start at the same time and the sampling times differ.
- These bits must not be set when the ADCDC1 bit is set in PWR_CR register.

Bits 15:0 Reserved, must be kept at reset value.

7.2.3 SYSCFG external interrupt configuration register 1 (SYSCFG_EXTICR1)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 0 to 3)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: Reserved

0110: Reserved

0111: PH[x] pin

7.2.4 SYSCFG external interrupt configuration register 2 (SYSCFG_EXTICR2)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 4 to 7)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: Reserved

0110: Reserved

0111: PH[x] pin

7.2.5 SYSCFG external interrupt configuration register 3 (SYSCFG_EXTICR3)

Address offset: 0x10

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 8 to 11)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: Reserved

0110: Reserved

0111: PH[x] pin

7.2.6 SYSCFG external interrupt configuration register 4 (SYSCFG_EXTICR4)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x = 12 to 15)

These bits are written by software to select the source input for the EXTIx external interrupt.

0000: PA[x] pin

0001: PB[x] pin

0010: PC[x] pin

0011: PD[x] pin

0100: PE[x] pin

0101: Reserved

0110: Reserved

0111: PH[x] pin

7.2.7 Compensation cell control register (SYSCFG_CMPCR)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							READY	Reserved						CMP_PD	
							r							rw	

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **READY**: Compensation cell ready flag
0: I/O compensation cell not ready
1: O compensation cell ready

Bits 7:2 Reserved, must be kept at reset value.

Bit 0 **CMP_PD**: Compensation cell power-down
0: I/O compensation cell power-down mode
1: I/O compensation cell enabled

7.2.8 SYSCFG register map

The following table gives the SYSCFG register map and the reset values.

Table 22. SYSCFG register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	SYSCFG_MEMRMP	Reserved																														MEM_MODE	
	Reset value																															x	x
0x04	SYSCFG_PMC	Reserved															ADC1DC2	Reserved															
	Reset value																	0															
0x08	SYSCFG_EXTICR1	Reserved															EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]				
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	SYSCFG_EXTICR2	Reserved															EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]				
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	SYSCFG_EXTICR3	Reserved															EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]				
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x14	SYSCFG_EXTICR4	Reserved															EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]				
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	SYSCFG_CMPCR	Reserved																								READY		Reserved				CMP_PD	
	Reset value																										0						

Refer to [Table 1](#) for the register boundary addresses.

8 General-purpose I/Os (GPIO)

GPIO F/G/H/I/J/K (except GPIOH0 and GPIOH1) are not available in STM32F411xC/E.

8.1 GPIO introduction

Each general-purpose I/O port has four 32-bit configuration registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR and GPIOx_PUPDR), two 32-bit data registers (GPIOx_IDR and GPIOx_ODR), a 32-bit set/reset register (GPIOx_BSRR), a 32-bit locking register (GPIOx_LCKR) and two 32-bit alternate function selection register (GPIOx_AFRH and GPIOx_AFRL).

8.2 GPIO main features

- Up to 16 I/Os under control
- Output states: push-pull or open drain + pull-up/down
- Output data from output data register (GPIOx_ODR) or peripheral (alternate function output)
- Speed selection for each I/O
- Input states: floating, pull-up/down, analog
- Input data to input data register (GPIOx_IDR) or peripheral (alternate function input)
- Bit set and reset register (GPIOx_BSRR) for bitwise write access to GPIOx_ODR
- Locking mechanism (GPIOx_LCKR) provided to freeze the I/O configuration
- Analog function
- Alternate function input/output selection registers (at most 16 AFs per I/O)
- Fast toggle capable of changing every two clock cycles
- Highly flexible pin multiplexing allows the use of I/O pins as GPIOs or as one of several peripheral functions

8.3 GPIO functional description

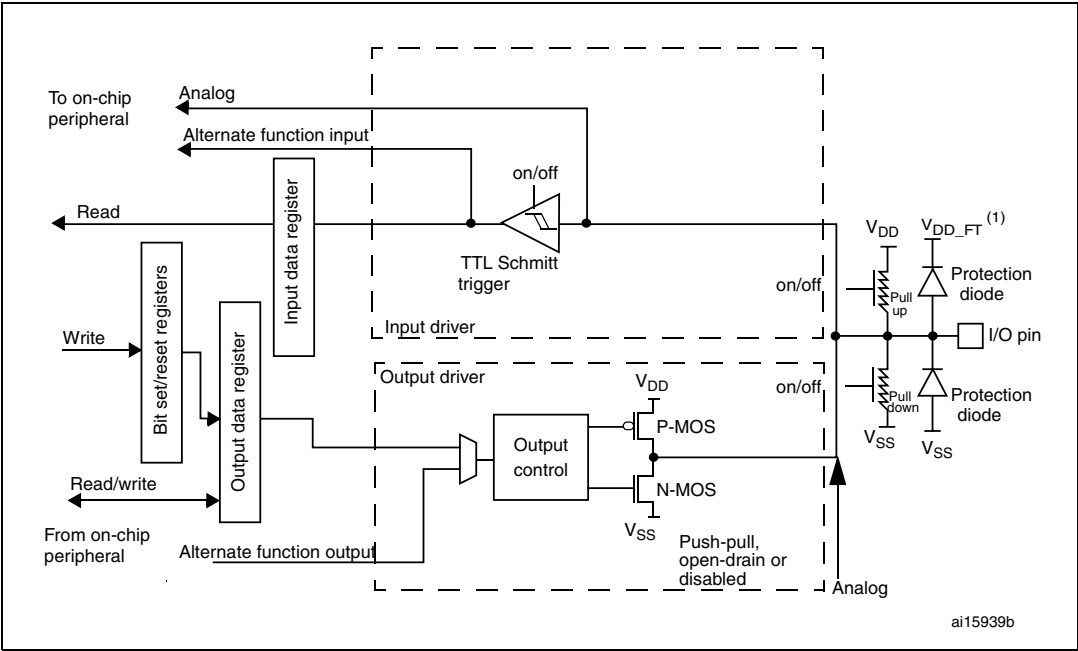
Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the general-purpose I/O (GPIO) ports can be individually configured by software in several modes:

- Input floating
- Input pull-up
- Input-pull-down
- Analog
- Output open-drain with pull-up or pull-down capability
- Output push-pull with pull-up or pull-down capability
- Alternate function push-pull with pull-up or pull-down capability
- Alternate function open-drain with pull-up or pull-down capability

Each I/O port bit is freely programmable, however the I/O port registers have to be accessed as 32-bit words, half-words or bytes. The purpose of the GPIOx_BSRR register is to allow atomic read/modify accesses to any of the GPIO registers. In this way, there is no risk of an IRQ occurring between the read and the modify access.

show the basic structure of a 5 V tolerant I/O port bit. [Table 26](#) gives the possible port bit configurations.

Figure 16. Basic structure of a five-volt tolerant I/O port bit



1. V_{DD_FT} is a potential specific to five-volt tolerant I/Os and different from V_{DD}.

Table 23. Port bit configuration table⁽¹⁾

MODER(i) [1:0]	OTYPER(i)	OSPEEDR(i) [B:A]	PUPDR(i) [1:0]		I/O configuration	
01	0	SPEED [B:A]	0	0	GP output	PP
	0		0	1	GP output	PP + PU
	0		1	0	GP output	PP + PD
	0		1	1	Reserved	
	1		0	0	GP output	OD
	1		0	1	GP output	OD + PU
	1		1	0	GP output	OD + PD
	1		1	1	Reserved (GP output OD)	

Table 23. Port bit configuration table⁽¹⁾ (continued)

MODER(i) [1:0]	OTYPER(i)	OSPEEDR(i) [B:A]		PUPDR(i) [1:0]		I/O configuration	
10	0	SPEED [B:A]		0	0	AF	PP
	0			0	1	AF	PP + PU
	0			1	0	AF	PP + PD
	0			1	1	Reserved	
	1			0	0	AF	OD
	1			0	1	AF	OD + PU
	1			1	0	AF	OD + PD
	1			1	1	Reserved	
00	x	x	x	0	0	Input	Floating
	x	x	x	0	1	Input	PU
	x	x	x	1	0	Input	PD
	x	x	x	1	1	Reserved (input floating)	
11	x	x	x	0	0	Input/output	Analog
	x	x	x	0	1	Reserved	
	x	x	x	1	0		
	x	x	x	1	1		

1. GP = general-purpose, PP = push-pull, PU = pull-up, PD = pull-down, OD = open-drain, AF = alternate function.

8.3.1 General-purpose I/O (GPIO)

During and just after reset, the alternate functions are not active and the I/O ports are configured in input floating mode.

The debug pins are in AF pull-up/pull-down after reset:

- PA15: JTDI in pull-up
- PA14: JTCK/SWCLK in pull-down
- PA13: JTMS/SWDAT in pull-up
- PB4: NJTRST in pull-up
- PB3: JTDO in floating state

When the pin is configured as output, the value written to the output data register (GPIOx_ODR) is output on the I/O pin. It is possible to use the output driver in push-pull mode or open-drain mode (only the N-MOS is activated when 0 is output).

The input data register (GPIOx_IDR) captures the data present on the I/O pin at every AHB clock cycle.

All GPIO pins have weak internal pull-up and pull-down resistors, which can be activated or not depending on the value in the GPIOx_PUPDR register.

8.3.2 I/O pin multiplexer and mapping

The microcontroller I/O pins are connected to onboard peripherals/modules through a multiplexer that allows only one peripheral's alternate function (AF) connected to an I/O pin at a time. In this way, there can be no conflict between peripherals sharing the same I/O pin.

Each I/O pin has a multiplexer with sixteen alternate function inputs (AF0 to AF15) that can be configured through the GPIOx_AFRL (for pin 0 to 7) and GPIOx_AFRH (for pin 8 to 15) registers:

- After reset all I/Os are connected to the system's alternate function 0 (AF0)
- The peripherals' alternate functions are mapped from AF1 to AF13
- Cortex®-M4 with FPU EVENTOUT is mapped on AF15

This structure is shown in [Figure 17](#) below.

In addition to this flexible I/O multiplexing architecture, each peripheral has alternate functions mapped onto different I/O pins to optimize the number of peripherals available in smaller packages.

To use an I/O in a given configuration, proceed as follows:

- **System function**

Connect the I/O to AF0 and configure it depending on the function used:

- JTAG/SWD, after each device reset these pins are assigned as dedicated pins immediately usable by the debugger host (not controlled by the GPIO controller)
- RTC_REFIN: this pin should be configured in Input floating mode
- MCO1 and MCO2: these pins have to be configured in alternate function mode.

Note: The user can disable some or all of the JTAG/SWD pins and so release the associated pins for GPIO usage (released pins highlighted in gray in the table).

For more details please refer to [Section 6.2.10: Clock-out capability](#).

Table 24. Flexible SWJ-DP pin assignment

Available debug ports	SWJ I/O pin assigned					
	PA13 / JTMS/ SWDIO	PA14 / JTCK/ SWCLK	PA15 / JTDI	PB3 / JTDO	PB4/ NJTRST	
Full SWJ (JTAG-DP + SW-DP) - Reset state	X	X	X	X	X	
Full SWJ (JTAG-DP + SW-DP) but without NJTRST	X	X	X	X		
JTAG-DP Disabled and SW-DP Enabled	X	X				
JTAG-DP Disabled and SW-DP Disabled	Released					

- **GPIO**

Configure the desired I/O as output or input in the GPIOx_MODER register.

- **Peripheral alternate function**

For the ADC, configure the desired I/O as analog in the GPIOx_MODER register.

For other peripherals:

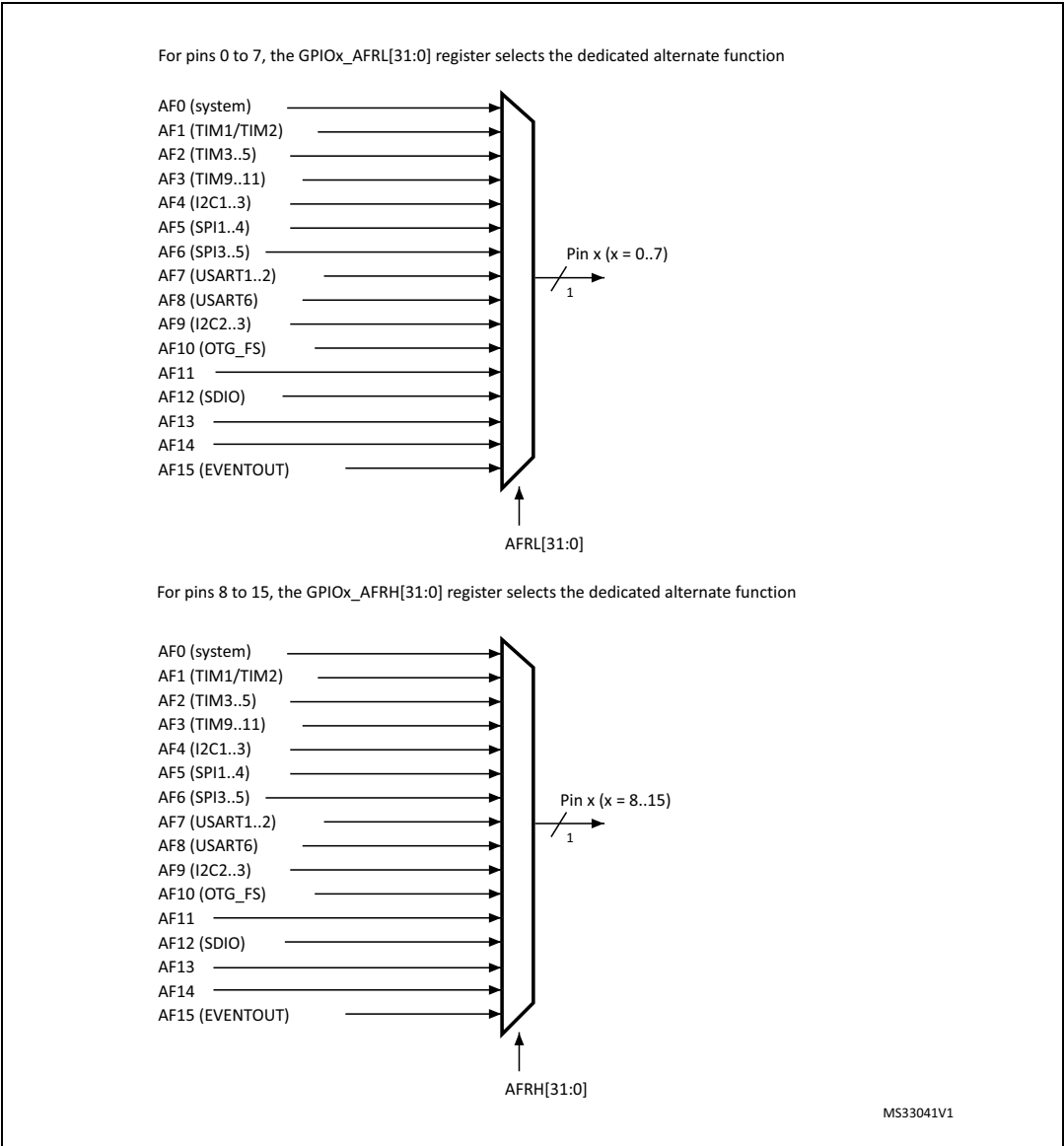
- Configure the desired I/O as an alternate function in the GPIOx_MODER register
- Select the type, pull-up/pull-down and output speed via the GPIOx_OTYPER, GPIOx_PUPDR and GPIOx_OSPEEDR registers, respectively
- Connect the I/O to the desired AFx in the GPIOx_AFRL or GPIOx_AFRH register

- **EVENTOUT**

Configure the I/O pin used to output the Cortex®-M4 with FPU EVENTOUT signal by connecting it to AF15

Please refer to the “Alternate function mapping” table in the datasheets for the detailed mapping of the system and peripherals’ alternate function I/O pins.

Figure 17. Selecting an alternate function on STM32F411xC/E



1. Configured in FS.

8.3.3 I/O port control registers

Each of the GPIOs has four 32-bit memory-mapped control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR) to configure up to 16 I/Os.

The GPIOx_MODER register is used to select the I/O direction (input, output, AF, analog). The GPIOx_OTYPER and GPIOx_OSPEEDR registers are used to select the output type (push-pull or open-drain) and speed (the I/O speed pins are directly connected to the corresponding GPIOx_OSPEEDR register bits whatever the I/O direction). The GPIOx_PUPDR register is used to select the pull-up/pull-down whatever the I/O direction.

8.3.4 I/O port data registers

Each GPIO has two 16-bit memory-mapped data registers: input and output data registers (GPIOx_IDR and GPIOx_ODR). GPIOx_ODR stores the data to be output, it is read/write accessible. The data input through the I/O are stored into the input data register (GPIOx_IDR), a read-only register.

See [Section 8.4.5: GPIO port input data register \(GPIOx_IDR\) \(x = A..E and H\)](#) and [Section 8.4.6: GPIO port output data register \(GPIOx_ODR\) \(x = A..E and H\)](#) for the register descriptions.

8.3.5 I/O data bitwise handling

The bit set reset register (GPIOx_BSRR) is a 32-bit register which allows the application to set and reset each individual bit in the output data register (GPIOx_ODR). The bit set reset register has twice the size of GPIOx_ODR.

To each bit in GPIOx_ODR, correspond two control bits in GPIOx_BSRR: BSRR(i) and BSRR(i+SIZE). When written to 1, bit BSRR(i) sets the corresponding ODR(i) bit. When written to 1, bit BSRR(i+SIZE) resets the ODR(i) corresponding bit.

Writing any bit to 0 in GPIOx_BSRR does not have any effect on the corresponding bit in GPIOx_ODR. If there is an attempt to both set and reset a bit in GPIOx_BSRR, the set action takes priority.

Using the GPIOx_BSRR register to change the values of individual bits in GPIOx_ODR is a “one-shot” effect that does not lock the GPIOx_ODR bits. The GPIOx_ODR bits can always be accessed directly. The GPIOx_BSRR register provides a way of performing atomic bitwise handling.

There is no need for the software to disable interrupts when programming the GPIOx_ODR at bit level: it is possible to modify one or more bits in a single atomic AHB1 write access.

8.3.6 GPIO locking mechanism

It is possible to freeze the GPIO control registers by applying a specific write sequence to the GPIOx_LCKR register. The frozen registers are GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH.

To write the GPIOx_LCKR register, a specific write / read sequence has to be applied. When the right LOCK sequence is applied to bit 16 in this register, the value of LCKR[15:0] is used to lock the configuration of the I/Os (during the write sequence the LCKR[15:0] value must be the same). When the LOCK sequence has been applied to a port bit, the value of the port bit can no longer be modified until the next MCU or peripheral reset. Each GPIOx_LCKR bit

freezes the corresponding bit in the control registers (GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH).

The LOCK sequence (refer to [Section 8.4.8: GPIO port configuration lock register \(GPIOx_LCKR\) \(x = A..E and H\)](#)) can only be performed using a word (32-bit long) access to the GPIOx_LCKR register due to the fact that GPIOx_LCKR bit 16 has to be set at the same time as the [15:0] bits.

For more details please refer to LCKR register description in [Section 8.4.8: GPIO port configuration lock register \(GPIOx_LCKR\) \(x = A..E and H\)](#).

8.3.7 I/O alternate function input/output

Two registers are provided to select one out of the sixteen alternate function inputs/outputs available for each I/O. With these registers, you can connect an alternate function to some other pin as required by your application.

This means that a number of possible peripheral functions are multiplexed on each GPIO using the GPIOx_AFRL and GPIOx_AFRH alternate function registers. The application can thus select any one of the possible functions for each I/O. The AF selection signal being common to the alternate function input and alternate function output, a single channel is selected for the alternate function input/output of one I/O.

To know which functions are multiplexed on each GPIO pin, refer to the datasheets.

Note: The application is allowed to select one of the possible peripheral functions for each I/O at a time.

8.3.8 External interrupt/wakeup lines

All ports have external interrupt capability. To use external interrupt lines, the port must be configured in input mode, refer to [Section 10.2: External interrupt/event controller \(EXTI\)](#) and [Section 10.2.3: Wakeup event management](#).

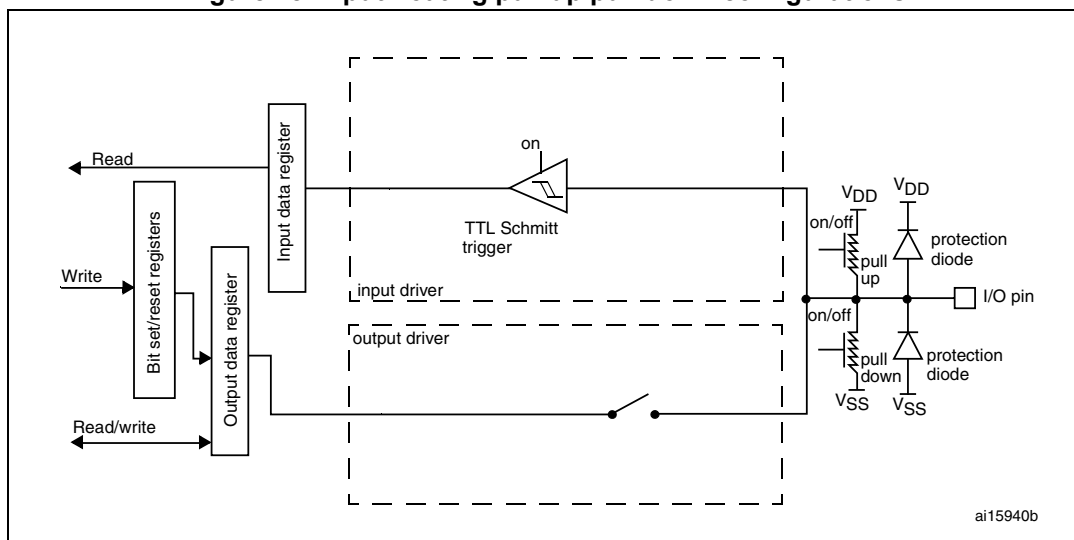
8.3.9 Input configuration

When the I/O port is programmed as Input:

- the output buffer is disabled
- the Schmitt trigger input is activated
- the pull-up and pull-down resistors are activated depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register provides the I/O State

[Figure 18](#) shows the input configuration of the I/O port bit.

Figure 18. Input floating/pull up/pull down configurations



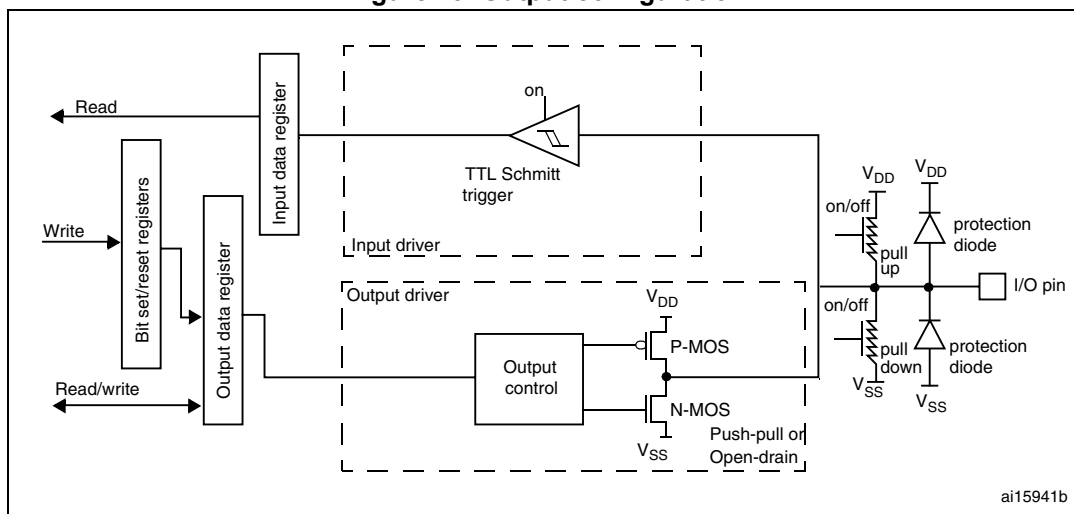
8.3.10 Output configuration

When the I/O port is programmed as output:

- The output buffer is enabled:
 - Open drain mode: A “0” in the Output register activates the N-MOS whereas a “1” in the Output register leaves the port in Hi-Z (the P-MOS is never activated)
 - Push-pull mode: A “0” in the Output register activates the N-MOS whereas a “1” in the Output register activates the P-MOS
- The Schmitt trigger input is activated
- The weak pull-up and pull-down resistors are activated or not depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register gets the I/O state
- A read access to the output data register gets the last written value

Figure 19 shows the output configuration of the I/O port bit.

Figure 19. Output configuration



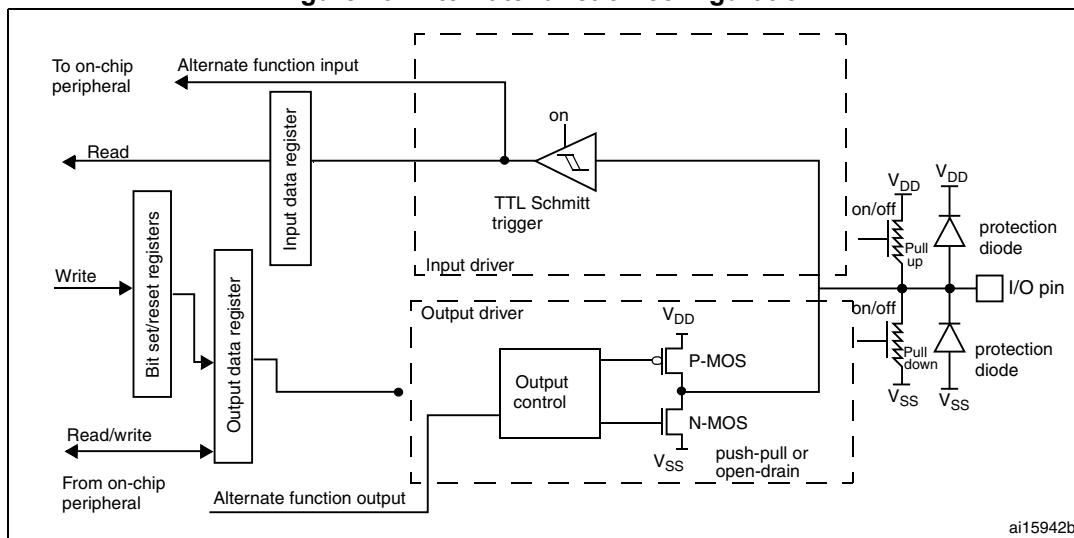
8.3.11 Alternate function configuration

When the I/O port is programmed as alternate function:

- The output buffer can be configured as open-drain or push-pull
- The output buffer is driven by the signal coming from the peripheral (transmitter enable and data)
- The Schmitt trigger input is activated
- The weak pull-up and pull-down resistors are activated or not depending on the value in the GPIOx_PUPDR register
- The data present on the I/O pin are sampled into the input data register every AHB clock cycle
- A read access to the input data register gets the I/O state

Figure 20 shows the Alternate function configuration of the I/O port bit.

Figure 20. Alternate function configuration



8.3.12 Analog configuration

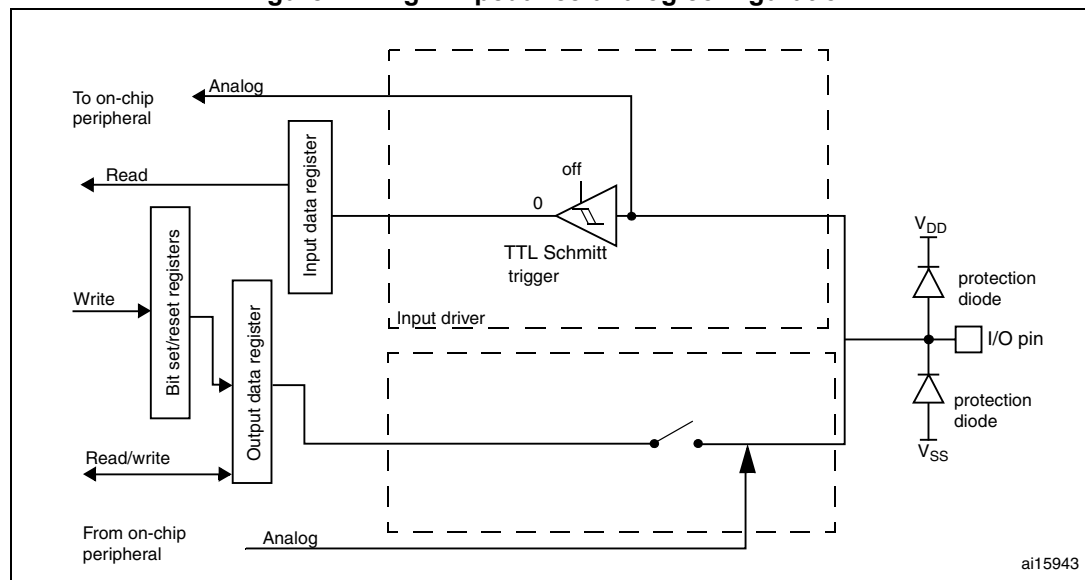
When the I/O port is programmed as analog configuration:

- The output buffer is disabled
- The Schmitt trigger input is deactivated, providing zero consumption for every analog value of the I/O pin. The output of the Schmitt trigger is forced to a constant value (0).
- The weak pull-up and pull-down resistors are disabled
- Read access to the input data register gets the value “0”

Note: In the analog configuration, the I/O pins cannot be 5 Volt tolerant.

Figure 21 shows the high-impedance, analog-input configuration of the I/O port bit.

Figure 21. High impedance-analog configuration



8.3.13 Using the OSC32_IN/OSC32_OUT pins as GPIO PC14/PC15 port pins

The LSE oscillator pins OSC32_IN and OSC32_OUT can be used as general-purpose PC14 and PC15 I/Os, respectively, when the LSE oscillator is off. The PC14 and PC15 I/Os are only configured as LSE oscillator pins OSC32_IN and OSC32_OUT when the LSE oscillator is ON. This is done by setting the LSEON bit in the RCC_BDCR register. The LSE has priority over the GPIO function.

Note: The PC14/PC15 GPIO functionality is lost when the 1.2 V domain is powered off (by the device entering the standby mode) or when the backup domain is supplied by V_{BAT} (V_{DD} no more supplied). In this case the I/Os are set in analog input mode.

8.3.14 Using the OSC_IN/OSC_OUT pins as GPIO PH0/PH1 port pins

The HSE oscillator pins OSC_IN/OSC_OUT can be used as general-purpose PH0/PH1 I/Os, respectively, when the HSE oscillator is OFF. (after reset, the HSE oscillator is off). The PH0/PH1 I/Os are only configured as OSC_IN/OSC_OUT HSE oscillator pins when the HSE oscillator is ON. This is done by setting the HSEON bit in the RCC_CR register. The HSE has priority over the GPIO function.

8.3.15 Selection of RTC functions

The STM32F4xx feature one GPIO pins RTC_AF1 that can be used for the detection of a tamper or time stamp event, or RTC_ALARM, or RTC_CALIB RTC outputs.

- The RTC_AF1 (PC13) can be used for the following purposes:

RTC_ALARM output: this output can be RTC Alarm A, RTC Alarm B or RTC Wakeup depending on the OSEL[1:0] bits in the RTC_CR register

- RTC_CALIB output: this feature is enabled by setting the COE[23] in the RTC_CR register
- RTC_TAMP1: tamper event detection
- RTC_TS: time stamp event detection

The selection of the corresponding pin is performed through the RTC_TAFCR register as follows:

- TAMP1INSEL is used to select which pin is used as the RTC_TAMP1 tamper input
- TSINSEL is used to select which pin is used as the RTC_TS time stamp input
- ALARMOUTTYPE is used to select whether the RTC_ALARM is output in push-pull or open-drain mode

The output mechanism follows the priority order listed in [Table 25](#)

Table 25. RTC additional functions⁽¹⁾

Pin configuration and function	enabled	enabled	Tamper enabled	Time stamp enabled	TAMP1INSEL TAMPER1 pin selection	TSINSEL TIMESTAMP pin selection	ALARMOUTTYPE configuration
Alarm out output OD	1	Don't care	Don't care	Don't care	Don't care	Don't care	0
Alarm out output PP	1	Don't care	Don't care	Don't care	Don't care	Don't care	1
Calibration out output PP	0	1	Don't care	Don't care	Don't care	Don't care	Don't care
TAMPER1 input floating	0	0	1	0	0	Don't care	Don't care
TIMESTAMP and TAMPER1 input floating	0	0	1	1	0	0	Don't care
TIMESTAMP input floating	0	0	0	1	Don't care	0	Don't care
Standard GPIO	0	0	0	0	Don't care	Don't care	Don't care

1. OD: open drain; PP: push-pull.

8.4 GPIO registers

This section gives a detailed description of the GPIO registers.

For a summary of register bits, register address offsets and reset values, refer to [Table 26](#).

The GPIO registers can be accessed by byte (8 bits), half-words (16 bits) or words (32 bits).

8.4.1 GPIO port mode register (GPIOx_MODER) (x = A..E and H)

Address offset: 0x00

Reset values:

- 0xA800 0000 for port A
- 0x0000 0280 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

8.4.2 GPIO port output type register (GPIOx_OTYPER) (x = A..E and H)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OTy**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the output type of the I/O port.

0: Output push-pull (reset state)

1: Output open-drain

8.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A..E and H)

Address offset: 0x08

Reset values:

- 0x0C00 0000 for port A
- 0x0000 00C0 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **OSPEEDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output speed.

00: Low speed

01: Medium speed

10: Fast speed

11: High speed

Note: Refer to the product datasheets for the values of OSPEEDRy bits versus V_{DD} range and external load.

8.4.4 GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A..E and H)

Address offset: 0x0C

Reset values:

- 0x6400 0000 for port A
- 0x0000 0100 for port B
- 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15 [1:0]		PUPDR14 [1:0]		PUPDR13 [1:0]		PUPDR12 [1:0]		PUPDR11 [1:0]		PUPDR10 [1:0]		PUPDR9 [1:0]		PUPDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7 [1:0]		PUPDR6 [1:0]		PUPDR5 [1:0]		PUPDR4 [1:0]		PUPDR3 [1:0]		PUPDR2 [1:0]		PUPDR1 [1:0]		PUPDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **PUPDRy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O pull-up or pull-down

00: No pull-up, pull-down

01: Pull-up

10: Pull-down

11: Reserved

8.4.5 GPIO port input data register (GPIOx_IDR) (x = A..E and H)

Address offset: 0x10

Reset value: 0x0000 XXXX (where X means undefined)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y = 0..15)

These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

8.4.6 GPIO port output data register (GPIOx_ODR) (x = A..E and H)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **ODRy**: Port output data (y = 0..15)

These bits can be read and written by software.

Note: For atomic bit set/reset, the ODR bits can be individually set and reset by writing to the GPIOx_BSRR register (x = A..E and H).

8.4.7 GPIO port bit set/reset register (GPIOx_BSRR) (x = A..E and H)

Address offset: 0x18

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x reset bit y (y = 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x set bit y (y = 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

8.4.8 GPIO port configuration lock register (GPIOx_LCKR) (x = A..E and H)

This register is used to lock the configuration of the port bits when a correct write sequence is applied to bit 16 (LCKK). The value of bits [15:0] is used to lock the configuration of the GPIO. During the write sequence, the value of LCKR[15:0] must not change. When the LOCK sequence has been applied on a port bit, the value of this port bit can no longer be modified until the next MCU or peripheral reset.

Note: A specific write sequence is used to write to the GPIOx_LCKR register. Only word access (32-bit long) is allowed during this write sequence.

Each lock bit freezes a specific configuration register (control and alternate function registers).

Address offset: 0x1C

Reset value: 0x0000 0000

Access: 32-bit word only, read/write register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															LCKK
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **LCKK[16]**: Lock key

This bit can be read any time. It can only be modified using the lock key write sequence.

0: Port configuration lock key not active

1: Port configuration lock key active. The GPIOx_LCKR register is locked until an MCU reset or a peripheral reset occurs.

LOCK key write sequence:

WR LCKR[16] = '1' + LCKR[15:0]

WR LCKR[16] = '0' + LCKR[15:0]

WR LCKR[16] = '1' + LCKR[15:0]

RD LCKR

RD LCKR[16] = '1' (this read operation is optional but it confirms that the lock is active)

Note: During the LOCK key write sequence, the value of LCK[15:0] must not change.

Any error in the lock sequence aborts the lock.

After the first lock sequence on any bit of the port, any read access on the LCKK bit will return '1' until the next CPU reset.

Bits 15:0 **LCKy**: Port x lock bit y (y = 0..15)

These bits are read/write but can only be written when the LCKK bit is '0.

0: Port configuration not locked

1: Port configuration locked

8.4.9 GPIO alternate function low register (GPIOx_AFRL) (x = A..E and H)

Address offset: 0x20

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRLy**: Alternate function selection for port x bit y (y = 0..7)

These bits are written by software to configure alternate function I/Os

AFRLy selection:

0000: AF0	1000: AF8
0001: AF1	1001: AF9
0010: AF2	1010: AF10
0011: AF3	1011: AF11
0100: AF4	1100: AF12
0101: AF5	1101: AF13
0110: AF6	1110: AF14
0111: AF7	1111: AF15

8.4.10 GPIO alternate function high register (GPIOx_AFRH)
(x = A..E and H)

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **AFRHy**: Alternate function selection for port x bit y (y = 8..15)
These bits are written by software to configure alternate function I/Os

AFRHy selection:	
0000: AF0	1000: AF8
0001: AF1	1001: AF9
0010: AF2	1010: AF10
0011: AF3	1011: AF11
0100: AF4	1100: AF12
0101: AF5	1101: AF13
0110: AF6	1110: AF14
0111: AF7	1111: AF15



8.4.11 GPIO register map

The following table gives the GPIO register map and the reset values.

Table 26. GPIO register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]		
	Reset value	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x00	GPIOB_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	
0x00	GPIOx_MODER (where x = C..E and H)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x04	GPIOx_OTYPER (where x = A..E and H)	Reserved																	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	GPIOx_OSPEEDR (where x = C..E andH)	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x08	GPIOA_OSPEEDER	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]		
	Reset value	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x08	GPIOB_OSPEEDR	OSPEEDR15[1:0]		OSPEEDR14[1:0]		OSPEEDR13[1:0]		OSPEEDR12[1:0]		OSPEEDR11[1:0]		OSPEEDR10[1:0]		OSPEEDR9[1:0]		OSPEEDR8[1:0]		OSPEEDR7[1:0]		OSPEEDR6[1:0]		OSPEEDR5[1:0]		OSPEEDR4[1:0]		OSPEEDR3[1:0]		OSPEEDR2[1:0]		OSPEEDR1[1:0]		OSPEEDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C	GPIOA_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]		
	Reset value	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Table 26. GPIO register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0C	GPIOB_PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0		
0x0C	GPIOx_PUPDR (where x = C..E and H)	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x10	GPIOx_IDR (where x = A..E and H)	Reserved																IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0	
	Reset value																	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	
0x14	GPIOx_ODR (where x = A..E and H)	Reserved																ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x18	GPIOx_BSRR (where x = A..E and H)	BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0	BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x1C	GPIOx_LCKR (where x = A..E and H)	Reserved																LCKK	LCK15	LCK14	LCK13	LCK12	LCK11	LCK10	LCK9	LCK8	LCK7	LCK6	LCK5	LCK4	LCK3	LCK2	LCK1	LCK0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	GPIOx_AFR1 (where x = A..E and H)	AFRL7[3:0]				AFRL6[3:0]				AFRL5[3:0]				AFRL4[3:0]				AFRL3[3:0]				AFRL2[3:0]				AFRL1[3:0]				AFRL0[3:0]				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x24	GPIOx_AFRH (where x = A..E and H)	AFRH15[3:0]				AFRH14[3:0]				AFRH13[3:0]				AFRH12[3:0]				AFRH11[3:0]				AFRH10[3:0]				AFRH9[3:0]				AFRH8[3:0]				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

Refer to [Section 2.3: Memory map](#) for the register boundary addresses.

9 DMA controller (DMA)

9.1 DMA introduction

Direct memory access (DMA) is used in order to provide high-speed data transfer between peripherals and memory and between memory and memory. Data can be quickly moved by DMA without any CPU action. This keeps CPU resources free for other operations.

The DMA controller combines a powerful dual AHB master bus architecture with independent FIFO to optimize the bandwidth of the system, based on a complex bus matrix architecture.

The two DMA controllers have 16 streams in total (8 for each controller), each dedicated to managing memory access requests from one or more peripherals. Each stream can have up to 8 channels (requests) in total. And each has an arbiter for handling the priority between DMA requests.

9.2 DMA main features

The main DMA features are:

- Dual AHB master bus architecture, one dedicated to memory accesses and one dedicated to peripheral accesses
 - AHB slave programming interface supporting only 32-bit accesses
 - 8 streams for each DMA controller, up to 8 channels (requests) per stream
 - Four-word depth 32 first-in, first-out memory buffers (FIFOs) per stream, that can be used in FIFO mode or direct mode:
 - FIFO mode: with threshold level software selectable between 1/4, 1/2 or 3/4 of the FIFO size
 - Direct mode
- Each DMA request immediately initiates a transfer from/to the memory. When it is configured in direct mode (FIFO disabled), to transfer data in memory-to-peripheral mode, the DMA preloads only one data from the memory to the internal

FIFO to ensure an immediate data transfer as soon as a DMA request is triggered by a peripheral.

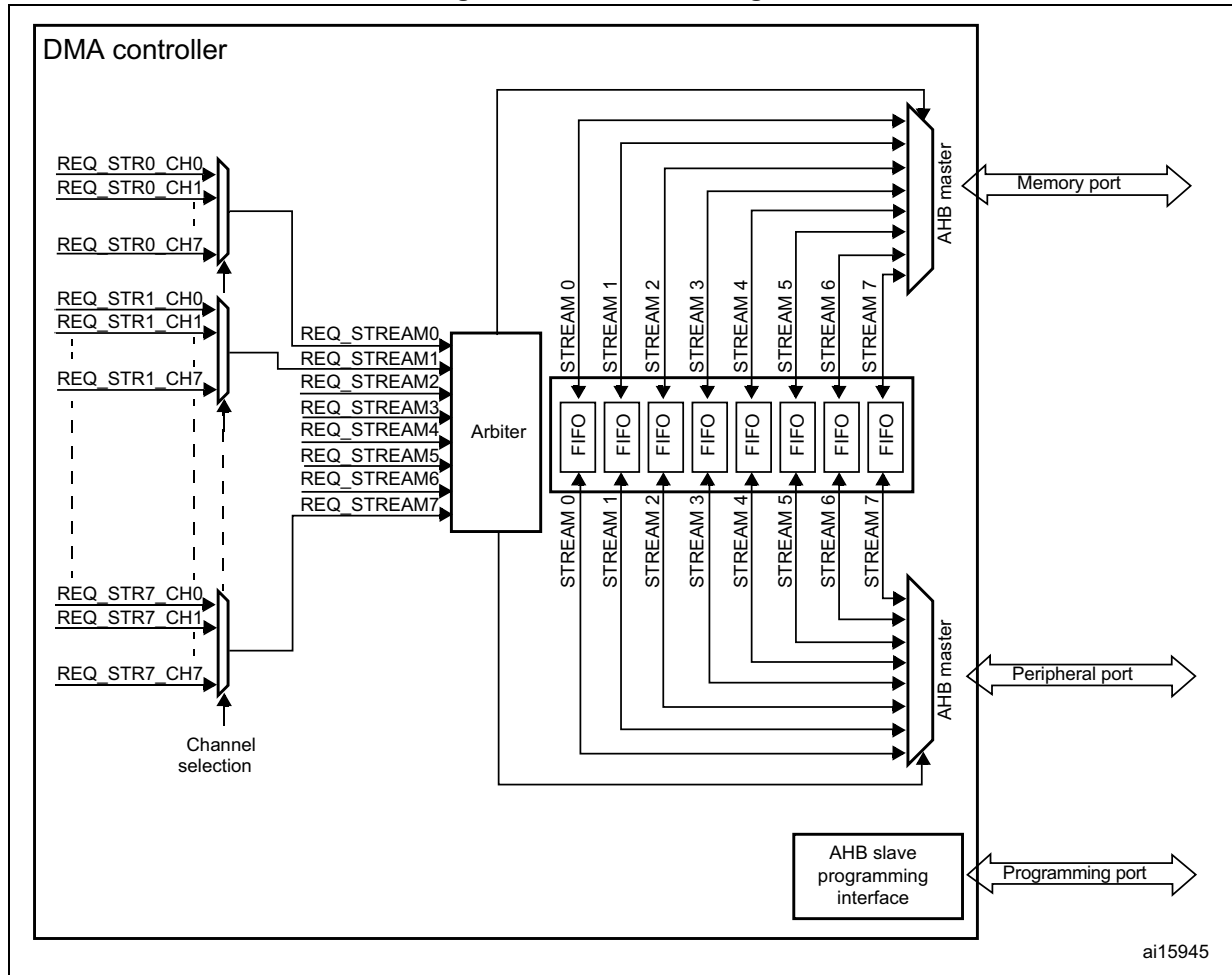
- Each stream can be configured by hardware to be:
 - a regular channel that supports peripheral-to-memory, memory-to-peripheral and memory-to-memory transfers
 - a double buffer channel that also supports double buffering on the memory side
- Each of the 8 streams are connected to dedicated hardware DMA channels (requests)
- Priorities between DMA stream requests are software-programmable (4 levels consisting of very high, high, medium, low) or hardware in case of equality (request 0 has priority over request 1, etc.)
- Each stream also supports software trigger for memory-to-memory transfers (only available for the DMA2 controller)
- Each stream request can be selected among up to 8 possible channel requests. This selection is software-configurable and allows several peripherals to initiate DMA requests
- The number of data items to be transferred can be managed either by the DMA controller or by the peripheral:
 - DMA flow controller: the number of data items to be transferred is software-programmable from 1 to 65535
 - Peripheral flow controller: the number of data items to be transferred is unknown and controlled by the source or the destination peripheral that signals the end of the transfer by hardware
- Independent source and destination transfer width (byte, half-word, word): when the data widths of the source and destination are not equal, the DMA automatically packs/unpacks the necessary transfers to optimize the bandwidth. This feature is only available in FIFO mode
- Incrementing or non-incrementing addressing for source and destination
- Supports incremental burst transfers of 4, 8 or 16 beats. The size of the burst is software-configurable, usually equal to half the FIFO size of the peripheral
- Each stream supports circular buffer management
- 5 event flags (DMA Half Transfer, DMA Transfer complete, DMA Transfer Error, DMA FIFO Error, Direct Mode Error) logically ORed together in a single interrupt request for each stream

9.3 DMA functional description

9.3.1 General description

Figure 22 shows the block diagram of a DMA.

Figure 22. DMA block diagram



The DMA controller performs direct memory transfer: as an AHB master, it can take the control of the AHB bus matrix to initiate AHB transactions.

It can carry out the following transactions:

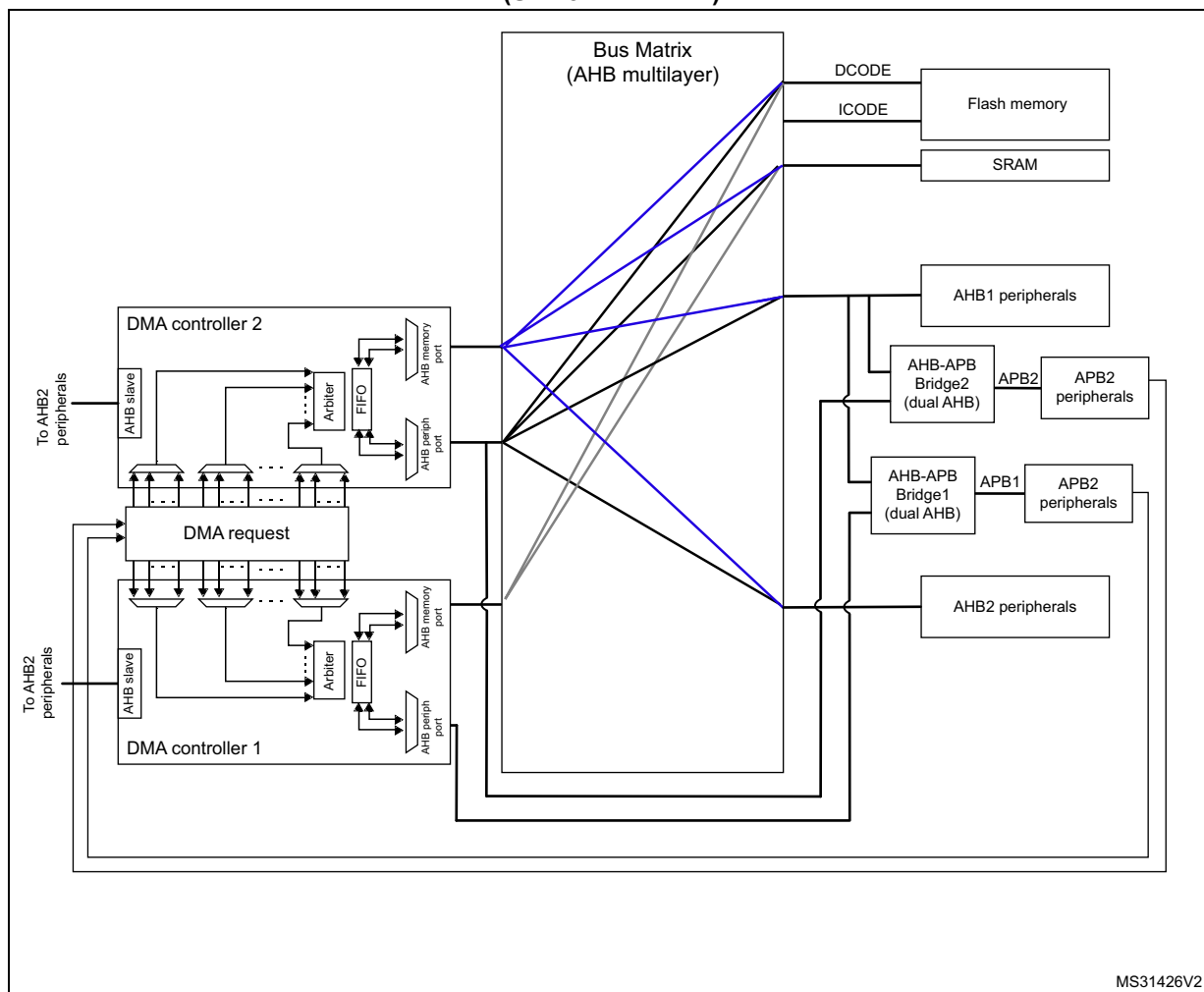
- peripheral-to-memory
- memory-to-peripheral
- memory-to-memory

The DMA controller provides two AHB master ports: the *AHB memory port*, intended to be connected to memories and the *AHB peripheral port*, intended to be connected to peripherals. However, to allow memory-to-memory transfers, the *AHB peripheral port* must also have access to the memories.

The AHB slave port is used to program the DMA controller (it supports only 32-bit accesses).

See [Figure 23](#) for the implementation of the system of two DMA controllers.

Figure 23. System implementation of the two DMA controllers (STM32F411xC/E)



1. The DMA1 controller AHB peripheral port is not connected to the bus matrix like DMA2 controller. As a result, only DMA2 streams are able to perform memory-to-memory transfers.

9.3.2 DMA transactions

A DMA transaction consists of a sequence of a given number of data transfers. The number of data items to be transferred and their width (8-bit, 16-bit or 32-bit) are software-programmable.

Each DMA transfer consists of three operations:

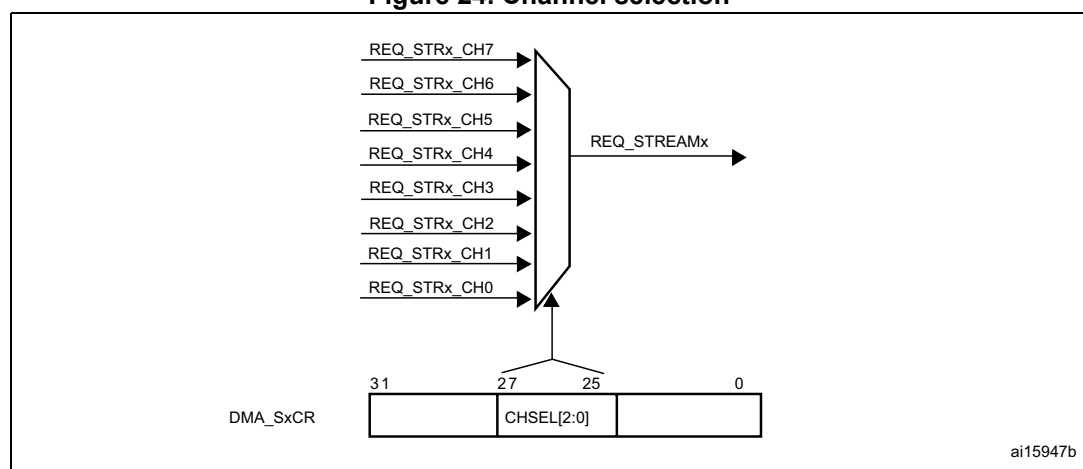
- A loading from the peripheral data register or a location in memory, addressed through the DMA_SxPAR or DMA_SxM0AR register
- A storage of the data loaded to the peripheral data register or a location in memory addressed through the DMA_SxPAR or DMA_SxM0AR register
- A post-decrement of the DMA_SxNDTR register, which contains the number of transactions that still have to be performed

After an event, the peripheral sends a request signal to the DMA controller. The DMA controller serves the request depending on the channel priorities. As soon as the DMA controller accesses the peripheral, an Acknowledge signal is sent to the peripheral by the DMA controller. The peripheral releases its request as soon as it gets the Acknowledge signal from the DMA controller. Once the request has been deasserted by the peripheral, the DMA controller releases the Acknowledge signal. If there are more requests, the peripheral can initiate the next transaction.

9.3.3 Channel selection

Each stream is associated with a DMA request that can be selected out of 8 possible channel requests. The selection is controlled by the CHSEL[2:0] bits in the DMA_SxCR register.

Figure 24. Channel selection



The 8 requests from the peripherals (TIM, ADC, SPI, I2C, etc.) are independently connected to each channel and their connection depends on the product implementation.

See the following table(s) for examples of DMA request mappings.

Table 27. DMA1 request mapping (STM32F411xC/E)

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	SPI3_RX	I2C1_TX	SPI3_RX	SPI2_RX	SPI2_TX	SPI3_TX	-	SPI3_TX
Channel 1	I2C1_RX	I2C3_RX	-	-	-	I2C1_RX	I2C1_TX	I2C1_TX
Channel 2	TIM4_CH1	-	I2S3_EXT_RX	TIM4_CH2	I2S2_EXT_TX	I2S3_EXT_TX	TIM4_UP	TIM4_CH3
Channel 3	I2S3_EXT_RX	TIM2_UP TIM2_CH3	I2C3_RX	I2S2_EXT_RX	I2C3_TX	TIM2_CH1	TIM2_CH2 TIM2_CH4	TIM2_UP TIM2_CH4
Channel 4	-	-	-	-	-	USART2_RX	USART2_TX	-
Channel 5	-	-	TIM3_CH4 TIM3_UP	-	TIM3_CH1 TIM3_TRIG	TIM3_CH2	-	TIM3_CH3
Channel 6	TIM5_CH3 TIM5_UP	TIM5_CH4 TIM5_TRIG	TIM5_CH1	TIM5_CH4 TIM5_TRIG	TIM5_CH2	I2C3_TX	TIM5_UP	USART2_RX
Channel 7	-	-	I2C2_RX	I2C2_RX	-	-	-	I2C2_TX

Table 28. DMA2 request mapping (STM32F411xC/E)

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	ADC1	-	-	-	ADC1	-	TIM1_CH1 TIM1_CH2 TIM1_CH3	-
Channel 1	-	-	-	-	-	-	-	-
Channel 2	-	-	SPI1_TX	SPI5_RX	SPI5_TX	-	-	-
Channel 3	SPI1_RX	-	SPI1_RX	SPI1_TX	-	SPI1_TX	-	-
Channel 4	SPI4_RX	SPI4_TX	USART1_RX	SDIO	SPI4_RX	USART1_RX	SDIO	USART1_TX
Channel 5	-	USART6_RX	USART6_RX	SPI4_RX	SPI4_TX	SPI5_TX	USART6_TX	USART6_TX
Channel 6	TIM1_TRIG	TIM1_CH1	TIM1_CH2	TIM1_CH1	TIM1_CH4 TIM1_TRIG TIM1_COM	TIM1_UP	TIM1_CH3	-
Channel 7	-	-	-	-	-	SPI5_RX	SPI5_TX	-

9.3.4 Arbiter

An arbiter manages the 8 DMA stream requests based on their priority for each of the two AHB master ports (memory and peripheral ports) and launches the peripheral/memory access sequences.

Priorities are managed in two stages:

- Software: each stream priority can be configured in the DMA_SxCR register. There are four levels:
 - Very high priority
 - High priority
 - Medium priority
 - Low priority
- Hardware: If two requests have the same software priority level, the stream with the lower number takes priority over the stream with the higher number. For example, Stream 2 takes priority over Stream 4.

9.3.5 DMA streams

Each of the 8 DMA controller streams provides a unidirectional transfer link between a source and a destination.

Each stream can be configured to perform:

- Regular type transactions: memory-to-peripherals, peripherals-to-memory or memory-to-memory transfers
- Double-buffer type transactions: double buffer transfers using two memory pointers for the memory (while the DMA is reading/writing from/to a buffer, the application can write/read to/from the other buffer).

The amount of data to be transferred (up to 65535) is programmable and related to the source width of the peripheral that requests the DMA transfer connected to the peripheral AHB port. The register that contains the amount of data items to be transferred is decremented after each transaction.

9.3.6 Source, destination and transfer modes

Both source and destination transfers can address peripherals and memories in the entire 4 GB area, at addresses comprised between 0x0000 0000 and 0xFFFF FFFF.

The direction is configured using the DIR[1:0] bits in the DMA_SxCR register and offers three possibilities: memory-to-peripheral, peripheral-to-memory or memory-to-memory transfers. [Table 29](#) describes the corresponding source and destination addresses.

Table 29. Source and destination address

Bits DIR[1:0] of the DMA_SxCR register	Direction	Source address	Destination address
00	Peripheral-to-memory	DMA_SxPAR	DMA_SxM0AR
01	Memory-to-peripheral	DMA_SxM0AR	DMA_SxPAR
10	Memory-to-memory	DMA_SxPAR	DMA_SxM0AR
11	reserved	-	-

When the data width (programmed in the PSIZE or MSIZE bits in the DMA_SxCR register) is a half-word or a word, respectively, the peripheral or memory address written into the DMA_SxPAR or DMA_SxM0AR/M1AR registers has to be aligned on a word or half-word address boundary, respectively.

Peripheral-to-memory mode

Figure 25 describes this mode.

When this mode is enabled (by setting the bit EN in the DMA_SxCR register), each time a peripheral request occurs, the stream initiates a transfer from the source to fill the FIFO.

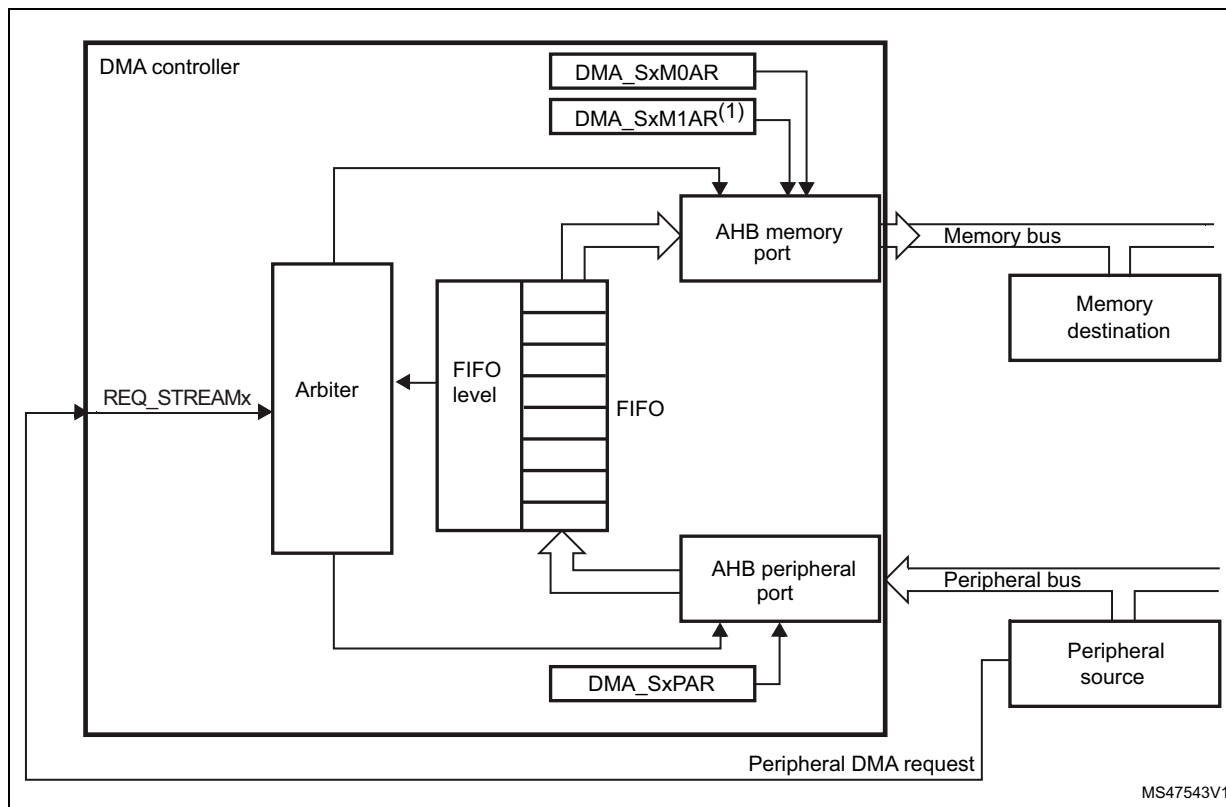
When the threshold level of the FIFO is reached, the contents of the FIFO are drained and stored into the destination.

The transfer stops once the DMA_SxNDTR register reaches zero, when the peripheral requests the end of transfers (in case of a peripheral flow controller) or when the EN bit in the DMA_SxCR register is cleared by software.

In direct mode (when the DMDIS value in the DMA_SxFCR register is '0'), the threshold level of the FIFO is not used: after each single data transfer from the peripheral to the FIFO, the corresponding data are immediately drained and stored into the destination.

The stream has access to the AHB source or destination port only if the arbitration of the corresponding stream is won. This arbitration is performed using the priority defined for each stream using the PL[1:0] bits in the DMA_SxCR register.

Figure 25. Peripheral-to-memory mode



1. For double-buffer mode.

Memory-to-peripheral mode

Figure 26 describes this mode.

When this mode is enabled (by setting the EN bit in the DMA_SxCR register), the stream immediately initiates transfers from the source to entirely fill the FIFO.

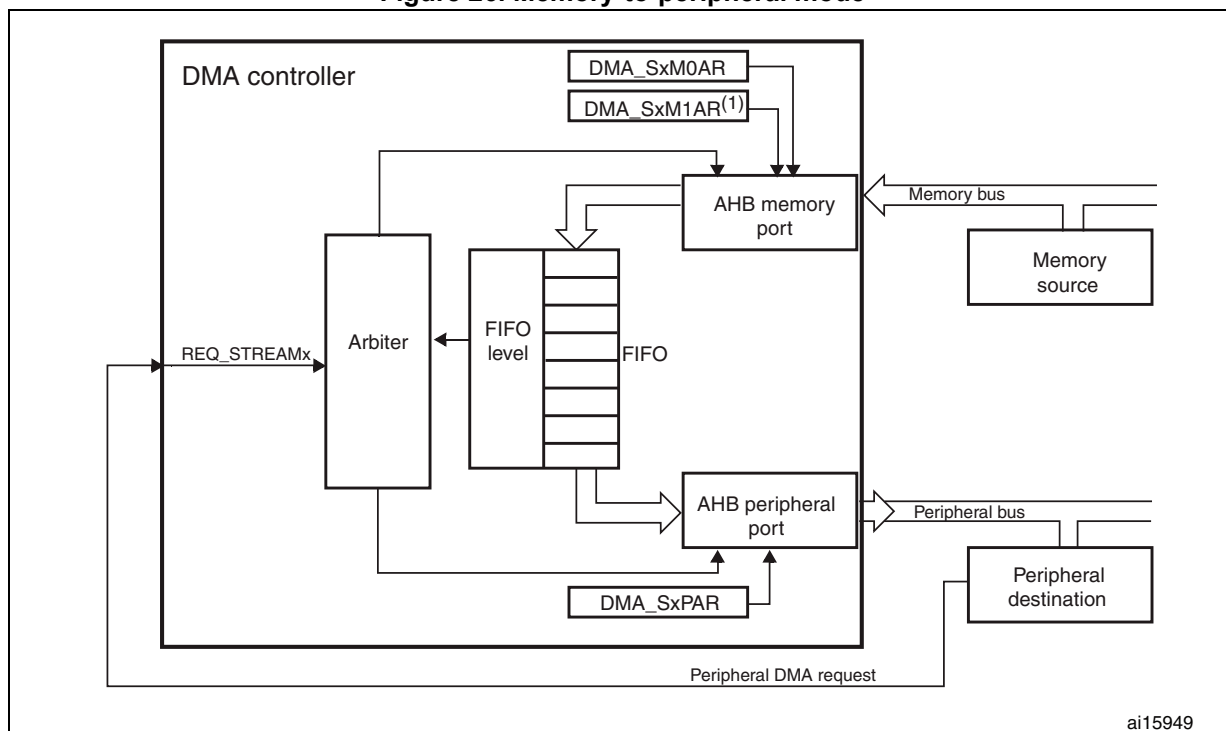
Each time a peripheral request occurs, the contents of the FIFO are drained and stored into the destination. When the level of the FIFO is lower than or equal to the predefined threshold level, the FIFO is fully reloaded with data from the memory.

The transfer stops once the DMA_SxNDTR register reaches zero, when the peripheral requests the end of transfers (in case of a peripheral flow controller) or when the EN bit in the DMA_SxCR register is cleared by software.

In direct mode (when the DMDIS value in the DMA_SxFCR register is '0'), the threshold level of the FIFO is not used. Once the stream is enabled, the DMA preloads the first data to transfer into an internal FIFO. As soon as the peripheral requests a data transfer, the DMA transfers the preloaded value into the configured destination. It then reloads again the empty internal FIFO with the next data to be transfer. The preloaded data size corresponds to the value of the PSIZE bitfield in the DMA_SxCR register.

The stream has access to the AHB source or destination port only if the arbitration of the corresponding stream is won. This arbitration is performed using the priority defined for each stream using the PL[1:0] bits in the DMA_SxCR register.

Figure 26. Memory-to-peripheral mode



1. For double-buffer mode.

Memory-to-memory mode

The DMA channels can also work without being triggered by a request from a peripheral. This is the memory-to-memory mode, described in [Figure 27](#).

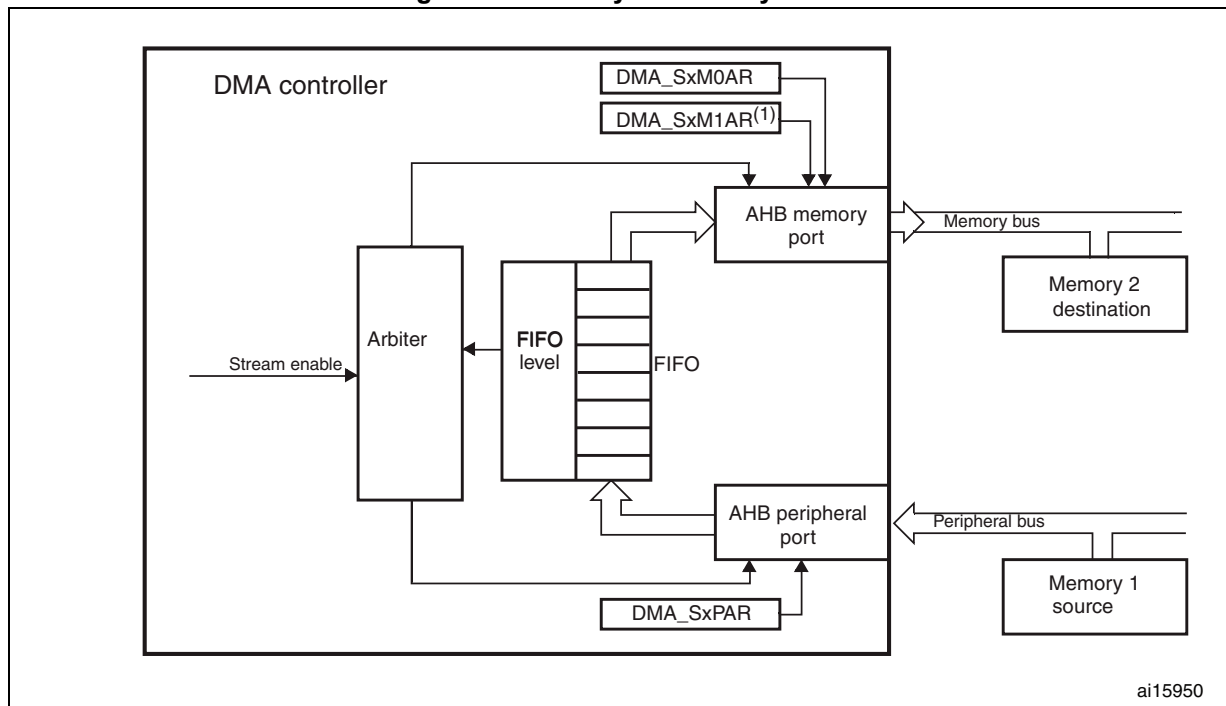
When the stream is enabled by setting the Enable bit (EN) in the DMA_SxCR register, the stream immediately starts to fill the FIFO up to the threshold level. When the threshold level is reached, the FIFO contents are drained and stored into the destination.

The transfer stops once the DMA_SxNDTR register reaches zero or when the EN bit in the DMA_SxCR register is cleared by software.

The stream has access to the AHB source or destination port only if the arbitration of the corresponding stream is won. This arbitration is performed using the priority defined for each stream using the PL[1:0] bits in the DMA_SxCR register.

Note: When memory-to-memory mode is used, the Circular and direct modes are not allowed. Only the DMA2 controller is able to perform memory-to-memory transfers.

Figure 27. Memory-to-memory mode



1. For double-buffer mode.

9.3.7 Pointer incrementation

Peripheral and memory pointers can optionally be automatically post-incremented or kept constant after each transfer depending on the PINC and MINC bits in the DMA_SxCR register.

Disabling the Increment mode is useful when the peripheral source or destination data are accessed through a single register.

If the Increment mode is enabled, the address of the next transfer will be the address of the previous one incremented by 1 (for bytes), 2 (for half-words) or 4 (for words) depending on the data width programmed in the PSIZE or MSIZE bits in the DMA_SxCR register.

In order to optimize the packing operation, it is possible to fix the increment offset size for the peripheral address whatever the size of the data transferred on the AHB peripheral port. The PINCOS bit in the DMA_SxCR register is used to align the increment offset size with the data size on the peripheral AHB port, or on a 32-bit address (the address is then incremented by 4). The PINCOS bit has an impact on the AHB peripheral port only.

If PINCOS bit is set, the address of the next transfer is the address of the previous one incremented by 4 (automatically aligned on a 32-bit address) whatever the PSIZE value. The AHB memory port, however, is not impacted by this operation.

9.3.8 Circular mode

The Circular mode is available to handle circular buffers and continuous data flows (e.g. ADC scan mode). This feature can be enabled using the CIRC bit in the DMA_SxCR register.

When the circular mode is activated, the number of data items to be transferred is automatically reloaded with the initial value programmed during the stream configuration phase, and the DMA requests continue to be served.

Note: *In the circular mode, it is mandatory to respect the following rule in case of a burst mode configured for memory:*

$DMA_SxNDTR = \text{Multiple of } ((Mburst\ beat) \times (Msize)/(Psize))$, where:

- $(Mburst\ beat) = 4, 8\text{ or }16$ (depending on the MBURST bits in the DMA_SxCR register)*
- $((Msize)/(Psize)) = 1, 2, 4, 1/2\text{ or }1/4$ (Msize and Psize represent the MSIZE and PSIZE bits in the DMA_SxCR register. They are byte dependent)*
- $DMA_SxNDTR = \text{Number of data items to transfer on the AHB peripheral port}$*

For example: Mburst beat = 8 (INCR8), MSIZE = '00' (byte) and PSIZE = '01' (half-word), in this case: DMA_SxNDTR must be a multiple of $(8 \times 1/2 = 4)$.

If this formula is not respected, the DMA behavior and data integrity are not guaranteed.

NDTR must also be a multiple of the Peripheral burst size multiplied by the peripheral data size, otherwise this could result in a bad DMA behavior.

9.3.9 Double buffer mode

This mode is available for all the DMA1 and DMA2 streams.

The Double buffer mode is enabled by setting the DBM bit in the DMA_SxCR register.

A double-buffer stream works as a regular (single buffer) stream with the difference that it has two memory pointers. When the Double buffer mode is enabled, the Circular mode is automatically enabled (CIRC bit in DMA_SxCR is don't care) and at each end of transaction, the memory pointers are swapped.

In this mode, the DMA controller swaps from one memory target to another at each end of transaction. This allows the software to process one memory area while the second memory area is being filled/used by the DMA transfer. The double-buffer stream can work in both directions (the memory can be either the source or the destination) as described in [Table 30: Source and destination address registers in Double buffer mode \(DBM=1\)](#).

Note: In Double buffer mode, it is possible to update the base address for the AHB memory port on-the-fly (DMA_SxM0AR or DMA_SxM1AR) when the stream is enabled, by respecting the following conditions:

- When the CT bit is '0' in the DMA_SxCR register, the DMA_SxM1AR register can be written. Attempting to write to this register while CT = '1' sets an error flag (TEIF) and the stream is automatically disabled.
- When the CT bit is '1' in the DMA_SxCR register, the DMA_SxM0AR register can be written. Attempting to write to this register while CT = '0', sets an error flag (TEIF) and the stream is automatically disabled.

To avoid any error condition, it is advised to change the base address as soon as the TCIF flag is asserted because, at this point, the targeted memory must have changed from memory 0 to 1 (or from 1 to 0) depending on the value of CT in the DMA_SxCR register in accordance with one of the two above conditions.

For all the other modes (except the Double buffer mode), the memory address registers are write-protected as soon as the stream is enabled.

Table 30. Source and destination address registers in Double buffer mode (DBM=1)

Bits DIR[1:0] of the DMA_SxCR register	Direction	Source address	Destination address
00	Peripheral-to-memory	DMA_SxPAR	DMA_SxM0AR / DMA_SxM1AR
01	Memory-to-peripheral	DMA_SxM0AR / DMA_SxM1AR	DMA_SxPAR
10	Not allowed ⁽¹⁾		
11	Reserved	-	-

1. When the Double buffer mode is enabled, the Circular mode is automatically enabled. Since the memory-to-memory mode is not compatible with the Circular mode, when the Double buffer mode is enabled, it is not allowed to configure the memory-to-memory mode.

9.3.10 Programmable data width, packing/unpacking, endianness

The number of data items to be transferred has to be programmed into DMA_SxNDTR (number of data items to transfer bit, NDT) before enabling the stream (except when the flow controller is the peripheral, PFCTRL bit in DMA_SxCR is set).

When using the internal FIFO, the data widths of the source and destination data are programmable through the PSIZE and MSIZE bits in the DMA_SxCR register (can be 8-, 16- or 32-bit).

When PSIZE and MSIZE are not equal:

- The data width of the number of data items to transfer, configured in the DMA_SxNDTR register is equal to the width of the peripheral bus (configured by the PSIZE bits in the DMA_SxCR register). For instance, in case of peripheral-to-memory, memory-to-peripheral or memory-to-memory transfers and if the PSIZE[1:0] bits are configured for half-word, the number of bytes to be transferred is equal to $2 \times \text{NDT}$.
- The DMA controller only copes with little-endian addressing for both source and destination. This is described in [Table 31: Packing/unpacking & endian behavior \(bit PINC = MINC = 1\)](#).

This packing/unpacking procedure may present a risk of data corruption when the operation is interrupted before the data are completely packed/unpacked. So, to ensure data coherence, the stream may be configured to generate burst transfers: in this case, each group of transfers belonging to a burst are indivisible (refer to [Section 9.3.11: Single and burst transfers](#)).

In direct mode (DMDIS = 0 in the DMA_SxFCR register), the packing/unpacking of data is not possible. In this case, it is not allowed to have different source and destination transfer data widths: both are equal and defined by the PSIZE bits in the DMA_SxCR MSIZE bits are don't care).

Table 31. Packing/unpacking & endian behavior (bit PINC = MINC = 1)

AHB memory port width	AHB peripheral port width	Number of data items to transfer (NDT)	Memory transfer number	Memory port address / byte lane	Peripheral transfer number	Peripheral port address / byte lane	
						PINCOS = 1	PINCOS = 0
8	8	4	1	0x0 / B0[7:0]	1	0x0 / B0[7:0]	0x0 / B0[7:0]
			2	0x1 / B1[7:0]	2	0x4 / B1[7:0]	0x1 / B1[7:0]
			3	0x2 / B2[7:0]	3	0x8 / B2[7:0]	0x2 / B2[7:0]
			4	0x3 / B3[7:0]	4	0xC / B3[7:0]	0x3 / B3[7:0]
8	16	2	1	0x0 / B0[7:0]	1	0x0 / B1 B0[15:0]	0x0 / B1 B0[15:0]
			2	0x1 / B1[7:0]	2	0x4 / B3 B2[15:0]	0x2 / B3 B2[15:0]
			3	0x2 / B2[7:0]			
			4	0x3 / B3[7:0]			
8	32	1	1	0x0 / B0[7:0]	1	0x0 / B3 B2 B1 B0[31:0]	0x0 / B3 B2 B1 B0[31:0]
			2	0x1 / B1[7:0]			
			3	0x2 / B2[7:0]			
			4	0x3 / B3[7:0]			
16	8	4	1	0x0 / B1 B0[15:0]	1	0x0 / B0[7:0]	0x0 / B0[7:0]
			2	0x2 / B3 B2[15:0]	2	0x4 / B1[7:0]	0x1 / B1[7:0]
			3		3	0x8 / B2[7:0]	0x2 / B2[7:0]
			4		4	0xC / B3[7:0]	0x3 / B3[7:0]
16	16	2	1	0x0 / B1 B0[15:0]	1	0x0 / B1 B0[15:0]	0x0 / B1 B0[15:0]
			2	0x2 / B1 B0[15:0]	2	0x4 / B3 B2[15:0]	0x2 / B3 B2[15:0]
16	32	1	1	0x0 / B1 B0[15:0]	1	0x0 / B3 B2 B1 B0[31:0]	0x0 / B3 B2 B1 B0[31:0]
			2	0x2 / B3 B2[15:0]			
32	8	4	1	0x0 / B3 B2 B1 B0[31:0]	1	0x0 / B0[7:0]	0x0 / B0[7:0]
					2	0x4 / B1[7:0]	0x1 / B1[7:0]
					3	0x8 / B2[7:0]	0x2 / B2[7:0]
					4	0xC / B3[7:0]	0x3 / B3[7:0]
32	16	2	1	0x0 / B3 B2 B1 B0[31:0]	1	0x0 / B1 B0[15:0]	0x0 / B1 B0[15:0]
					2	0x4 / B3 B2[15:0]	0x2 / B3 B2[15:0]
32	32	1	1	0x0 / B3 B2 B1 B0 [31:0]	1	0x0 / B3 B2 B1 B0 [31:0]	0x0 / B3 B2 B1 B0[31:0]

Note: Peripheral port may be the source or the destination (it could also be the memory source in the case of memory-to-memory transfer).

PSIZE, MSIZE and NDT[15:0] have to be configured so as to ensure that the last transfer will not be incomplete. This can occur when the data width of the peripheral port (PSIZE bits) is lower than the data width of the memory port (MSIZE bits). This constraint is summarized in [Table 32](#).

Table 32. Restriction on NDT versus PSIZE and MSIZE

PSIZE[1:0] of DMA_SxCR	MSIZE[1:0] of DMA_SxCR	NDT[15:0] of DMA_SxNDTR
00 (8-bit)	01 (16-bit)	must be a multiple of 2
00 (8-bit)	10 (32-bit)	must be a multiple of 4
01 (16-bit)	10 (32-bit)	must be a multiple of 2

9.3.11 Single and burst transfers

The DMA controller can generate single transfers or incremental burst transfers of 4, 8 or 16 beats.

The size of the burst is configured by software independently for the two AHB ports by using the MBURST[1:0] and PBURST[1:0] bits in the DMA_SxCR register.

The burst size indicates the number of beats in the burst, not the number of bytes transferred.

To ensure data coherence, each group of transfers that form a burst are indivisible: AHB transfers are locked and the arbiter of the AHB bus matrix does not degrant the DMA master during the sequence of the burst transfer.

Depending on the single or burst configuration, each DMA request initiates a different number of transfers on the AHB peripheral port:

- When the AHB peripheral port is configured for single transfers, each DMA request generates a data transfer of a byte, half-word or word depending on the PSIZE[1:0] bits in the DMA_SxCR register
- When the AHB peripheral port is configured for burst transfers, each DMA request generates 4, 8 or 16 beats of byte, half word or word transfers depending on the PBURST[1:0] and PSIZE[1:0] bits in the DMA_SxCR register.

The same as above has to be considered for the AHB memory port considering the MBURST and MSIZE bits.

In direct mode, the stream can only generate single transfers and the MBURST[1:0] and PBURST[1:0] bits are forced by hardware.

The address pointers (DMA_SxPAR or DMA_SxM0AR registers) must be chosen so as to ensure that all transfers within a burst block are aligned on the address boundary equal to the size of the transfer.

The burst configuration has to be selected in order to respect the AHB protocol, where bursts must *not* cross the 1 KB address boundary because the minimum address space that can be allocated to a single slave is 1 KB. This means that the 1 KB address boundary should not be crossed by a burst block transfer, otherwise an AHB error would be generated, that is not reported by the DMA registers.

9.3.12 FIFO

FIFO structure

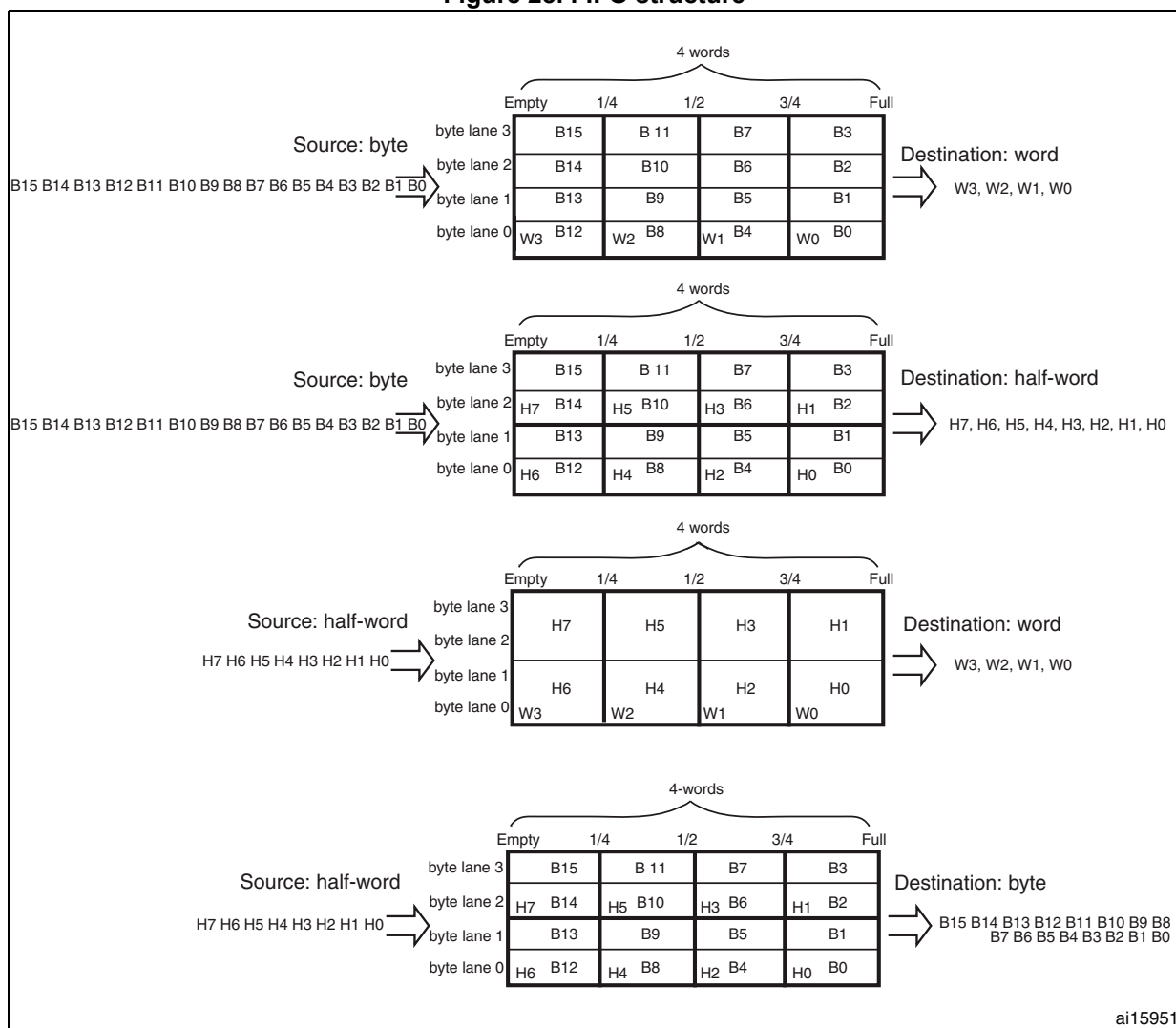
The FIFO is used to temporarily store data coming from the source before transmitting them to the destination.

Each stream has an independent 4-word FIFO and the threshold level is software-configurable between 1/4, 1/2, 3/4 or full.

To enable the use of the FIFO threshold level, the direct mode must be disabled by setting the DMDIS bit in the DMA_SxFCR register.

The structure of the FIFO differs depending on the source and destination data widths, and is described in [Figure 28: FIFO structure](#).

Figure 28. FIFO structure



FIFO threshold and burst configuration

Caution is required when choosing the FIFO threshold (bits FTH[1:0] of the DMA_SxFCR register) and the size of the memory burst (MBURST[1:0] of the DMA_SxCR register): The content pointed by the FIFO threshold must exactly match to an integer number of memory burst transfers. If this is not in the case, a FIFO error (flag FEIFx of the DMA_HISR or DMA_LISR register) will be generated when the stream is enabled, then the stream will be automatically disabled. The allowed and forbidden configurations are described in the [Table 33: FIFO threshold configurations](#).

Table 33. FIFO threshold configurations

MSIZE	FIFO level	MBURST = INCR4	MBURST = INCR8	MBURST = INCR16	
Byte	1/4	1 burst of 4 beats	forbidden	forbidden	
	1/2	2 bursts of 4 beats	1 burst of 8 beats		
	3/4	3 bursts of 4 beats	forbidden		
	Full	4 bursts of 4 beats	2 bursts of 8 beats	1 burst of 16 beats	
Half-word	1/4	forbidden	forbidden	forbidden	
	1/2	1 burst of 4 beats			
	3/4	forbidden			
	Full	2 bursts of 4 beats	1 burst of 8 beats		
Word	1/4	forbidden	forbidden		
	1/2				
	3/4				
	Full	1 burst of 4 beats			

In all cases, the burst size multiplied by the data size must not exceed the FIFO size (data size can be: 1 (byte), 2 (half-word) or 4 (word)).

Incomplete Burst transfer at the end of a DMA transfer may happen if one of the following conditions occurs:

- For the AHB peripheral port configuration: the total number of data items (set in the DMA_SxNDTR register) is not a multiple of the burst size multiplied by the data size
- For the AHB memory port configuration: the number of remaining data items in the FIFO to be transferred to the memory is not a multiple of the burst size multiplied by the data size

In such cases, the remaining data to be transferred will be managed in single mode by the DMA, even if a burst transaction was requested during the DMA stream configuration.

Note: *When burst transfers are requested on the peripheral AHB port and the FIFO is used (DMDIS = 1 in the DMA_SxCR register), it is mandatory to respect the following rule to avoid permanent underrun or overrun conditions, depending on the DMA stream direction:*

If $(PBURST \times PSIZE) = FIFO_SIZE$ (4 words), $FIFO_Threshold = 3/4$ is forbidden with $PSIZE = 1, 2$ or 4 and $PBURST = 4, 8$ or 16 .

This rule ensures that enough FIFO space at a time will be free to serve the request from the peripheral.

FIFO flush

The FIFO can be flushed when the stream is disabled by resetting the EN bit in the DMA_SxCR register and when the stream is configured to manage peripheral-to-memory or memory-to-memory transfers: If some data are still present in the FIFO when the stream is disabled, the DMA controller continues transferring the remaining data to the destination (even though stream is effectively disabled). When this flush is completed, the transfer complete status bit (TCIFx) in the DMA_LISR or DMA_HISR register is set.

The remaining data counter DMA_SxNDTR keeps the value in this case to indicate how many data items are currently available in the destination memory.

Note that during the FIFO flush operation, if the number of remaining data items in the FIFO to be transferred to memory (in bytes) is less than the memory data width (for example 2 bytes in FIFO while MSIZE is configured to word), data will be sent with the data width set in the MSIZE bit in the DMA_SxCR register. This means that memory will be written with an undesired value. The software may read the DMA_SxNDTR register to determine the memory area that contains the good data (start address and last address).

If the number of remaining data items in the FIFO is lower than a burst size (if the MBURST bits in DMA_SxCR register are set to configure the stream to manage burst on the AHB memory port), single transactions will be generated to complete the FIFO flush.

Direct mode

By default, the FIFO operates in direct mode (DMDIS bit in the DMA_SxFCR is reset) and the FIFO threshold level is not used. This mode is useful when the system requires an immediate and single transfer to or from the memory after each DMA request.

When the DMA is configured in direct mode (FIFO disabled), to transfer data in memory-to-peripheral mode, the DMA preloads one data from the memory to the internal FIFO to ensure an immediate data transfer as soon as a DMA request is triggered by a peripheral.

To avoid saturating the FIFO, it is recommended to configure the corresponding stream with a high priority.

This mode is restricted to transfers where:

- The source and destination transfer widths are equal and both defined by the PSIZE[1:0] bits in DMA_SxCR (MSIZE[1:0] bits are don't care)
- Burst transfers are not possible (PBURST[1:0] and MBURST[1:0] bits in DMA_SxCR are don't care)

Direct mode must not be used when implementing memory-to-memory transfers.

9.3.13 DMA transfer completion

Different events can generate an end of transfer by setting the TCIFx bit in the DMA_LISR or DMA_HISR status register:

- In DMA flow controller mode:
 - The DMA_SxNDTR counter has reached zero in the memory-to-peripheral mode
 - The stream is disabled before the end of transfer (by clearing the EN bit in the DMA_SxCR register) and (when transfers are peripheral-to-memory or memory-

to-memory) all the remaining data have been flushed from the FIFO into the memory

- In Peripheral flow controller mode:
 - The last external burst or single request has been generated from the peripheral and (when the DMA is operating in peripheral-to-memory mode) the remaining data have been transferred from the FIFO into the memory
 - The stream is disabled by software, and (when the DMA is operating in peripheral-to-memory mode) the remaining data have been transferred from the FIFO into the memory

Note: The transfer completion is dependent on the remaining data in FIFO to be transferred into memory only in the case of peripheral-to-memory mode. This condition is not applicable in memory-to-peripheral mode.

If the stream is configured in noncircular mode, after the end of the transfer (that is when the number of data to be transferred reaches zero), the DMA is stopped (EN bit in DMA_SxCR register is cleared by Hardware) and no DMA request is served unless the software reprograms the stream and re-enables it (by setting the EN bit in the DMA_SxCR register).

9.3.14 DMA transfer suspension

At any time, a DMA transfer can be suspended to be restarted later on or to be definitively disabled before the end of the DMA transfer.

There are two cases:

- The stream disables the transfer with no later-on restart from the point where it was stopped. There is no particular action to do, except to clear the EN bit in the DMA_SxCR register to disable the stream. The stream may take time to be disabled (ongoing transfer is completed first). The transfer complete interrupt flag (TCIF in the DMA_LISR or DMA_HISR register) is set in order to indicate the end of transfer. The value of the EN bit in DMA_SxCR is now '0' to confirm the stream interruption. The DMA_SxNDTR register contains the number of remaining data items at the moment when the stream was stopped so that the software can determine how many data items have been transferred before the stream was interrupted.
- The stream suspends the transfer before the number of remaining data items to be transferred in the DMA_SxNDTR register reaches 0. The aim is to restart the transfer later by re-enabling the stream. In order to restart from the point where the transfer was stopped, the software has to read the DMA_SxNDTR register after disabling the stream by writing the EN bit in DMA_SxCR register (and then checking that it is at '0') to know the number of data items already collected. Then:
 - The peripheral and/or memory addresses have to be updated in order to adjust the address pointers
 - The SxNDTR register has to be updated with the remaining number of data items to be transferred (the value read when the stream was disabled)
 - The stream may then be re-enabled to restart the transfer from the point it was stopped

Note: Note that a Transfer complete interrupt flag (TCIF in DMA_LISR or DMA_HISR) is set to indicate the end of transfer due to the stream interruption.

9.3.15 Flow controller

The entity that controls the number of data to be transferred is known as the flow controller. This flow controller is configured independently for each stream using the PFCTRL bit in the DMA_SxCR register.

The flow controller can be:

- The DMA controller: in this case, the number of data items to be transferred is programmed by software into the DMA_SxNDTR register before the DMA stream is enabled.
- The peripheral source or destination: this is the case when the number of data items to be transferred is unknown. The peripheral indicates by hardware to the DMA controller when the last data are being transferred. This feature is only supported for peripherals which are able to signal the end of the transfer, that is:
 - SDIO

When the peripheral flow controller is used for a given stream, the value written into the DMA_SxNDTR has no effect on the DMA transfer. Actually, whatever the value written, it will be forced by hardware to 0xFFFF as soon as the stream is enabled, to respect the following schemes:

- Anticipated stream interruption: EN bit in DMA_SxCR register is reset to 0 by the software to stop the stream before the last data hardware signal (single or burst) is sent by the peripheral. In such a case, the stream is switched off and the FIFO flush is triggered in the case of a peripheral-to-memory DMA transfer. The TCIFx flag of the corresponding stream is set in the status register to indicate the DMA completion. To know the number of data items transferred during the DMA transfer, read the DMA_SxNDTR register and apply the following formula:
 - $\text{Number_of_data_transferred} = 0xFFFF - \text{DMA_SxNDTR}$
- Normal stream interruption due to the reception of a last data hardware signal: the stream is automatically interrupted when the peripheral requests the last transfer (single or burst) and when this transfer is complete. the TCIFx flag of the corresponding stream is set in the status register to indicate the DMA transfer completion. To know the number of data items transferred, read the DMA_SxNDTR register and apply the same formula as above.
- The DMA_SxNDTR register reaches 0: the TCIFx flag of the corresponding stream is set in the status register to indicate the forced DMA transfer completion. The stream is automatically switched off even though the last data hardware signal (single or burst) has not been yet asserted. The already transferred data will not be lost. This means that a maximum of 65535 data items can be managed by the DMA in a single transaction, even in peripheral flow control mode.

Note: When configured in memory-to-memory mode, the DMA is always the flow controller and the PFCTRL bit is forced to 0 by hardware.

The Circular mode is forbidden in the peripheral flow controller mode.

9.3.16 Summary of the possible DMA configurations

Table 34 summarizes the different possible DMA configurations.

Table 34. Possible DMA configurations

DMA transfer mode	Source	Destination	Flow controller	Circular mode	Transfer type	Direct mode	Double buffer mode
Peripheral-to-memory	AHB peripheral port	AHB memory port	DMA	possible	single	possible	possible
					burst	forbidden	
			Peripheral	forbidden	single	possible	forbidden
					burst	forbidden	
Memory-to-peripheral	AHB memory port	AHB peripheral port	DMA	possible	single	possible	possible
					burst	forbidden	
			Peripheral	forbidden	single	possible	forbidden
					burst	forbidden	
Memory-to-memory	AHB peripheral port	AHB memory port	DMA only	forbidden	single	forbidden	forbidden
					burst		

9.3.17 Stream configuration procedure

The following sequence should be followed to configure a DMA stream x (where x is the stream number):

1. If the stream is enabled, disable it by resetting the EN bit in the DMA_SxCR register, then read this bit in order to confirm that there is no ongoing stream operation. Writing this bit to 0 is not immediately effective since it is actually written to 0 once all the current transfers have finished. When the EN bit is read as 0, this means that the stream is ready to be configured. It is therefore necessary to wait for the EN bit to be cleared before starting any stream configuration. All the stream dedicated bits set in the status register (DMA_LISR and DMA_HISR) from the previous data block DMA transfer should be cleared before the stream can be re-enabled.
2. Set the peripheral port register address in the DMA_SxPAR register. The data will be moved from/ to this address to/ from the peripheral port after the peripheral event.
3. Set the memory address in the DMA_SxMA0R register (and in the DMA_SxMA1R register in the case of a double buffer mode). The data will be written to or read from this memory after the peripheral event.
4. Configure the total number of data items to be transferred in the DMA_SxNDTR register. After each peripheral event or each beat of the burst, this value is decremented.
5. Select the DMA channel (request) using CHSEL[2:0] in the DMA_SxCR register.
6. If the peripheral is intended to be the flow controller and if it supports this feature, set the PFCTRL bit in the DMA_SxCR register.
7. Configure the stream priority using the PL[1:0] bits in the DMA_SxCR register.
8. Configure the FIFO usage (enable or disable, threshold in transmission and reception)
9. Configure the data transfer direction, peripheral and memory incremented/fixed mode, single or burst transactions, peripheral and memory data widths, Circular mode,

Double buffer mode and interrupts after half and/or full transfer, and/or errors in the DMA_SxCR register.

10. Activate the stream by setting the EN bit in the DMA_SxCR register.

As soon as the stream is enabled, it can serve any DMA request from the peripheral connected to the stream.

Once half the data have been transferred on the AHB destination port, the half-transfer flag (HTIF) is set and an interrupt is generated if the half-transfer interrupt enable bit (HTIE) is set. At the end of the transfer, the transfer complete flag (TCIF) is set and an interrupt is generated if the transfer complete interrupt enable bit (TCIE) is set.

Warning: To switch off a peripheral connected to a DMA stream request, it is mandatory to, first, switch off the DMA stream to which the peripheral is connected, then to wait for EN bit = 0. Only then can the peripheral be safely disabled.

9.3.18 Error management

The DMA controller can detect the following errors:

- **Transfer error:** the transfer error interrupt flag (TEIFx) is set when:
 - A bus error occurs during a DMA read or a write access
 - A write access is requested by software on a memory address register in Double buffer mode whereas the stream is enabled and the current target memory is the one impacted by the write into the memory address register (refer to [Section 9.3.9: Double buffer mode](#))
- **FIFO error:** the FIFO error interrupt flag (FEIFx) is set if:
 - A FIFO underrun condition is detected
 - A FIFO overrun condition is detected (no detection in memory-to-memory mode because requests and transfers are internally managed by the DMA)
 - The stream is enabled while the FIFO threshold level is not compatible with the size of the memory burst (refer to [Table 33: FIFO threshold configurations](#))
- **Direct mode error:** the direct mode error interrupt flag (DMEIFx) can only be set in the peripheral-to-memory mode while operating in direct mode and when the MINC bit in the DMA_SxCR register is cleared. This flag is set when a DMA request occurs while the previous data have not yet been fully transferred into the memory (because the memory bus was not granted). In this case, the flag indicates that 2 data items were be transferred successively to the same destination address, which could be an issue if the destination is not able to manage this situation

In direct mode, the FIFO error flag can also be set under the following conditions:

- In the peripheral-to-memory mode, the FIFO can be saturated (overrun) if the memory bus is not granted for several peripheral requests
- In the memory-to-peripheral mode, an underrun condition may occur if the memory bus has not been granted before a peripheral request occurs

If the TEIFx or the FEIFx flag is set due to incompatibility between burst size and FIFO threshold level, the faulty stream is automatically disabled through a hardware clear of its EN bit in the corresponding stream configuration register (DMA_SxCR).

If the DMEIFx or the FEIFx flag is set due to an overrun or underrun condition, the faulty stream is not automatically disabled and it is up to the software to disable or not the stream by resetting the EN bit in the DMA_SxCR register. This is because there is no data loss when this kind of errors occur.

When the stream's error interrupt flag (TEIF, FEIF, DMEIF) in the DMA_LISR or DMA_HISR register is set, an interrupt is generated if the corresponding interrupt enable bit (TEIE, FEIE, DMIE) in the DMA_SxCR or DMA_SxFCR register is set.

Note: When a FIFO overrun or underrun condition occurs, the data are not lost because the peripheral request is not acknowledged by the stream until the overrun or underrun condition is cleared. If this acknowledge takes too much time, the peripheral itself may detect an overrun or underrun condition of its internal buffer and data might be lost.

9.4 DMA interrupts

For each DMA stream, an interrupt can be produced on the following events:

- Half-transfer reached
- Transfer complete
- Transfer error
- Fifo error (overrun, underrun or FIFO level error)
- Direct mode error

Separate interrupt enable control bits are available for flexibility as shown in [Table 35](#).

Table 35. DMA interrupt requests

Interrupt event	Event flag	Enable control bit
Half-transfer	HTIF	HTIE
Transfer complete	TCIF	TCIE
Transfer error	TEIF	TEIE
FIFO overrun/underrun	FEIF	FEIE
Direct mode error	DMEIF	DMEIE

Note: Before setting an Enable control bit to '1', the corresponding event flag should be cleared, otherwise an interrupt is immediately generated.

9.5 DMA registers

The DMA registers have to be accessed by words (32 bits).

9.5.1 DMA low interrupt status register (DMA_LISR)

Address offset: 0x00

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF3	HTIF3	TEIF3	DMEIF3	Reserv ed	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Reserv ed	FEIF2
r	r	r	r	r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF1	HTIF1	TEIF1	DMEIF1	Reserv ed	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Reserv ed	FEIF0
r	r	r	r	r	r	r	r		r	r	r	r	r		r

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **TCIFx**: Stream x transfer complete interrupt flag (x = 3..0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_LIFCR register.

0: No transfer complete event on stream x

1: A transfer complete event occurred on stream x

Bits 26, 20, 10, 4 **HTIFx**: Stream x half transfer interrupt flag (x=3..0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_LIFCR register.

0: No half transfer event on stream x

1: A half transfer event occurred on stream x

Bits 25, 19, 9, 3 **TEIFx**: Stream x transfer error interrupt flag (x=3..0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_LIFCR register.

0: No transfer error on stream x

1: A transfer error occurred on stream x

Bits 24, 18, 8, 2 **DMEIFx**: Stream x direct mode error interrupt flag (x=3..0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_LIFCR register.

0: No Direct Mode Error on stream x

1: A Direct Mode Error occurred on stream x

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **FEIFx**: Stream x FIFO error interrupt flag (x=3..0)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_LIFCR register.

0: No FIFO Error event on stream x

1: A FIFO Error event occurred on stream x

9.5.2 DMA high interrupt status register (DMA_HISR)

Address offset: 0x04

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				TCIF7	HTIF7	TEIF7	DMEIF7	Reserved	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Reserved	FEIF6
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				TCIF5	HTIF5	TEIF5	DMEIF5	Reserved	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Reserved	FEIF4
				r	r	r	r		r	r	r	r	r		r

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **TCIFx**: Stream x transfer complete interrupt flag (x=7..4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_HIFCR register.

0: No transfer complete event on stream x

1: A transfer complete event occurred on stream x

Bits 26, 20, 10, 4 **HTIFx**: Stream x half transfer interrupt flag (x=7..4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_HIFCR register.

0: No half transfer event on stream x

1: A half transfer event occurred on stream x

Bits 25, 19, 9, 3 **TEIFx**: Stream x transfer error interrupt flag (x=7..4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_HIFCR register.

0: No transfer error on stream x

1: A transfer error occurred on stream x

Bits 24, 18, 8, 2 **DMEIFx**: Stream x direct mode error interrupt flag (x=7..4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_HIFCR register.

0: No Direct mode error on stream x

1: A Direct mode error occurred on stream x

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **FEIFx**: Stream x FIFO error interrupt flag (x=7..4)

This bit is set by hardware. It is cleared by software writing 1 to the corresponding bit in the DMA_HIFCR register.

0: No FIFO error event on stream x

1: A FIFO error event occurred on stream x

9.5.3 DMA low interrupt flag clear register (DMA_LIFCR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Reserved	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Reserved	CFEIF2
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Reserved	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Reserved	CFEIF0
				w	w	w	w		w	w	w	w	w		w

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **CTCIFx**: Stream x clear transfer complete interrupt flag (x = 3..0)

Writing 1 to this bit clears the corresponding TCIFx flag in the DMA_LISR register

Bits 26, 20, 10, 4 **CHTIFx**: Stream x clear half transfer interrupt flag (x = 3..0)

Writing 1 to this bit clears the corresponding HTIFx flag in the DMA_LISR register

Bits 25, 19, 9, 3 **CTEIFx**: Stream x clear transfer error interrupt flag (x = 3..0)

Writing 1 to this bit clears the corresponding TEIFx flag in the DMA_LISR register

Bits 24, 18, 8, 2 **CDMEIFx**: Stream x clear direct mode error interrupt flag (x = 3..0)

Writing 1 to this bit clears the corresponding DMEIFx flag in the DMA_LISR register

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **CFEIFx**: Stream x clear FIFO error interrupt flag (x = 3..0)

Writing 1 to this bit clears the corresponding CFEIFx flag in the DMA_LISR register

9.5.4 DMA high interrupt flag clear register (DMA_HIFCR)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Reserved	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Reserved	CFEIF6
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Reserved	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Reserved	CFEIF4
				w	w	w	w		w	w	w	w	w		w

Bits 31:28, 15:12 Reserved, must be kept at reset value.

Bits 27, 21, 11, 5 **CTCIFx**: Stream x clear transfer complete interrupt flag (x = 7..4)

Writing 1 to this bit clears the corresponding TCIFx flag in the DMA_HISR register

Bits 26, 20, 10, 4 **CHTIFx**: Stream x clear half transfer interrupt flag (x = 7..4)

Writing 1 to this bit clears the corresponding HTIFx flag in the DMA_HISR register

Bits 25, 19, 9, 3 **CTEIFx**: Stream x clear transfer error interrupt flag (x = 7..4)

Writing 1 to this bit clears the corresponding TEIFx flag in the DMA_HISR register

Bits 24, 18, 8, 2 **CDMEIFx**: Stream x clear direct mode error interrupt flag (x = 7..4)

Writing 1 to this bit clears the corresponding DMEIFx flag in the DMA_HISR register

Bits 23, 17, 7, 1 Reserved, must be kept at reset value.

Bits 22, 16, 6, 0 **CFEIFx**: Stream x clear FIFO error interrupt flag (x = 7..4)

Writing 1 to this bit clears the corresponding CFEIFx flag in the DMA_HISR register

9.5.5 DMA stream x configuration register (DMA_SxCR) (x = 0..7)

This register is used to configure the concerned stream.

Address offset: $0x10 + 0x18 \times \text{stream number}$

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				CHSEL[2:0]			MBURST [1:0]		PBURST[1:0]		Reserved	CT	DBM	PL[1:0]	
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:25 **CHSEL[2:0]**: Channel selection

These bits are set and cleared by software.

000: channel 0 selected

001: channel 1 selected

010: channel 2 selected

011: channel 3 selected

100: channel 4 selected

101: channel 5 selected

110: channel 6 selected

111: channel 7 selected

These bits are protected and can be written only if EN is '0'

Bits 24:23 **MBURST**: Memory burst transfer configuration

These bits are set and cleared by software.

00: single transfer

01: INCR4 (incremental burst of 4 beats)

10: INCR8 (incremental burst of 8 beats)

11: INCR16 (incremental burst of 16 beats)

These bits are protected and can be written only if EN is '0'

In direct mode, these bits are forced to 0x0 by hardware as soon as bit EN= '1'.

Bits 22:21 **PBURST[1:0]**: Peripheral burst transfer configuration

These bits are set and cleared by software.

00: single transfer

01: INCR4 (incremental burst of 4 beats)

10: INCR8 (incremental burst of 8 beats)

11: INCR16 (incremental burst of 16 beats)

These bits are protected and can be written only if EN is '0'

In direct mode, these bits are forced to 0x0 by hardware.

Bit 20 Reserved, must be kept at reset value.

Bit 19 **CT**: Current target (only in double buffer mode)

This bit is set and cleared by hardware. It can also be written by software.

0: The current target memory is Memory 0 (addressed by the DMA_SxM0AR pointer)

1: The current target memory is Memory 1 (addressed by the DMA_SxM1AR pointer)

This bit can be written only if EN is '0' to indicate the target memory area of the first transfer.

Once the stream is enabled, this bit operates as a status flag indicating which memory area is the current target.

Bit 18 **DBM**: Double buffer mode

This bit is set and cleared by software.

0: No buffer switching at the end of transfer

1: Memory target switched at the end of the DMA transfer

This bit is protected and can be written only if EN is '0'.

Bits 17:16 **PL[1:0]**: Priority level

These bits are set and cleared by software.

00: Low

01: Medium

10: High

11: Very high

These bits are protected and can be written only if EN is '0'.

Bit 15 **PINCOS**: Peripheral increment offset size

This bit is set and cleared by software

0: The offset size for the peripheral address calculation is linked to the PSIZE

1: The offset size for the peripheral address calculation is fixed to 4 (32-bit alignment).

This bit has no meaning if bit PINC = '0'.

This bit is protected and can be written only if EN = '0'.

This bit is forced low by hardware when the stream is enabled (bit EN = '1') if the direct mode is selected or if PBURST are different from "00".

Bits 14:13 **MSIZE[1:0]**: Memory data size

These bits are set and cleared by software.

00: byte (8-bit)

01: half-word (16-bit)

10: word (32-bit)

11: reserved

These bits are protected and can be written only if EN is '0'.

In direct mode, MSIZE is forced by hardware to the same value as PSIZE as soon as bit EN = '1'.

Bits 12:11 **PSIZE[1:0]**: Peripheral data size

These bits are set and cleared by software.

00: Byte (8-bit)

01: Half-word (16-bit)

10: Word (32-bit)

11: reserved

These bits are protected and can be written only if EN is '0'.

Bit 10 **MINC**: Memory increment mode

This bit is set and cleared by software.

0: Memory address pointer is fixed

1: Memory address pointer is incremented after each data transfer (increment is done according to MSIZE)

This bit is protected and can be written only if EN is '0'.

Bit 9 PINC: Peripheral increment mode

This bit is set and cleared by software.

0: Peripheral address pointer is fixed

1: Peripheral address pointer is incremented after each data transfer (increment is done according to PSIZE)

This bit is protected and can be written only if EN is '0'.

Bit 8 CIRC: Circular mode

This bit is set and cleared by software and can be cleared by hardware.

0: Circular mode disabled

1: Circular mode enabled

When the peripheral is the flow controller (bit PFCTRL=1) and the stream is enabled (bit EN=1), then this bit is automatically forced by hardware to 0.

It is automatically forced by hardware to 1 if the DBM bit is set, as soon as the stream is enabled (bit EN = '1').

Bits 7:6 DIR[1:0]: Data transfer direction

These bits are set and cleared by software.

00: Peripheral-to-memory

01: Memory-to-peripheral

10: Memory-to-memory

11: reserved

These bits are protected and can be written only if EN is '0'.

Bit 5 PFCTRL: Peripheral flow controller

This bit is set and cleared by software.

0: The DMA is the flow controller

1: The peripheral is the flow controller

This bit is protected and can be written only if EN is '0'.

When the memory-to-memory mode is selected (bits DIR[1:0]=10), then this bit is automatically forced to 0 by hardware.

Bit 4 TCIE: Transfer complete interrupt enable

This bit is set and cleared by software.

0: TC interrupt disabled

1: TC interrupt enabled

Bit 3 HTIE: Half transfer interrupt enable

This bit is set and cleared by software.

0: HT interrupt disabled

1: HT interrupt enabled

Bit 2 TEIE: Transfer error interrupt enable

This bit is set and cleared by software.

0: TE interrupt disabled

1: TE interrupt enabled

Bit 1 DMEIE: Direct mode error interrupt enable

This bit is set and cleared by software.

0: DME interrupt disabled

1: DME interrupt enabled

Bit 0 **EN**: Stream enable / flag stream ready when read low

This bit is set and cleared by software.

0: Stream disabled

1: Stream enabled

This bit may be cleared by hardware:

- on a DMA end of transfer (stream ready to be configured)
- if a transfer error occurs on the AHB master buses
- when the FIFO threshold on memory AHB port is not compatible with the size of the burst

When this bit is read as 0, the software is allowed to program the Configuration and FIFO bits registers. It is forbidden to write these registers when the EN bit is read as 1.

Note: Before setting EN bit to '1' to start a new transfer, the event flags corresponding to the stream in DMA_LISR or DMA_HISR register must be cleared.

9.5.6 DMA stream x number of data register (DMA_SxNDTR) (x = 0..7)

Address offset: $0x14 + 0x18 \times \text{stream number}$

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **NDT[15:0]**: Number of data items to transfer

Number of data items to be transferred (0 up to 65535). This register can be written only when the stream is disabled. When the stream is enabled, this register is read-only, indicating the remaining data items to be transmitted. This register decrements after each DMA transfer.

Once the transfer has completed, this register can either stay at zero (when the stream is in normal mode) or be reloaded automatically with the previously programmed value in the following cases:

- when the stream is configured in Circular mode.
- when the stream is enabled again by setting EN bit to '1'

If the value of this register is zero, no transaction can be served even if the stream is enabled.

9.5.7 DMA stream x peripheral address register (DMA_SxPAR) (x = 0..7)

Address offset: $0x18 + 0x18 \times \text{stream number}$

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PAR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PAR[31:0]**: Peripheral address

Base address of the peripheral data register from/to which the data will be read/written.

These bits are write-protected and can be written only when bit EN = '0' in the DMA_SxCR register.

9.5.8 DMA stream x memory 0 address register (DMA_SxM0AR) (x = 0..7)

Address offset: $0x1C + 0x18 \times \text{stream number}$

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M0A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M0A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **M0A[31:0]**: Memory 0 address

Base address of Memory area 0 from/to which the data will be read/written.

These bits are write-protected. They can be written only if:

- the stream is disabled (bit EN = '0' in the DMA_SxCR register) or
- the stream is enabled (EN = '1' in DMA_SxCR register) and bit CT = '1' in the DMA_SxCR register (in Double buffer mode).

9.5.9 DMA stream x memory 1 address register (DMA_SxM1AR) (x = 0..7)

Address offset: $0x20 + 0x18 \times \text{stream number}$

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M1A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M1A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **M1A[31:0]**: Memory 1 address (used in case of Double buffer mode)

Base address of Memory area 1 from/to which the data will be read/written.

This register is used only for the Double buffer mode.

These bits are write-protected. They can be written only if:

- the stream is disabled (bit EN= '0' in the DMA_SxCR register) or
- the stream is enabled (EN='1' in DMA_SxCR register) and bit CT = '0' in the DMA_SxCR register.

9.5.10 DMA stream x FIFO control register (DMA_SxFCR) (x = 0..7)

Address offset: $0x24 + 0x24 \times \text{stream number}$

Reset value: 0x0000 0021

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved								FEIE	Reserved	FS[2:0]			DMDIS	FTH[1:0]	
								rw		r	r	r	rw	rw	rw

Bits 31:8 Reserved, must be kept at reset value.

Bit 7 **FEIE**: FIFO error interrupt enable

This bit is set and cleared by software.

0: FE interrupt disabled

1: FE interrupt enabled

Bit 6 Reserved, must be kept at reset value.

Bits 5:3 **FS[2:0]**: FIFO status

These bits are read-only.

000: $0 < \text{fifo_level} < 1/4$

001: $1/4 \leq \text{fifo_level} < 1/2$

010: $1/2 \leq \text{fifo_level} < 3/4$

011: $3/4 \leq \text{fifo_level} < \text{full}$

100: FIFO is empty

101: FIFO is full

others: no meaning

These bits are not relevant in the direct mode (DMDIS bit is zero).

Bit 2 **DMDIS**: Direct mode disable

This bit is set and cleared by software. It can be set by hardware.

0: Direct mode enabled

1: Direct mode disabled

This bit is protected and can be written only if EN is '0'.

This bit is set by hardware if the memory-to-memory mode is selected (DIR bit in DMA_SxCR are "10") and the EN bit in the DMA_SxCR register is '1' because the direct mode is not allowed in the memory-to-memory configuration.

Bits 1:0 **FTH[1:0]**: FIFO threshold selection

These bits are set and cleared by software.

00: 1/4 full FIFO

01: 1/2 full FIFO

10: 3/4 full FIFO

11: full FIFO

These bits are not used in the direct mode when the DMIS value is zero.

These bits are protected and can be written only if EN is '0'.

9.5.11 DMA register map

Table 36 summarizes the DMA registers.

Table 36. DMA register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0000	DMA_LISR	Reserved				TCIF3	HTIF3	TEIF3	DMEIF3	Reserved	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Reserved	FEIF2	Reserved					TCIF1	HTIF1	TEIF1	DMEIF1	Reserved	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Reserved	FEIF0
	Reset value					0	0	0	0		0	0	0	0	0		0						0	0	0	0		0	0	0	0	0		0
0x0004	DMA_HISR	Reserved				TCIF7	HTIF7	TEIF7	DMEIF7	Reserved	FEIF7	TCIF6	HTIF6	TEIF6	DMEIF6	Reserved	FEIF6	Reserved					TCIF5	HTIF5	TEIF5	DMEIF5	Reserved	FEIF5	TCIF4	HTIF4	TEIF4	DMEIF4	Reserved	FEIF4
	Reset value					0	0	0	0		0	0	0	0	0		0						0	0	0	0		0	0	0	0	0		0
0x0008	DMA_LIFCR	Reserved				CTCIF3	CHTIF3	TEIF3	CDMEIF3	Reserved	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Reserved	CFEIF2	Reserved					CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Reserved	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Reserved	CFEIF0
	Reset value					0	0	0	0		0	0	0	0	0		0						0	0	0	0		0	0	0	0	0		0
0x000C	DMA_HIFCR	Reserved				CTCIF7	CHTIF7	CTEIF7	CDMEIF7	Reserved	CFEIF7	CTCIF6	CHTIF6	CTEIF6	CDMEIF6	Reserved	CFEIF6	Reserved					CTCIF5	CHTIF5	CTEIF5	CDMEIF5	Reserved	CFEIF5	CTCIF4	CHTIF4	CTEIF4	CDMEIF4	Reserved	CFEIF4
	Reset value					0	0	0	0		0	0	0	0	0		0						0	0	0	0		0	0	0	0	0		0
0x0010	DMA_S0CR	Reserved				CHSEL[2:0]		MBURST[1:0]		PBURST[1:0]		Reserved	CT	DBM	PL[1:0]	PINCOS	MSIZE[1:0]		PSIZE[1:0]		MNC	PINC	CIRC	DIR[1:0]		PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN			
	0																															0	0	0
0x0014	DMA_S0NDTR	Reserved															NDT[15:]																	
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0018	DMA_S0PAR	PA[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x001C	DMA_S0M0AR	M0A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0020	DMA_S0M1AR	M1A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0024	DMA_S0FCR	Reserved																								FEIE	Reserved	FS[2:0]		DMDIS		FTH[1:0]		
	0																									1							0	0
0x0028	DMA_S1CR	Reserved				CHSEL[2:0]		MBURST[1:0]		PBURST[1:0]		Reserved	CT	DBM	PL[1:0]	PINCOS	MSIZE[1:0]		PSIZE[1:0]		MNC	PINC	CIRC	DIR[1:0]		PFCCTRL	TCIE	HTIE	TEIE	DMEIE	EN			
	0																															0	0	0
0x002C	DMA_S1NDTR	Reserved															NDT[15:]																	
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 36. DMA register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x0030	DMA_S1PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0034	DMA_S1M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0038	DMA_S1M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x003C	DMA_S1FCR	Reserved																								FEIE	Reserved	FS[2:0]		DMDIS		FTH [1:0]	
	Reset value																									0		1	0	0	0	0	0
0x0040	DMA_S2CR	Reserved		CHSEL [2:0]		MBURST[1:0]		PBURST[1:0]		Reserved	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR [1:0]		PFCTRL		TCIE	HTIE	TEIE	DMEIE	EN		
	Reset value			0		0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0044	DMA_S2NDTR	Reserved															NDT[15:.]																
	Reset value																0																
0x0048	DMA_S2PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x004C	DMA_S2M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0050	DMA_S2M1AR	M1A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0054	DMA_S2FCR	Reserved																								FEIE	Reserved	FS[2:0]		DMDIS		FTH [1:0]	
	Reset value																									0		1	0	0	0	0	0
0x0058	DMA_S3CR	Reserved		CHSEL[2:0]		MBURST[1:0]		PBURST[1:0]		Reserved	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL		TCIE	HTIE	TEIE	DMEIE	EN		
	Reset value			0		0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x005C	DMA_S3NDTR	Reserved															NDT[15:.]																
	Reset value																0																
0x0060	DMA_S3PAR	PA[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0064	DMA_S3M0AR	M0A[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 36. DMA register map and reset values (continued)

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0068	DMA_S3M1AR	M1A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x006C	DMA_S3FCR	Reserved																								FEIE	Reserved	FS[2:0]			DMDIS		FTH [1:0]	
	Reset value																									0		1	0	0	0	0	0	1
0x0070	DMA_S4CR	Reserved				CHSEL[2:0]		MBURST[1:0]			PBURST[1:0]			Reserved	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR [1:0]		PFCtrl	TCIE	HTIE	TEIE	DMEIE	EN
	Reset value					0		0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0074	DMA_S4NDTR	Reserved															NDT[15:]																	
	Reset value																0															0	0	0
0x0078	DMA_S4PAR	PA[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x007C	DMA_S4M0AR	M0A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0080	DMA_S4M1AR	M1A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0084	DMA_S4FCR	Reserved																								FEIE	Reserved	FS[2:0]			DMDIS		FTH [1:0]	
	Reset value																									0		1	0	0	0	0	0	0
0x0088	DMA_S5CR	Reserved				CHSEL[2:0]		MBURST[1:0]			PBURST[1:0]			Reserved	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCtrl	TCIE	HTIE	TEIE	DMEIE	EN
	Reset value					0		0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008C	DMA_S5NDTR	Reserved															NDT[15:]																	
	Reset value																0															0	0	0
0x0090	DMA_S5PAR	PA[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0094	DMA_S5M0AR	M0A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0098	DMA_S5M1AR	M1A[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x009C	DMA_S5FCR	Reserved																								FEIE	Reserved	FS[2:0]			DMDIS		FTH [1:0]	
	Reset value																									0		1	0	0	0	0	0	0
0x00A0	DMA_S6CR	Reserved				CHSEL[2:0]		MBURST[1:0]			PBURST[1:0]			Reserved	CT	DBM	PL[1:0]		PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCtrl	TCIE	HTIE	TEIE	DMEIE	EN
	Reset value					0		0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00A4	DMA_S6NDTR	Reserved															NDT[15:]																	
	Reset value																0															0	0	0
0x00A8	DMA_S6PAR	PA[31:0]																																
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0